基于 RISC-V CPU 与 NPU 的异构视觉处理系统

摘 要

目 录

摘 要 **I**

第一部分 作品概述 **1**
1.1 功能与特性 . . . 1
1.2 应用领域 . . . 2
1.3 主要技术特点 . . . 2
1.4 设计流程 . . . 2

第二部分 系统组成及功能说明 **5**
2.1 系统整体介绍 . . . 5
2.1.1 系统架构 . . . 5
2.1.2 模块功能 . . . 6
2.2 CPU 设计 . . . 7
2.2.1 CPU 微架构 . . . 7
2.2.2 指令提取 . . . 9
2.2.3 指令译码 . . . 13
2.2.4 指令执行 . . . 15
2.2.5 存储访问 . . . 17
2.2.6 指令写回 . . . 18
2.2.7 流水线控制 . . . 18
2.2.8 存储系统 . . . 19
2.2.9 中断和异常 . . . 21
2.3 NPU 设计 . . . 22
2.3.1 NPU 微架构 . . . 22
2.3.2 PE 计算单元 . . . 22
2.3.3 卷积计算通路 . . . 22
2.3.4 算子控制器 . . . 23
2.3.5 片上缓冲 . . . 23
2.3.6 DMA 控制器 . . . 23
2.3.7 卷积窗口生成 . . . 23
2.3.8 辅助算子单元 . . . 24
2.3.9 定点后处理单元 . . . 24
2.3.10 寄存器接口 . . . 24
2.4 系统互连与存储 . . . 24
2.4.1 总线结构 . . . 24
2.4.2 存储组织 . . . 25
2.4.3 访问仲裁 . . . 25
2.4.4 共享数据访问约定 . . . 26
2.5 图像采集与显示链路 . . . 26
2.5.1 图像采集 . . . 27
2.5.2 输入预处理 . . . 27
2.5.3 帧缓存管理 . . . 28
2.5.4 头像叠加 . . . 29

2.5.5 显示输出 29
2.6 模型部署与应用软件 30
2.6.1 模型选型 30
2.6.2 量化方案 30
2.6.3 模型导出 31
2.6.4 算子映射 31
2.6.5 检测后处理 31
2.6.6 裸机运行时 32
2.6.7 帧调度 32
2.6.8 错误恢复 33
2.7 FPGA 实现 33
2.7.1 目标平台 33
2.7.2 工程配置 34
2.7.3 时钟复位 34
2.7.4 板级接口 34
2.7.5 实现约束 35

第三部分 完成情况及性能参数 **36**
3.1 整体介绍 36
3.2 工程成果 36
3.2.1 硬件成果 36
3.2.2 软件成果 36
3.2.3 功能验证 36
3.2.4 应用演示 36
3.3 特性成果 37
3.3.1 CPU 性能 37
3.3.2 CPU 流水线效率 41
3.3.3 NPU 性能 44
3.3.4 系统性能 44
3.3.5 FPGA 实现指标 44

第四部分 总 结 **45**
4.1 可扩展之处 45
4.2 工程取舍与心得体会 45

第五部分 参考文献 **46**

附 录 **47**
A 仿真环境与验证方法 47
A.1 工具配置 47
A.2 测试平台 47
A.3 判定标准 47
B CPU 仿真验证 47
B.1 指令集验证 47
B.2 单元级仿真验证 50
B.3 模块级仿真验证 54
B.4 系统级仿真验证 58
C NPU 仿真验证 62

C.1 计算正确性 62
C.2 分块计算 62
C.3 DMA 传输 62
C.4 任务控制 62
C.5 辅助算子 62
D 系统互连与存储仿真验证 62
D.1 CPU 主端控制与外设访问 63
D.2 共享存储与地址译码 64
E 图像采集与显示链路仿真验证 65
E.1 四类系统级场景 65
E.2 采集与槽所有权 66
E.3 共享 DDR 背压与采显联合 66
F 系统联合仿真验证 67
F.1 完整模型推理 67
F.2 连续帧处理 67
F.3 运行配置 67
F.4 结果核对 67

第一部分 作品概述

1.1 功能与特性

TriHaRiscv[1]是基于自研 RISC-V CPU 与 NPU 的异构视觉处理系统。设计以 CPU 实现、国产 FPGA 集成及性能分析为主线，通过人脸检测和预设人物头像叠加展示处理器、计算加速和视频处理的协同工作。系统在盘古 100pro+ 平台上集成 CPU、NPU、AXI 互连、存储及 HDMI 视频链路，具备以下功能与特性：

- CPU 采用六级流水线、单发射、顺序执行与顺序提交结构，支持 RV32IM 指令集、CSR 访问及 `FENCE.I` 指令。
- CPU 运行于机器模式，支持异常处理、中断响应与返回，提供周期计数；仿真通过 WB 提交观测与流水线事件分析执行效率。
- I-Cache 与 D-Cache 各为 16 KiB，采用两路组相联、32 B 缓存行；D-Cache 采用写回、写分配策略，支持缓存行清理与失效。
- 动态分支预测采用 512 项两位计数器 BHT、512 项直接映射 BTB 和 4 项返回地址栈；64 bit 取指包与两项取指 FIFO 为单发射流水线供给指令。
- NPU 采用 INT8 乘法、INT32 累加，提供 64 和 128 乘加通路配置，支持普通卷积、逐点卷积、深度卷积及网络所需辅助算子。
- CPU 按算子配置 NPU，NPU 在内部完成分块循环；DMA 搬运权重、特征图和计算结果，完成通知与输出写响应保持一致。
- 采用 32 bit AXI4-Lite 控制接口和 128 bit AXI4 数据 DMA 接口，通过桥接连接 CPU 访存接口与 DDR 控制器。
- 集成 Boot ROM、片上 SRAM 及 1 GB 板载 DDR3，划分 CPU 私有可缓存区和 DMA 不可缓存共享区。
- 集成 UART、GPIO、定时器、SPI、I^2C 和外部中断控制，支持程序装载、板级初始化及运行状态记录。
- HDMI 输入与输出均为 800×600、标称 60 Hz，检测覆盖完整输入画面，显示端按帧切换 RAW、BOX 和 AVATAR 全屏模式。
- 视频硬件生成 400×300 RGB888 预览图和 320×320 UINT8 BGR NCHW 输入，完成双线性缩放、补边、行缓冲、头像映射及 Alpha 混合，CPU 提交帧地址及显示参数。
- 裸机运行 YuNet 人脸检测，CPU 完成定点解码、NMS 和主人脸选择，三帧槽管理采集、处理和显示的数据生命周期。

[1]TriHaRiscv 的命名灵感来源于团队名称“哈哈哈”（即 Triple Ha）。

1.2 应用领域

系统面向交互式视觉终端、数字镜面和嵌入式视觉等场景。输入视频经 YuNet 检测后，显示端在完整原图上叠加人脸框、关键点或主人脸对应的预设动漫头像。用户通过 MODE 按键切换原图、检测标记和头像叠加三种全屏模式，切换在有效图像起点生效。主人脸按有效框面积、置信度及固定候选顺序选择，头像由软件配置，叠加采用二维缩放、平移与透明混合。

模型准备和素材转换在主机离线完成。程序、权重和素材装载后，FPGA 独立执行图像处理、网络推理、检测后处理和视频输出。同一硬件平台还用于 CPU 指令集验证、Cache 与分支预测分析，以及 CPU、NPU 和视频 DMA 并发访问 DDR 时的系统性能研究。

1.3 主要技术特点

- 六级单发射 CPU 将取指请求与指令返回分开，通过旁路、停顿和冲刷处理相关性与控制转移，配合 Cache 和动态分支预测减少执行等待。
- CPU、NPU 和 Video 按计算粒度分工：CPU 执行控制与有限候选后处理，NPU 执行卷积和张量运算，Video 执行逐像素预处理及显示合成。
- 普通卷积与深度卷积采用不同的数据组织方式，结合局部缓冲、跨通道块部分和保存和 DMA 搬运，提高乘加通路的有效利用率。
- 原图、预览图、模型输入及结果按帧绑定，显示地址和叠加参数在有效图像边界整体切换，旧槽在读事务、行缓冲及像素引用退休并完成释放确认后复用。
- 显示端按行读取原分辨率底图，在像素流水中完成框、关键点及头像合成，合成结果直接输出，省去整帧叠加图的 DDR 写回。

1.4 设计流程

本设计以自研 RISC-V CPU 的实现、国产 FPGA 验证及性能分析为目标，以 YuNet 人脸检测和单 HDMI 全屏头像叠加作为系统应用。开发分为 CPU、NPU 和 Video 三条并行路线，分别完成独立验证与板级集成，再通过统一的地址、寄存器和帧接口连接，整体流程如图1-1所示。

1. CPU 路线完成六级单发射流水线、指令集、Cache、分支预测及异常中断设计，集成 SoC 互连与外设，实现定点检测后处理和帧调度。CoreMark、IPC 及资源和时序分析贯穿各个可运行版本。
2. NPU 路线围绕固定 YuNet 计算图完成算子审计、量化和整数参考，设计 PE、普通与深度卷积通路、片上缓冲及 DMA，配套辅助算子和逐层数值核对，形成软件可调度的计算硬件与模型包。
3. Video 路线完成 HDMI 接收与发送、全帧采集、预览降采样及模型输入双线性缩

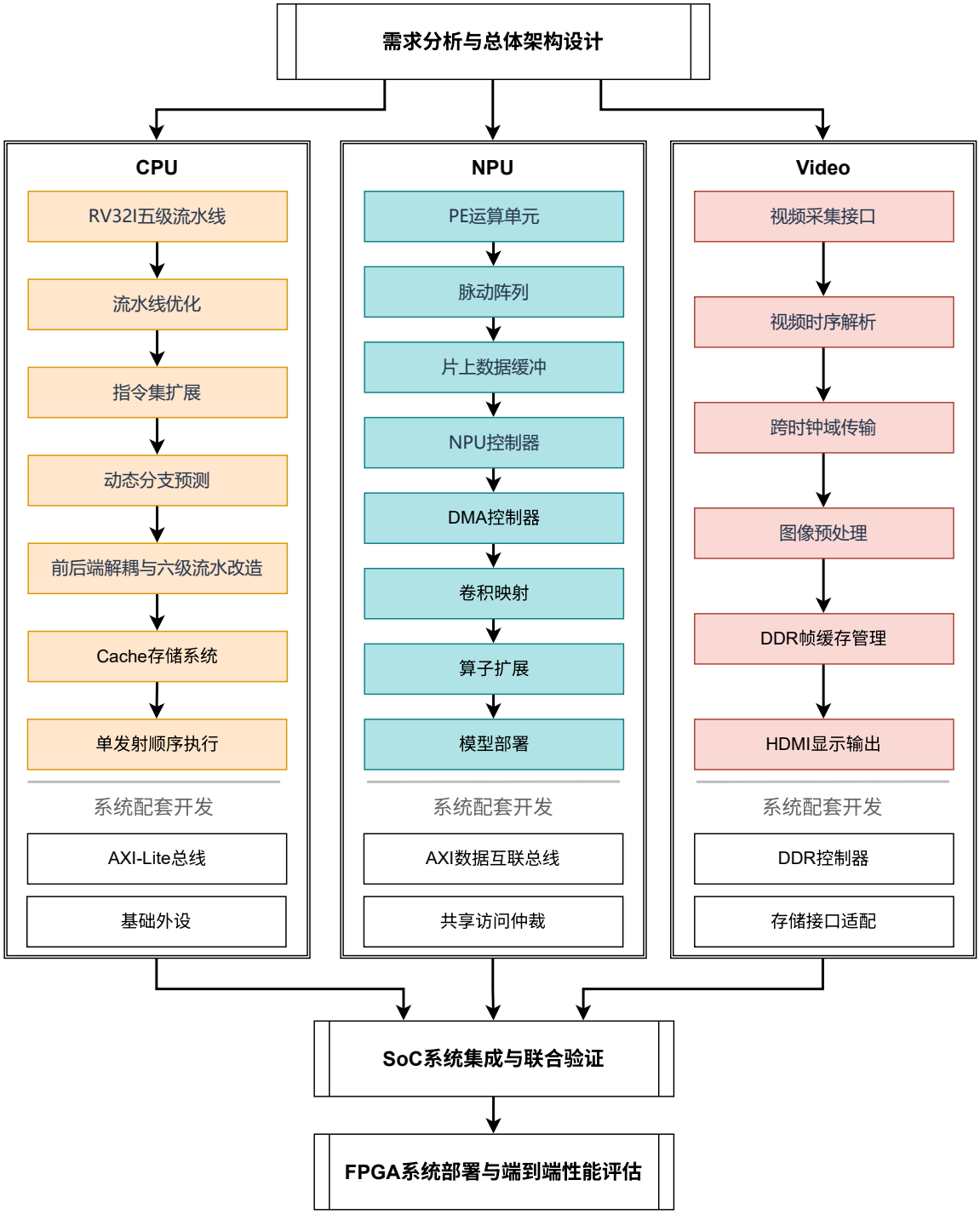


图 1-1 TriHaRiscv 设计流程

放，设计三帧槽、输入格式化、显示行缓冲和头像叠加，通过显示记录提交、实际像素生效及旧帧释放确认连接软件调度和连续视频扫描。

系统配套开发统一了 DDR 封装、AXI 互连、时钟复位和公共配置。CPU 负责互连和软件集成，Video 负责 DDR 与像素域封装，NPU 提供计算接口、整数语义和模型数据。各路线通过 CPU 与 NPU 静态推理、CPU 与 Video 控制、预处理与整数参考核对完成子系统联合验证。

FPGA 部署将处理器、计算加速和视频链路集成到 PG2L100H-6IFBG676，完成时钟、引脚和接口约束配置。CPU 性能采用 CoreMark、CoreMark/MHz、CoreMark/LUT 和 IPC 统计；FPGA 实现分析最高工作频率（Fmax）与 LUT、FF、DRM 和 APM 资源占用。应用端到端性能评估统计有效输出帧率、板内可见延迟、更新间隔及丢帧，结合逐层计算、DDR 等待和 CPU 后处理定位开销。

第二部分 系统组成及功能说明

2.1 系统整体介绍

2.1.1 系统架构

系统由 TriHaRiscv SoC 与外部视频子系统组成，整体架构如图2-1所示。SoC 以六级单发射 RISC-V CPU 为控制核心，集成 NPU、存储互连和基础外设；视频子系统在 FPGA 顶层与 SoC 并列连接，完成图像采集、输入预处理和同帧全屏显示。CPU 执行裸机程序，按算子配置 NPU，处理检测输出并发布显示参数；NPU 完成 YuNet 的卷积及辅助运算。

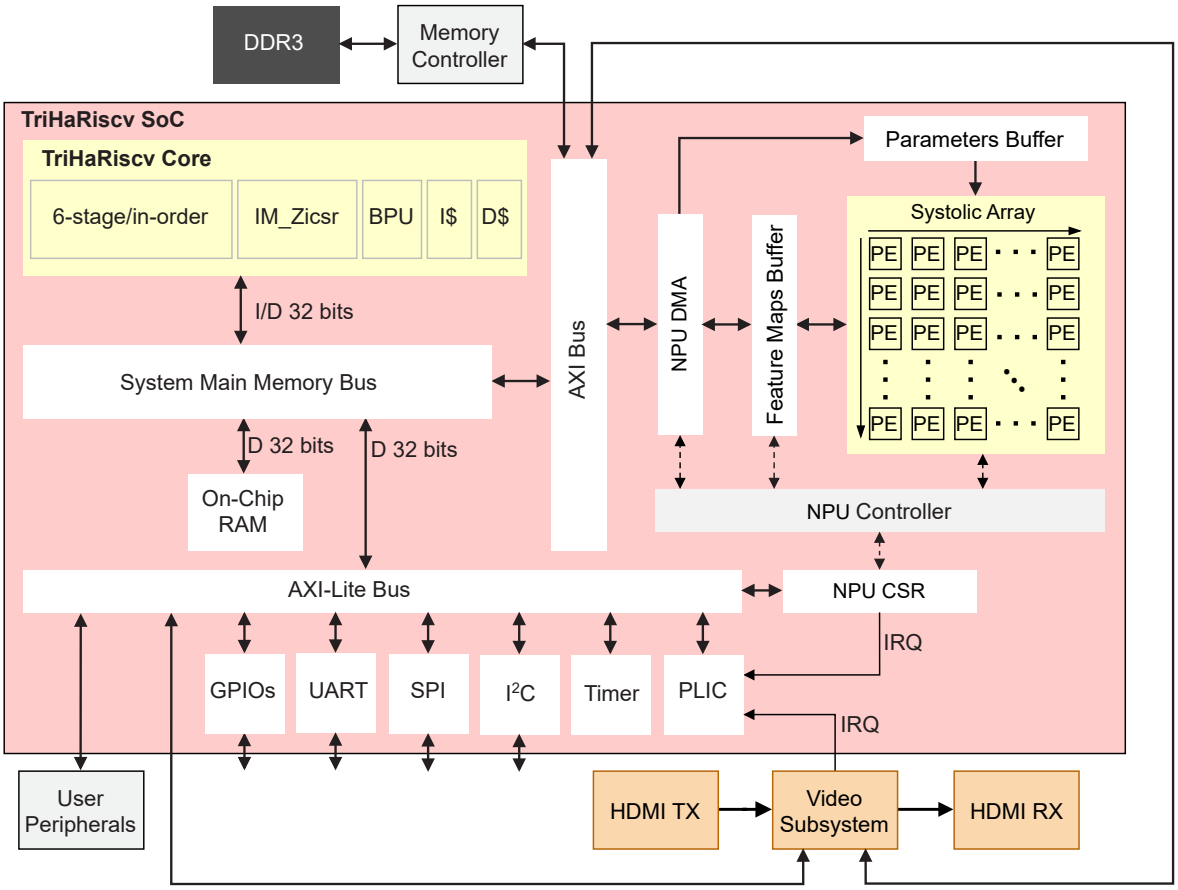


图 2-1 TriHaRiscv 异构 SoC 系统架构图

控制通路采用 32 bit AXI4-Lite，连接 SoC 内部的 NPU、基础外设和性能计数寄存器，并通过外部控制接口访问 Video CSR。视频 DMA 作为独立 AXI 主设备接入数据互连，与 CPU 访存桥和 NPU DMA 共享 DDR 访问通路，逻辑数据接口宽度为 128 bit。视频采集、显示提交及错误事件通过 IRQ 送入 SoC 内的 PLIC。Boot ROM 和片上 SRAM 支持 CPU 启动，DDR 按访问属性划分为 CPU 私有区和共享区。DDR 控制器、视频子

系统及 HDMI 接收、发送封装均位于 SoC 之外，由 FPGA 顶层完成连接。

输入的 800×600 RGB888 画面同时生成原图、400×300 RGB888 预览图和 320×320 UINT8 BGR NCHW 检测输入。NPU 对每个被接纳的输入帧执行一次网络，CPU 完成定点解码、NMS、主人脸选择及头像参数计算。视频 DMA 将三条采集支路的数据写入 DDR，显示端预取原图和头像素材，经行缓存、像素叠加与时序生成模块输出一路 800×600 全屏 HDMI 画面。显示端支持 RAW、BOX 和 AVATAR 三种模式，帧控制模块按帧锁存原图地址与叠加参数，保持图像、检测框和头像位置的一致性。

2.1.2 模块功能

系统按 CPU、NPU、总线、存储、外设和视频六类模块组织，视频模块在 FPGA 顶层与 SoC 连接，SoC 内部模块组成如图2-2所示。硬件模块通过总线和事件接口连接，裸机软件负责算子调度、帧所有权管理及错误恢复。

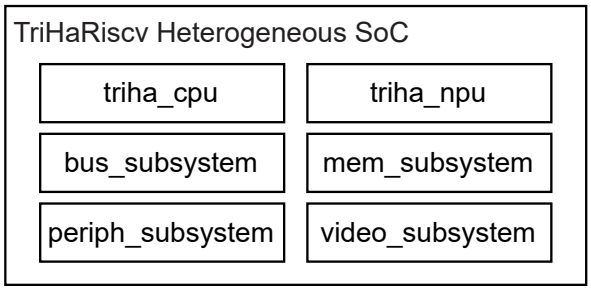


图 2-2 TriHaRiscv SoC 模块图

CPU 经总线模块读取模型输出、配置计算任务并提交显示记录。NPU 通过 DMA 访问权重、输入张量和中间张量；Video 通过独立 DMA 写入采集数据、读取背景及头像素材。存储模块组织 ROM、SRAM 和 DDR 访问，外设模块汇聚完成与错误中断。各模块的主要功能与接口见表2-1。

表 2-1 SoC 模块功能表

模块	主要功能	主要接口
`triha_cpu`	执行程序、处理异常中断，运行算子调度、检测后处理和帧管理软件	64 bit 取指返回、32 bit 数据接口、中断输入
`triha_npu`	执行普通/深度卷积及辅助算子，管理分块、缓冲与 DMA	AXI4 数据、AXI4-Lite 控制、完成/错误中断
`bus_subsystem`	地址译码、访问仲裁、CPU 接口桥接和错误响应	CPU 访存、AXI4 及 AXI4-Lite 接口
`mem_subsystem`	组织 ROM、SRAM、DDR 窗口及访问属性	片上存储接口、DDR 访问接口
`periph_subsystem`	提供 UART、GPIO、SPI、I^2C、定时器、PLIC 及运行计数	AXI4-Lite、外设信号、中断接口

表 2-1 SoC 模块功能表（续）

模块	主要功能	主要接口
`video_subsystem`	完成全帧采集、预处理、输入格式化、全屏显示及头像叠加	像素流、AXI4 DMA、控制寄存器、帧事件

2.2 CPU 设计

2.2.1 CPU 微架构

CPU 采用 RV32IM 指令集，提供 CSR 访问、机器模式异常处理和 `FENCE.I` 指令，承担裸机程序执行、外设控制、NPU 任务配置及检测后处理。指令长度固定为 32 bit，整数寄存器与地址宽度均为 32 bit。处理器采用六级流水线，每周期最多发射一条指令，按程序顺序执行和提交。程序通过物理地址访问存储空间，启动入口映射到 `0x0000 0000` 处的 Boot ROM。

微架构由取指前端、译码模块、ALU、分支单元、CSR、乘除法单元和加载存储单元组成。取指前端生成 PC、进行分支预测，并通过指令 FIFO 提供待执行指令；译码模块完成指令分类、寄存器读取、操作数选择与发射控制，执行和写回通路完成运算、访存及体系结构状态更新。I-Cache 与 D-Cache 分别服务于取指和数据访问，缓存未命中时经下级存储接口完成缓存行填充；本地 ROM、SRAM 及不可缓存设备由存储路由选择访问路径。CPU 核通过取指和数据接口与存储系统连接，可在独立板级配置中直接接入本地 ROM 及作为 TCM 的片上 SRAM。TCM 配置不实例化 I/D Cache 及 DDR 控制器，用于核性能和流水状态采集；L1 Cache 配置通过 I/D Cache 访问板载 DDR，用于评估外存环境下的缓存性能，两种配置的实测结果分别统计。CPU 主要配置见表2-2。

CPU 按经典五级流水线的取指、译码、执行、访存和写回组织功能，并根据 FPGA 同步 RAM 及运算通路的时序设置寄存边界。取指功能分为请求与返回两级，分别完成地址提交和返回指令处理；译码、寄存器读取及发射判定在同一级完成。执行与访存通路按各类结果的产生时机保存状态，整数运算结果和访存地址在 ID/E1 边界锁存，乘法采用寄存分段，除法采用迭代运算。指令状态经 E1、E2 传递至 WB，由有效位、旁路和暂停控制保持操作数及执行顺序。

表 2-2 CPU 主要配置

项目	配置
指令与运行模式	RV32IM，定长 32 bit 指令，机器模式，CSR 访问与 `FENCE.I`
流水线	六级，单发射，顺序执行、顺序提交
通用寄存器	32 个 32 bit 寄存器，`x0` 读值恒为零
取指接口	32 bit 地址、64 bit 返回数据，8 B 对齐取指包
数据接口	32 bit 地址与数据，4 bit 写字节使能

表 2-2 CPU 主要配置（续）

项目	配置
取指 FIFO	2 个取指包，最多容纳 4 条指令
分支预测	512 项直接映射 BTB、512 项两位计数器 BHT、4 项 RAS
指令缓存	16 KiB，两路组相联，256 组，32 B 缓存行
数据缓存	16 KiB，两路组相联，256 组，32 B 缓存行，写回、写分配
结果旁路	整数结果旁路、加载结果旁路、乘法结果旁路

CPU 微架构如图2-3所示。图上方以取指（IF）、译码（ID）、执行（EX）、访存（MEM）和写回（WB）标明功能区域，并列出取指请求（IF1）、取指返回（IF2）、译码/发射（ID）、E1、E2 和 WB 六个实际流水级。五条寄存边界依次为 IF1/IF2、IF2/ID、ID/E1、E1/E2 和 E2/WB，各功能在这些边界之间按数据到达和结果有效时机展开。

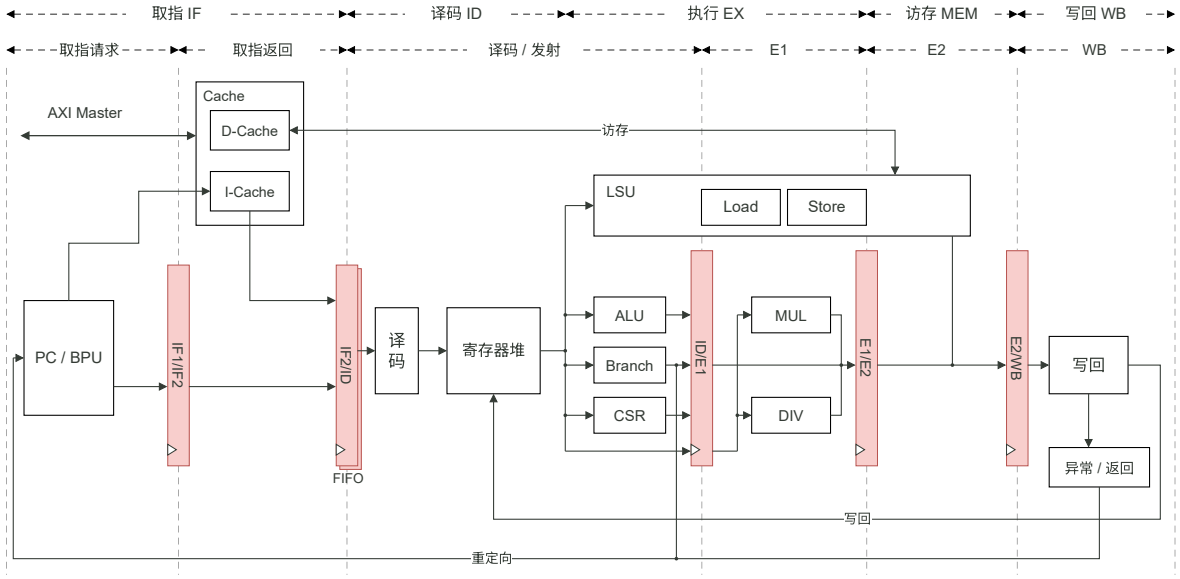


图 2-3 TriHaRiscv CPU 流水线架构图

图中 PC/BPU 概括取指地址生成与分支预测。IF1 向 I-Cache 提交请求，在请求接纳时将 PC 及预测信息保存至 IF1/IF2。I-Cache 采用同步 RAM，指令数据在后续周期返回；IF2 将返回数据与已保存的上下文组合，经响应缓冲送入指令 FIFO。响应缓冲在后端可接纳时直通，在背压时暂存返回包；FIFO 承担 IF2/ID 边界的存储，将取指包的返回与逐条发射分开。

FIFO 输出的指令在 ID 级完成组合译码，寄存器堆提供源操作数，旁路网络补充尚未写回的最新结果。发射条件满足后，操作数进入相应运算通路。ALU 运算、分支判断和访存地址计算在 ID/E1 边界之前完成；该边界归并表示同一时钟沿保存的指令状态、整数与 CSR 结果、访存请求信息及乘除法输入。E1 保持这些状态并发出访存请求，同时推进乘除法运算；E1/E2 保存继续传递的指令状态和已有结果。

图中 LSU 的 Load 和 Store 分别表示加载与存储通路，二者共用地址生成、请求接口及事务跟踪逻辑。访存请求由 E1 发出，E2 根据完成状态接收加载数据或确认存储完成；未收到响应时，流水线保持相应指令。E2 将整数、CSR、除法、乘法或加载结果送入写回选择通路，E2/WB 保存最终写回值及指令状态，WB 据此更新通用寄存器。E1、E2 和 WB 的可用结果经旁路返回操作数选择端，数据相关和资源互锁共同控制后续发射。分支纠正及异常、返回控制向前端提供重定向地址，取消错误路径上的指令。实际流水级与经典五级功能的对应关系见表2-3。

表 2-3 CPU 流水级与功能对应

实际流水级	主要功能	经典五级功能
IF1（取指请求）	选择顺序、预测或重定向地址，发出取指请求，保存已接纳请求的 PC 和预测信息	IF：请求生成
IF2（取指返回）	接收指令包，匹配 PC、预测和错误状态，经响应缓冲写入 FIFO	IF：返回处理
ID（译码/发射）	组合译码、读取与旁路选择操作数、检查发射条件，完成整数运算、分支判断及访存地址计算	ID、EX
E1	保持指令状态和整数结果，发出访存请求，推进乘除法运算并控制等待	EX、MEM
E2	汇集执行结果与访存返回，在加载或存储未完成时保持流水线	MEM 及结果选择
WB	根据指令有效性、异常状态和目的寄存器使能完成写回，形成指令完成信息	WB：寄存器更新

流水线状态包含有效位、PC、指令字、目的寄存器、执行结果及异常信息。暂停保持在途状态，冲刷取消受影响指令的有效性。各级仅在数据有效且下一级可接纳时推进；Cache 未命中、总线等待和迭代除法可使指令停留多个周期。后续按经典五级功能分别说明指令提取、译码、执行、存储访问和写回，并结合上述寄存边界描述实际时序。

2.2.2 指令提取

取指前端由 PC 选择、取指接口、PC 与预测信息寄存器、响应缓冲、分支预测器和指令 FIFO 组成，如图2-4所示。数据通路从 I-Cache 返回指令包，经响应缓冲写入 FIFO，再送入译码/发射级；预测结果和后端重定向反馈至 PC 选择通路。图中的 IF1/IF2 标明请求上下文的寄存边界，IF2/ID 标明指令 FIFO 所在的流水边界；两个 D 符号分别表示训练信息和重定向信息的寄存器。带 * 的标注归并表示同一接口的相关信号。

IF1 根据顺序地址、分支预测目标和后端重定向生成取指 PC，保存在 `pc_f_q` 中。取指接口以 `icache_rd_o` 发起请求，以 `icache_pc_o` 给出 8 B 对齐地址。请求与 `icache_accept_i` 同时有效时完成接纳，`pc_d_q` 和 `pred_d_q` 分别锁存对应 PC 与包内预测跳转标志，形成 IF1/IF2 边界。取指器跟踪一个未返回请求；前一请求返回的同

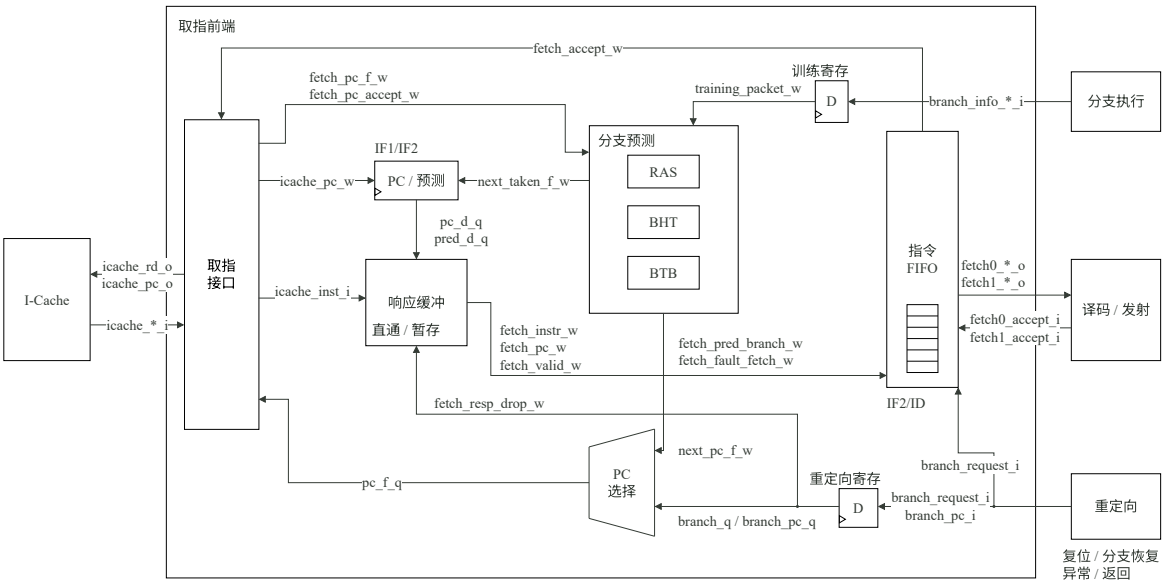


图 2-4 取指前端逻辑框图

一周期可接纳下一请求，连续命中时使请求与返回重叠。

IF2 在 `icache_valid_i` 有效时接收 64 bit 指令数据，并将其与已保存的 PC、预测标志及取指错误状态组合。当 FIFO 允许接纳时，响应缓冲将该包直接送出；若返回时 FIFO 已满，则暂存整包数据，等待 `fetch_accept_w` 恢复后写入 FIFO。这一缓冲只在背压时保存数据，无阻塞时不增加固定流水周期。`fetch_valid_w` 指示返回包有效，PC、指令和错误信息随该有效包共同传递。

每个取指包包含两条定长 32 bit 指令。设 8 B 对齐包地址为 P，低字和高字的指令地址分别为 P 与 $P+4$，下一顺序包地址为 $P+8$。后端顺序执行 PC 每次增加 4；跳转入口位于高字时，发射逻辑依据预期 PC 跳过低字。

指令 FIFO 的内部结构如图2-5所示，由取指包存储、有效位控制、读出选择和队列控制逻辑组成。两个 64 bit 包槽共用读写指针 `wr_ptr_q`、`rd_ptr_q` 和占用计数 `count_q`，最多容纳四条指令。每个包槽还保存 32 bit 取指 PC、两条指令各自的 2 bit 取指错误信息、CSR 分类提示及独立有效位。CSR 提示在入队前提取，完整指令译码位于 FIFO 输出端，与后端译码/发射级衔接。

队列未满时 `accept_o` 有效，包数据和 PC 在写指针指向的空闲槽位采样；`push_i` 与 `accept_o` 同时有效时产生 `push_w`，确认入队并更新元数据、有效位和写指针。入队将低字有效位置 1，高字有效位取 `pred_in_i[0]` 的反值；低字预测跳转时，高字被标记为无效。`accept_o` 由当前占用计数产生，满队列即使在同周期释放包槽，也在下一周期恢复接纳。

输出侧按 `rd_ptr_q` 读取队首包槽，分别给出低字、高字及其 PC、错误信息和 CSR 提示。两路 PC 由保存的包 PC 按 8 B 对齐后分别加 0 和 4 生成；`valid0_o` 和 `valid1_o` 由队列非空条件与对应字有效位共同确定。`pop0_i`、`pop1_i` 分别与对应输出有效位相与，形成 `pop1_w`、`pop2_w`，清除已消费字的有效位。当本周期消费了包内最后一个有

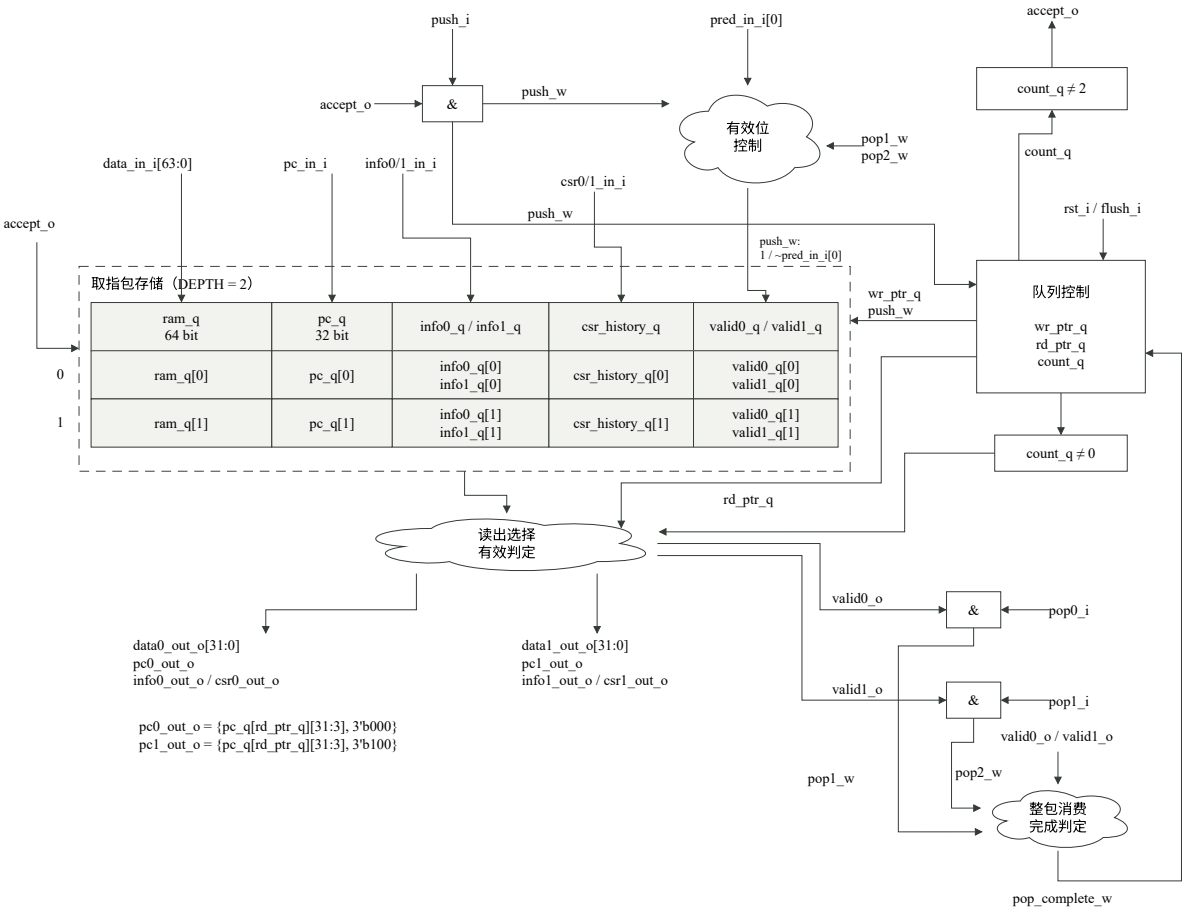


图 2-5 指令 FIFO 结构框图

效字，或同时消费两个有效字时，`pop_complete_w` 有效，读指针推进并释放整个包槽。仅入队时占用计数加 1，仅释放包槽时减 1，二者同时发生时计数保持不变。

FIFO 通过 `fetch0_*_o` 和 `fetch1_*_o` 提供低字、高字两组候选指令及状态，译码/发射级用 `fetch0_accept_i` 和 `fetch1_accept_i` 分别确认消费。两组输出对应一个取指包内的两个位置，发射端按程序顺序每周期选择至多一条有效指令进入执行通路。跳转入口位于高字时，低字丢弃和高字发射可在同周期确认，因此两路消费信号同时有效仍符合单发射结构。指令 FIFO 保存待发射指令，响应缓冲则承接来不及入队的返回包，二者共同处理取指返回与后端发射速率的差异。`flush_i` 清零占用计数和读写指针，并清除取指错误元数据；队列非空条件随之撤销，使旧路径输出失效。

分支预测器由分支目标缓冲器（BTB）、分支历史表（BHT）、返回地址栈（RAS）和预测控制逻辑组成，如图2-6所示。取指接口提供查询地址 `pc_f_i`，预测器生成下一取指地址 `next_pc_f_o` 和两位包内跳转标志 `next_taken_f_o`。两组输出分别接入 PC 选择通路和 IF1/IF2 预测信息寄存器，对应前端框图中的 `next_pc_f_w` 与 `next_taken_f_w`。BTB 与 BHT 采用组合查询，预测结果在取指请求被接纳时随 PC 保存；图中的 D 表示实际分支事件进入预测器前的训练寄存器。

BTB 保存分支 PC 标签、目标地址及调用、返回、无条件跳转类型，共 512 项，采

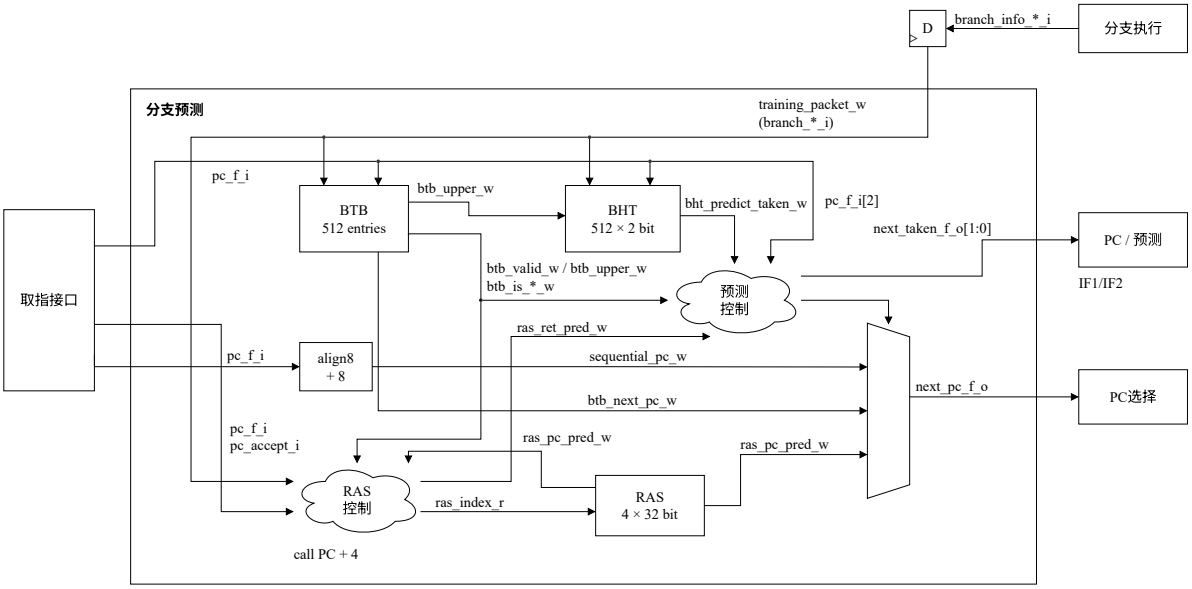


图 2-6 分支预测逻辑框图

用直接映射结构。BTB 分成两个各 256 项的存储体，以 PC 的第 2 位区分包内位置，以 PC 的第 10 至 3 位索引行，以高位地址进行标签匹配。查询起点为低字时优先采用低字命中，低字未命中则选择高字；查询起点为高字时仅采用高字命中结果。`btb_valid_w` 表示查询命中，`btb_upper_w` 表示选中高字，`btb_is_*_w` 提供该条目的控制转移类型。命中时由 `btb_next_pc_w` 给出目标地址，未命中时该通路返回下一顺序包地址。

BHT 包含 512 个两位饱和计数器，查询索引由 PC 的第 10 至 3 位及 `btb_upper_w` 组成，使方向预测对应 BTB 选中的包内指令。低字与高字对应的计数器并行读出，再按选中位置生成 `bht_predict_taken_w`；计数值为 2 或 3 时预测跳转。计数器复位为 3，训练时按实际分支 PC 的第 10 至 2 位索引。实际跳转使计数器递增，实际不跳转使其递减，并在 0 和 3 处饱和。

RAS 保存 4 个 32 bit 返回地址，以 `x1` 作为调用和返回识别依据，调用时记录该指令的 $PC + 4$。RAS 控制逻辑结合 BTB 给出的调用、返回类型和栈顶有效状态，生成预测压栈、出栈及返回选择条件。`pc_accept_i` 对应前端的 `fetch_pc_accept_w`，表示取指地址通路可以推进；调用或返回预测被接纳时，RAS 推进预测指针。RAS 同时维护恢复参考指针，在调用或返回误预测时据此修正预测位置。`ras_pc_pred_w` 给出栈顶地址，`ras_ret_pred_w` 表示有效返回预测。

下一取指地址从顺序地址、BTB 目标和 RAS 栈顶地址中选择。图中的 `align8 + 8` 先将 PC 向下对齐到 8 B 边界，再加 8，生成 `sequential_pc_w`。有效返回预测优先选择 RAS 栈顶；其余情况下，BHT 预测跳转或 BTB 标记为无条件跳转时采用 BTB 目标，否则采用顺序地址。BTB 未命中时，取指沿顺序地址推进。预测控制结合命中状态、跳转条件、包内位置及 PC 第 2 位生成两位跳转标志：`01` 表示低字预测跳转，`10` 表示高字预测跳转，`00` 表示无预测跳转。该标志随对应请求保存，返回后用于控制 FIFO 中包内指令的有效性；低字预测跳转时取消同包高字。

分支执行结果通过 branch_info_*_i 送入训练寄存器，形成 training_packet_w 后更新预测器。图中 branch_*_i 表示预测器接收的训练信号组，包含恢复请求、实际方向、分支 PC、目标地址及调用、返回和跳转类型。BHT 依据实际方向更新，方向训练同时覆盖预测正确的分支；BTB 在误预测恢复时分配或修正条目，已命中条目在实际不跳转时保留原目标地址。RAS 根据训练事件修正调用返回状态。训练寄存将执行结果与预测状态更新分隔在时钟边界两侧，前端查询使用已保存的预测状态。

复位入口、分支恢复及异常返回通过 branch_request_i 和 branch_pc_i 进入重定向通路。重定向请求直接触发 FIFO 清空，同时将恢复地址保存到重定向寄存器；branch_q 与 branch_pc_q 驱动后续 PC 恢复。已被接口接纳的旧请求继续完成返回，fetch_resp_drop_w 在恢复期间屏蔽旧响应的有效性，取消响应缓冲中的错误路径数据。前端在请求条件允许后从恢复地址重新取指。重定向寄存将后端控制与取指地址选择分开，FIFO 冲刷和响应丢弃共同保证恢复后的指令流正确。

2.2.3 指令译码

译码模块位于取指前端与执行通路之间，由指令译码、寄存器堆、指令选择、操作数选择和发射控制组成，如图2-7所示。前端通过 fetch0_*_i 和 fetch1_*_i 提供同一取指包内的低字、高字及其 PC、有效位和错误状态。两条候选指令并行译码和预读寄存器，随后按预期执行 PC 选择一条发射，每周期最多向执行通路发送一条有效指令。

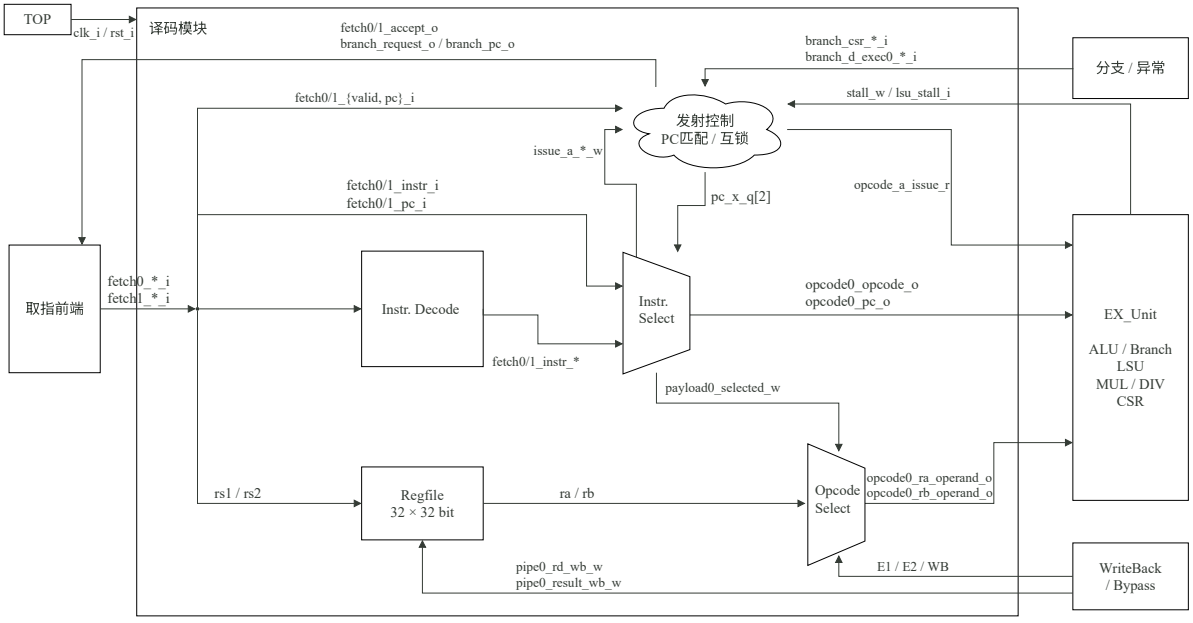


图 2-7 译码模块逻辑框图

指令译码逻辑（Instr. Decode）检查 32 bit 指令的操作码和功能字段，将指令分类为整数运算、控制转移、加载存储、乘法、除法及 CSR 操作，生成执行单元分类、目的寄存器有效位和非法指令标志。两路分类结果以 fetch0/1_instr_* 归并表示，随对

应指令送入指令选择通路。未识别的编码进入非法指令异常通路；取指访问错误随指令保留，并与非法指令分别形成异常原因。

指令选择逻辑（Instr. Select）依据已寄存的预期执行 PC 第 2 位 `pc_x_q[2]` 选择包内低字或高字，输出指令字、PC 和分类信息。`payload0_selected_w` 在选择低字时为 1，并用于选择同一路候选指令的操作数。完整 PC 匹配和有效位检查由发射控制并行完成，仅在检查通过时允许正常发射。宽数据选择直接使用已寄存的 PC 位，使其不必等待完整地址比较结果。

通用寄存器堆（Regfile）包含 32 个 32 bit 体系结构寄存器，源寄存器索引取自 `rs1` 和 `rs2` 字段，目的寄存器索引取自 `rd` 字段。物理实现采用异步读分布式 RAM，通过同值副本提供两条候选指令的四路预读。读取地址直接来自各自指令字段，使寄存器读取与指令选择并行进行。单发射通路提供一个有效写回源，`pipe0_rd_wb_w` 和 `pipe0_result_wb_w` 分别给出写回地址与数据，并同步更新各副本。`x0` 的读值固定为零，对 `x0` 的写入不改变体系结构状态；复位后的寄存器有效掩码使尚未写入的寄存器读出零值。

操作数选择通路接收寄存器读值及 E1、E2、WB 返回的可用结果，先对两路候选指令分别完成相关性比较与旁路选择，再按 `payload0_selected_w` 选出待发射指令的两个操作数。图中的 Opcode Select 归并表示这一过程。存在多个匹配结果时，按 E1、E2、WB、寄存器堆的优先级选用最新值；结果尚不可用时，由互锁逻辑阻止相关指令发射。输出 `opcode0_ra_operand_o` 和 `opcode0_rb_operand_o` 与 `opcode0_opcode_o`、`opcode0_pc_o` 共同送入执行通路。

发射控制根据 `issue_a_*_w` 分类及寄存器索引，检查数据相关、执行资源和流水线状态。指令 PC 匹配、操作数可用、流水线允许推进且无冲刷时，`opcode_a_issue_r` 形成发射事件；各执行单元结合指令类型及相应有效信号接收指令。

`stall_w`、`lsu_stall_i` 以及除法和 CSR 在途状态共同限制发射，暂停期间 FIFO 保留未消费的指令。`fetch0/1_accept_o` 分别确认两路候选指令的消费；高字入口下可同时丢弃低字并发射高字，两路确认同时有效仍只产生一次正常发射。

分支执行结果更新预期执行 PC，CSR 异常和返回请求具有更高优先级。若候选指令均不匹配预期 PC，或发生 CSR 重定向，`branch_request_o` 与 `branch_pc_o` 通知前端清空旧指令并从恢复地址取指。

译码、寄存器读取、操作数选择与发射判定在同一流水级内完成，完整译码位于 FIFO 输出端，不额外增加译码寄存级。执行通路依据选中的指令字生成 I、S、B、U 和 J 型立即数，并完成移位量、访存宽度及 CSR 操作类型的解释。整数结果、访存请求信息及其他在途状态在 ID/E1 边界保存，各类数据相关和资源等待的处理见流水线控制部分。

2.2.4 指令执行

执行阶段由整数与分支单元（ALU/Branch）、乘法单元（MUL）、除法单元（DIV）、CSR 单元和加载存储单元（LSU）组成，整体连接如图2-8所示。译码/发射模块提供指令字、PC、寄存器操作数和有效信号，各功能单元依据指令类型完成运算，并将结果或完成状态送入流水线控制、写回和旁路通路。

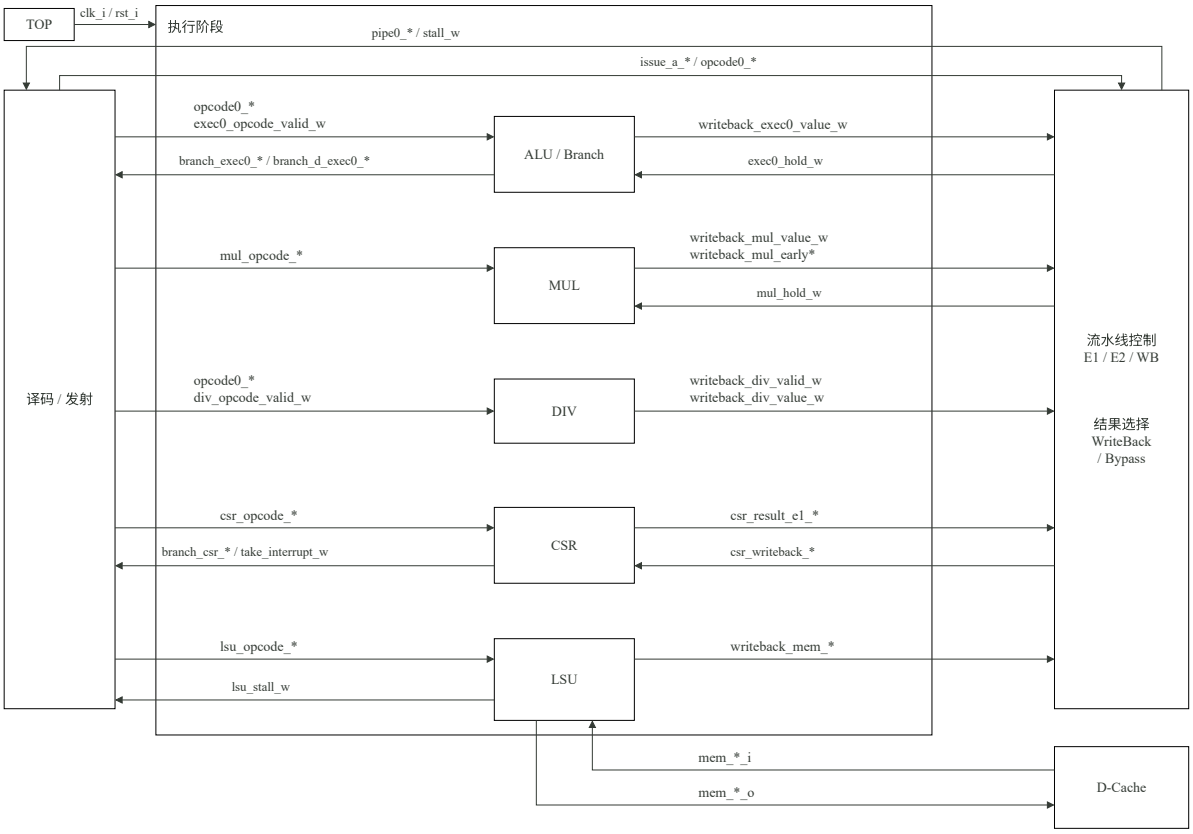


图 2-8 执行阶段模块框图

图中 `opcode0_*` 表示单发射通路选出的指令及操作数，整数与分支单元和除法单元共用该组数据，并分别接收 `exec0_opcode_valid_w` 和 `div_opcode_valid_w`。乘法、CSR 和 LSU 通过各自的 `opcode` 接口接收同一发射通路的指令，图中以 `*` 归并表示数据与控制信号。

整数执行单元完成加减、逻辑运算、移位、有符号和无符号比较，ALU 数据通路如图2-9所示。操作数选择逻辑根据指令类型，从旁路网络提供的寄存器值、PC、立即数及常数中形成两个 32 bit 运算输入，分别由 `alu_a_i` 和 `alu_b_i` 送入组合 ALU，`alu_op_i` 指定运算类型。

加减通路执行 `ADD/SUB`，逻辑通路执行按位与、或、异或，比较通路完成 `SLT/SLTU` 并将比较结果扩展为 32 bit 的 0 或 1。移位通路使用第二操作数的低 5 位确定移位量，支持逻辑左移、逻辑右移和算术右移；算术右移按第一操作数的符号位填充高位。结果选择器按运算类型选取各通路输出，并保留第一操作数直通路径。`LUI` 将 U 型立即数

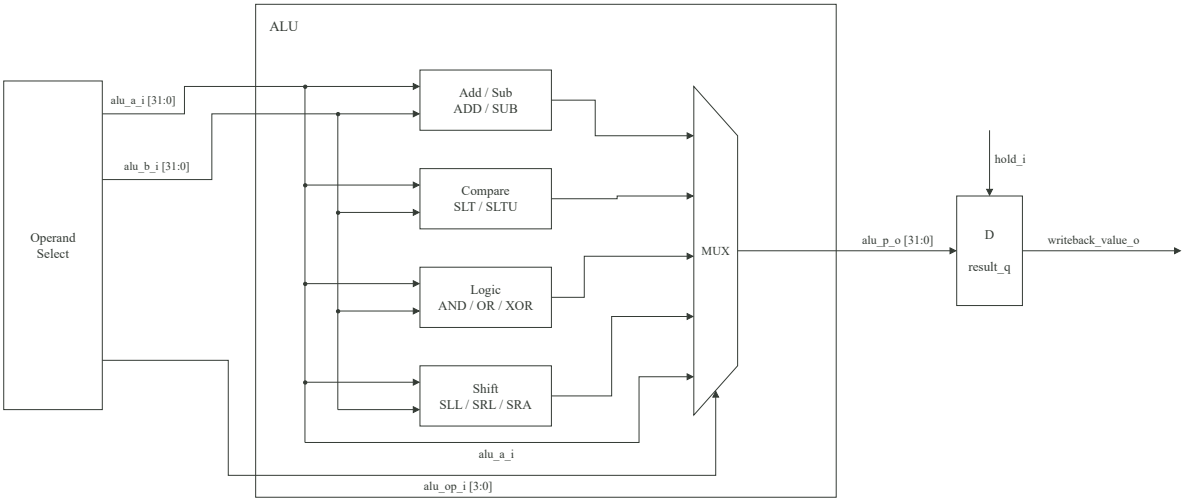


图 2-9 ALU 数据通路框图

送入直通路径，`AUIPC` 计算 PC 与 U 型立即数之和，跳转指令通过加法通路形成 PC+4 链接地址。

组合结果由 `alu_p_o` 输出，在 ID/E1 边界保存至 32 bit 结果寄存器 `result_q`。`hold_i` 有效时保持寄存器原值，解除暂停后随对应指令推进；`writeback_value_o` 提供保存的整数结果，并接入 E1 早期旁路，使相关指令能够在操作数可用时继续发射。

分支判断与整数运算并行进行，条件分支比较两个源操作数，直接跳转根据 PC 和立即数形成目标，间接跳转根据基址和立即数相加并清除目标地址最低位。跳转指令将 PC+4 作为链接地址，目标地址按 4 B 指令边界进行对齐检查。分支方向、目标地址及调用返回类型经 `branch_exec0_*` 和 `branch_d_exec0_*` 反馈至译码/发射模块，用于更新预期执行 PC、组织预测训练并发起必要的取指恢复。

乘法单元支持 `MUL`、`MULH`、`MULHSU` 和 `MULHU`，分别产生低 32 位乘积或不同符号组合的高 32 位乘积。常规乘法结果在 E2 寄存后进入结果选择和旁路通路。针对 `MUL` 且两个源操作数高 16 位均为零的情形，直接从 E1 操作数形成的组合低位部分积旁路，缩短小整数乘法后的相关等待。其余乘法指令使用常规结果通路，发射控制依据结果有效时机决定后续指令能否推进。

除法单元采用迭代运算，支持有符号和无符号除法及求余。执行期间保持除法在途状态，完成后由 `writeback_div_valid_w` 确认结果有效，并将商或余数送入流水线。除数为零时，商为全 1，余数等于被除数；有符号最小负数除以 −1 时，商保持最小负数，余数为零。正常有符号除法按向零截断规则产生商，余数符号与被除数一致。

CSR 运算根据指令类型读取寄存器，并形成交换、置位或清位所需的值。CSR 读值、写入信息与早期异常状态在 ID/E1 边界保存，通过 `csr_result_e1_*` 送入后续流水线；写回侧的 `csr_writeback_*` 返回寄存器写入信息及异常状态。异常入口和返回控制生成 `branch_csr_*` 重定向，数据缓存维护指令由加载存储通路处理。

图2-8右侧模块归并表示 E1、E2、WB 的指令状态跟踪、结果选择、写回和旁

路。各单元的结果按对应指令的类型和有效状态选取，`exec0_hold_w` 与 `mul_hold_w` 在暂停时保持整数和乘法通路状态，`pipe0_*` 与 `stall_w` 向发射端反馈相关信息及推进条件。LSU 经 `mem_*_o` 和 `mem_*_i` 与 D-Cache 交换访存请求及响应，并通过 `writeback_mem_*` 返回访存结果、完成和异常状态，其内部访问过程在下一节展开。

2.2.5 存储访问

加载存储单元接收发射端提供的指令和源操作数，完成有效地址计算、访问类型译码、数据对齐、请求跟踪和响应处理，内部结构如图2-10所示。加载和存储共用请求控制及存储接口，并分别处理加载返回数据和存储写数据。

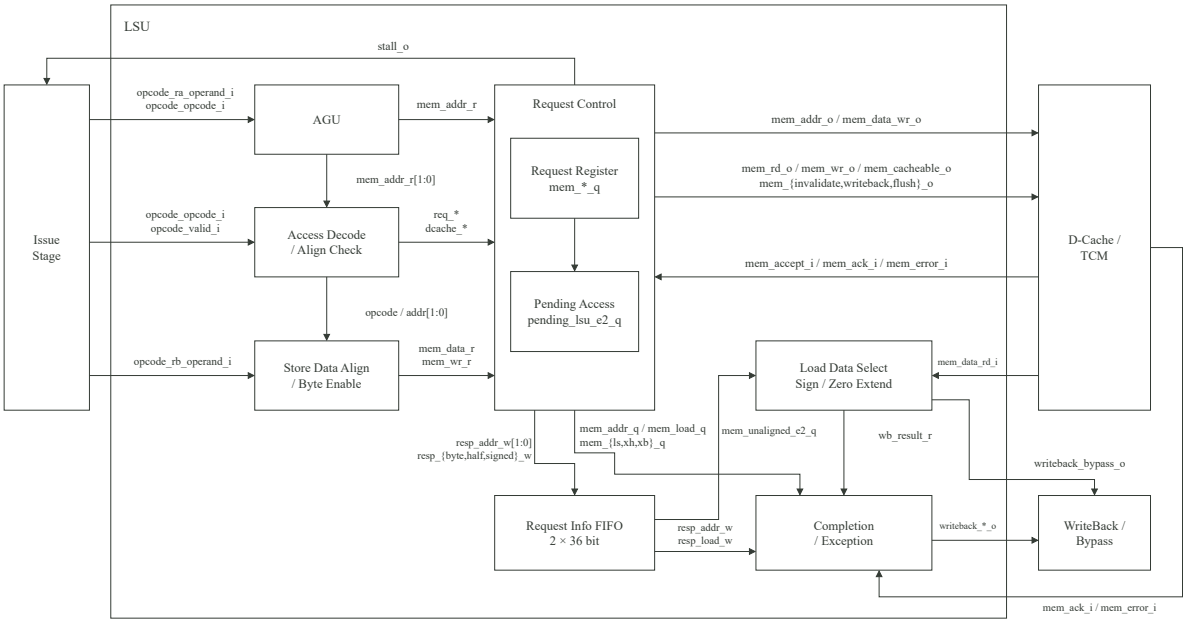


图 2-10 加载存储单元逻辑框图

地址生成单元将基址寄存器值与符号扩展立即数相加，加载指令使用 I 型立即数，存储指令使用 S 型立即数。访问译码确定加载或存储、字节宽度及加载扩展方式，并检查有效地址的自然对齐要求：字节访问允许任意字节地址，半字访问要求最低地址位为零，字访问要求最低两位为零。不满足要求时阻止相应外部读写，并由内部完成通路报告非对齐异常。存储通路根据有效地址低位排列写数据，生成 4 bit 字节使能，配合按字对齐的接口地址完成选中字节更新。

访存地址、写数据、读写控制及缓存属性在 ID/E1 边界保存，E1 向存储系统发出请求。接口将请求接纳与响应完成分开：`mem_accept_i` 确认本次请求被接纳，`mem_ack_i` 确认访问完成，`mem_error_i` 指示响应错误。请求尚未接纳时保持地址、写数据和控制信息；`pending_lsu_e2_q` 记录已接纳访问的等待状态，并通过 `stall_o` 限制后续发射。成功响应到达后，可在满足流水线推进条件时接纳后续请求。缓存清理、写回和失效控制经同一请求接口送往存储系统。

请求信息 FIFO 深度为 2，每项保存 32 bit 有效地址及加载、字节访问、半字访问

和符号扩展四个标志，共 36 bit。请求被接纳时写入对应信息，响应完成时释放队首；非对齐访问通过内部完成事件维护相同的入队、出队关系。接口保留 11 bit 事务标签，顺序访存通路使用固定标签，通过请求信息 FIFO 将返回数据与地址低位及访问类型关联。

加载返回后，数据选择逻辑根据队首地址低位选取字节、半字或整字，再执行符号扩展或零扩展，形成 32 bit 结果。`writeback_bypass_o` 直接提供扩展后的数据，旁路使用资格由流水线中的指令类型及完成、异常状态共同限定。`writeback_valid_o` 在外部响应或内部非对齐访问完成时有效；成功加载经 `writeback_value_o` 返回数据，访问错误或非对齐异常时则携带故障地址，并通过 `writeback_exception_o` 给出异常原因。

请求被存储系统接纳后，加载存储单元继续跟踪返回归属和完成状态。后端暂停期间保持在途指令，已接纳的事务按原归属返回；冲刷通过指令有效位抑制正常写回，并按接口协议回收已经发出的请求。

加载指令等待成功响应后提供写回数据，访存错误取消正常写回并转入异常处理。可缓存存储在 D-Cache 完成选中字节更新及脏位设置后得到完成响应，数据随后通过缓存行清理或替换写入 DDR。不可缓存存储则等待下级写事务成功后完成。系统据此将 DMA 数据放置在不可缓存共享区，以成功写响应、访问排序和所有权发布确定交付时机。

2.2.6 指令写回

指令在 E2 完成结果选择后，将有效状态、PC、指令字、目的寄存器及写回值传递到 WB。WB 根据指令有效性、异常状态和目的寄存器写使能更新通用寄存器，单周期最多完成一条指令的写回。分支及存储等指令不产生通用寄存器写入，其完成状态仍参与流水线控制和指令执行观测。CSR 修改和异常状态更新由专用控制通路处理，并通过发射互锁与顺序流水线保持程序要求的执行次序。

异常原因、故障 PC 及访问状态随指令传递。故障指令到达写回边界时取消正常通用寄存器写入，异常控制保存处理程序所需的状态，并向取指前端提供恢复地址。

验证环境在 WB 边界采样 PC、指令字、目的寄存器、写回值及异常状态，形成指令执行观测记录。计数依据有效完成事件，并包含无寄存器写回的指令，不能用寄存器写使能直接代替完成指令数。周期计数器随 CPU 时钟运行，WB 观测计数由仿真监视器维护；IPC 统计使用同一计时窗口内的 WB 观测数和周期数，记录采样条件及异常排除规则。

2.2.7 流水线控制

流水线控制根据在途指令的有效位、目的寄存器及执行资源状态决定后续指令能否发射，并在结果可用时选择旁路，在执行或访存未完成时保持相关状态。

相关性检查比较候选指令的寄存器字段与在途指令的目的寄存器。能够提前取得结果时，操作数按 E1、E2、WB、寄存器堆的优先级选择最新值；匹配 `x0` 时强制使用零值。整数运算结果可从 E1 旁路，加载结果和普通乘法结果从 E2 旁路，已经到达 WB 的结果可直接送回发射端。结果尚不可用时，记分牌阻止相关指令发射，避免读取旧寄存器值。

加载指令位于 E1 时，其目的寄存器保持相关性阻塞；进入 E2 后仍需等待访存完成。乘法指令通常等待 E2 结果，满足低位乘法早期旁路条件时可提前解除相关等待。除法和 CSR 操作设置在途标志，在完成前限制后续发射；E1 存在加载或存储时，对下一条乘法、除法及 CSR 操作施加资源互锁。这些控制将变长执行和访存等待纳入同一条顺序流水线。

暂停期间保留待执行指令，高字入口之前的低字可按 PC 选择规则丢弃。正常执行请求仅在满足对应发射条件时产生。后端同时保持在途指令的 PC、指令字、目的寄存器、结果及异常状态；流水线恢复推进后，这些状态同步传递到后一级。

分支单元计算实际方向和目标，发射逻辑据此维护预期执行 PC。候选指令 PC 与预期 PC 不一致时，控制逻辑发出分支重定向，清空取指 FIFO 中的错误路径指令并从正确地址恢复。CSR 异常和返回重定向具有更高优先级。冲刷撤销受影响指令的有效性和正常写回资格；已经由存储系统接纳的事务继续完成协议处理。

2.2.8 存储系统

存储系统按物理地址选择本地存储、缓存 DDR 或不可缓存访问。Boot ROM 和片上 SRAM 通过本地接口访问；`0x8000 0000` 至 `0x81FF FFFF` 为 CPU 私有可缓存 DDR，`0x8200 0000` 至 `0x87FF FFFF` 为不可缓存共享 DDR，MMIO 按设备访问处理。缓存子系统同时检查地址范围和请求缓存属性，使共享区及设备寄存器始终绕过缓存。地址路由在请求接纳时保存返回归属，已接纳请求的目标与响应关系保持到事务结束。

I-Cache 和 D-Cache 各为 16 KiB，采用两路组相联、256 组和 32 B 缓存行，数据阵列使用同步 RAM。32 bit 地址划分为 19 bit 标签、8 bit 组索引和 5 bit 行内偏移。查询根据组索引读取两路标签及数据，标签匹配且有效时命中，否则选择替换行并启动填充。替换控制使用全局交替状态选择路，D-Cache 在目标组存在无效行时优先使用无效路。下级 DDR 读通路以 128 bit 返回数据，一条缓存行由两个 16 B 数据拍组成。填充控制接收完整且无错误的响应后才建立有效缓存行；失败填充保持新行无效，并向原访问返回错误。I-Cache 从命中行中选择 64 bit 取指包，D-Cache 向加载存储单元提供 32 bit 数据。

D-Cache 采用写回、写分配策略。存储命中时按字节使能更新行内数据并设置脏位，完成响应由缓存内部产生。存储未命中时，先处理被替换行，再读入目标缓存行，合并本次写数据并标记为脏行；后续对同一行的多次存储可在片上合并。加载命中从数据阵列取得选中字，成功的可缓存加载响应可与满足条件的下一请求接纳重叠，返回数据通过寄存保持。缓存未命中、维护操作和错误处理由阻塞式状态机串行完成。

替换脏行时，控制器将 32 B 行拆为 8 个 32 bit 字，逐字通过下级写通路提交，并等待各字的成功响应。只有 8 个字全部写回成功，才清除原行脏状态并继续替换或维护。任一写响应失败时，原行保持有效和脏状态，终止后续填充并上报错误。重试以整行为单位重新写回，以覆盖错误发生前已经到达 DDR 的部分数据。

缓存维护操作控制私有缓存数据的写回与失效，具体语义见表2-4。其中“清理”表示将脏数据写回并保留缓存行，“丢弃”表示直接取消有效性，“清理并失效”表示先写回再取消有效性。`0x3A0` 至 `0x3A2` 在本处理器中用于专用缓存维护接口，仅由 `CSRRW` 指令触发；单行维护以源寄存器提供目标地址，按缓存行匹配。

表 2-4 CPU 缓存维护操作

操作	功能	缓存侧完成条件
CSR `0x3A1`	清理匹配的 D-Cache 行，保留有效性	脏行全部写响应成功；未命中或干净行无需写回
CSR `0x3A2`	丢弃匹配的 D-Cache 行，包括其中的脏数据	行失效完成，不产生写回
CSR `0x3A0`	遍历 D-Cache 全部组和路，清理并失效	全部处理完成，或在首个写回错误后产生缓存错误响应
`FENCE.I`	排空先前数据事务，清理并失效 D-Cache，再请求 I-Cache 失效	D-Cache 维护成功且 I-Cache 失效请求脉冲结束后恢复数据接纳；可缓存取指等待 I-Cache 失效完成

表中前三项的完成和错误条件指缓存接口响应，`FENCE.I` 一项列出后续访问的接纳条件。维护指令按 CSR 通路进入 WB，维护事务由加载存储单元和缓存继续跟踪，指令到达 WB 不等同于脏行已经成功写回。维护错误也不经普通加载存储指令的 E2 异常条件上报，软件接口应区分 CSR 指令完成与缓存事务结果。

CPU 私有区用于程序、栈及内部工作数据，DMA 使用的帧缓存、共享张量和模型数据放置在不可缓存共享区，不建立可缓存别名。软件在共享区写入完成后执行 `fence rw,rw`，再发布缓冲区所有权；消费者根据完成事件及所有权状态读取数据。普通访问屏障用于约束访问顺序，脏数据写回由专用缓存维护负责。

`FENCE.I` 发起后，缓存控制暂时停止接纳新的 DDR 取指和数据请求，排空已接纳的数据事务，完成 D-Cache 清理失效后向 I-Cache 发送失效请求。D-Cache 维护成功且 I-Cache 失效请求脉冲结束后，数据通路恢复接纳；可缓存取指继续由 I-Cache 自身状态机阻塞，直到逐组失效完成。该操作通过缓存侧接纳控制约束后续取指，不向核心提供独立的维护完成响应。内部清理若遇到写回错误，缓存子系统锁存协议故障并停止接纳请求，SoC 路由器将后续普通 DDR 访问转换为错误响应，交由协调复位恢复。CPU、缓存及外部事务控制共同管理复位；需要保留私有脏数据的计划复位应先完成缓存清理，再开始新的缓存有效状态。

2.2.9 中断和异常

CPU 在机器模式下处理异常和中断，机器 CSR 保存处理器状态、使能位、待决位及异常返回信息，主要接口见表2-5。CSR 指令支持寄存器和立即数两种操作数形式，提供交换、按位置位和按位清位操作，各寄存器的可写字段按实现掩码更新。软件使用已定义的 CSR 地址进行初始化和事件处理；未定义地址按读零、写忽略处理。

表 2-5 CPU 主要 CSR 接口

寄存器	地址	功能
mstatus	0x300	中断使能及陷入、返回状态
misa	0x301	固定返回 0x40001100，表示 RV32IM
mie、mip	0x304、0x344	机器中断使能及锁存的待决状态
mtvec	0x305	四字节对齐的机器异常、中断直接入口
mscratch	0x340	机器处理程序暂存数据
mepc	0x341	陷入时保存的返回 PC
mcause、mtval	0x342、0x343	事件原因及对应异常信息
cycle、time	0xC00、0xC01	读取同一周期计数器的低 32 位
timeh	0xC81	读取周期计数器的高 32 位
内部计时比较	0x7C0	比较低 32 位周期值，写入后重新布防
mhartid	0xF14	处理器编号，固定为 0

CPU 提供一根外部 IRQ 输入，接收核外中断请求并在 mip 中形成 MEIP 待决状态。机器软件中断通过 CSR 操作设置 MSIP；内置计时比较器在周期计数低 32 位等于 0x7C0 中的比较值时设置 MTIP，单次触发后撤销布防。下一次计时事件由软件写入新的比较值重新设置。外部事件与内部计时事件均使用锁存的待决位，处理程序在处置中断源后清除相应状态。

中断受 mstatus.MIE 全局使能和 mie 分项使能控制。多个中断同时满足受理条件时，按机器软件中断、机器计时中断、机器外部中断的次序选择，原因号分别为 3、7 和 11。中断在流水线允许推进且没有受抑制的在途操作时进入异常控制路径，抑制被接管指令的正常副作用，并将中断状态传递至 WB 异常处理边界。

同步异常包括非法指令、ECALL、EBREAK、指令访问错误、指令地址不对齐、加载或存储访问错误以及数据地址不对齐。异常原因与指令 PC 随流水线传递。mtval 在数据访问及数据对齐异常时记录有效地址，在非法指令时记录指令字；取指类异常记录异常 PC，分支目标不对齐时记录分支指令 PC。中断、ECALL 和 EBREAK 对应的 mtval 为零，软件结合 mcause 判断其含义。

陷入时，异常控制写入 mepc、mcause 和 mtval，将原 MIE 保存到 MPIE 并清除 MIE，然后将取指地址重定向至 mtvec。软件将 mtvec 配置为四字节对齐的统一入口，由处理程序根据原因寄存器分派事件。MRET 从 mepc 恢复 PC，将 MPIE 恢复到 MIE 并

置 MPIE 为 1；程序继续在机器模式下运行。异常返回前，处理程序完成现场恢复，并根据异常类型决定保留或调整返回地址。

周期计数器按 CPU 时钟逐拍递增，流水线暂停期间继续计时。软件读取 64 bit 计数时依次读取高位、低位和高位，若两次高位不一致则重读，避免低位进位造成计时撕裂。板级基准使用该计数接口计算运行时间，指令完成数和流水线事件则由验证环境独立统计。

2.3 NPU 设计

2.3.1 NPU 微架构

NPU 由 PE 计算通路、算子控制器、片上缓冲、DMA、卷积窗口生成、辅助算子、定点后处理和寄存器接口组成。计算数据采用有符号 INT8，部分和采用 INT32，乘加并行度支持 64 和 128 两种配置。普通卷积与深度卷积按各自的数据依赖组织供数和累加，CPU 通过 MMIO 启动单算子，NPU 内部完成该算子的分块循环。

一次任务依次经历参数锁存、输入与权重搬入、窗口供数、计算、重定标和输出写回。每次启动只接受一个算子，算子完成前保持参数稳定。网络跨层调度由 CPU 执行，NPU 提供当前算子编号、帧编号、周期计数和错误状态。

2.3.2 PE 计算单元

PE 对两个有符号 INT8 操作数相乘，将乘积累加到 INT32 部分和。控制信号区分清零、首次累加和续累加，输入有效与计算使能共同控制状态更新。跨输入通道块的部分和保留 INT32 精度，直至当前输出的归约结束。

普通卷积模式把同一输出的不同输入通道乘积归约到相应累加器；深度卷积模式按通道独立维护部分和。尾通道和尾空间块使用有效掩码，屏蔽填充数据对结果的影响。乘法逻辑映射到 APM，累加与缓冲通路按输出位宽组织。

2.3.3 卷积计算通路

普通卷积沿输入通道和卷积窗口完成归约，沿输出通道和输出空间位置分配并行工作。1×1 卷积省去空间窗口展开，按输入通道组织向量乘加。3×3 卷积结合片上窗口生成器提供相邻像素，保留跨通道块部分和。

深度卷积将每个输入通道映射到对应输出通道，按通道及空间位置并行执行，累加器之间保持通道独立。两种模式通过供数选择与累加控制使用乘加通路，分别统计有效乘加数、供数等待和尾块开销。阵列利用率按实际有效运算计数，64/128 通路配置采用相同整数语义。

2.3.4 算子控制器

算子控制器锁存形状、核尺寸、步长、补边、分组、张量地址及量化参数，生成输出通道块、输入通道块和空间块循环。块间控制保证权重、激活与部分和对应同一输出范围，完成判断包含有效计算、后处理和所有输出写事务。

控制状态与 DMA 握手衔接，背压时保持地址、数据和计数状态。当前算子全部输出获得成功写响应后置 DONE；错误响应记录错误码并结束当前算子，软件据帧编号和算子编号处理整帧状态。

2.3.5 片上缓冲

片上缓冲分别存放输入 tile、权重 tile、INT32 部分和与输出数据，各工作区按 32 KiB 容量组织，并按并行供数要求划分存储银行。深度卷积窗口和辅助操作复用缓冲接口，银行分配与访问顺序匹配计算模式。跨输入通道块累加期间，部分和所在区域保持占用，输出归约结束后再交给定点后处理。

DMA 与计算使用明确的读写所有权。局部双缓冲通过当前块和预取块标识交接，只有数据完整且写入结束后才交换角色。没有重叠的阶段按搬入、计算、搬出的顺序推进，计数器分别记录有效计算与等待时间。

2.3.6 DMA 控制器

DMA 经 128 bit AXI4 接口搬入输入与权重，写回特征图和检测头输出。采用 INCR 突发，单次突发上限为 16 拍，遇到 4 KiB 边界或尾部数据时拆分事务并生成字节使能。每个主端读、写方向各保留一个未完成事务，维护地址、长度与对应缓冲区状态。

读返回按目标缓冲写入，写完成以 B 通道成功响应为准。背压不能改变已提交的事务信息，错误响应向算子控制器上报。DMA 不在 DDR 中生成完整 Im2Col 矩阵，卷积窗口由片上逻辑形成。

2.3.7 卷积窗口生成

窗口生成器根据输入尺寸、有效通道、步长和补边计算输入地址，从片上缓冲提取 3×3 或 1×1 操作数。图像边界使用量化零值填充，通道对齐区由有效掩码屏蔽。深度卷积供数保持通道对应关系，普通卷积供数按输入通道归约顺序排列。

张量的逻辑形状与物理步长分别配置，第一层有效通道为 3，物理通道数按 8 对齐。地址生成覆盖空间尾块、通道尾块和跨行访问，输出坐标与局部缓冲位置在整个算子执行期间保持一致。

2.3.8 辅助算子单元

辅助算子单元完成网络中的最大池化、尺度对齐加法和最近邻上采样，并通过 DMA 及局部缓冲访问张量。最大池化按核与步长选择最大值，存在边界填充时使用数据类型的最小值。Add 先将两个输入转换到约定尺度，再相加并重定标；2 倍上采样通过源坐标映射复制像素。

张量重排由导出布局、地址步长或显式 DMA 复制实现，数据顺序与模型算子表一致。这些操作在 NPU 的数据通路内执行，CPU 只提交算子参数和处理完成事件。

2.3.9 定点后处理单元

定点后处理接收 INT32 累加结果，完成 Bias 相加、整数系数乘法、移位、舍入与饱和。普通特征图输出重定标为 INT8，检测头保留 INT16 输出精度。ReLU 或线性旁路由算子参数选择。权重采用逐输出通道尺度，激活采用逐张量尺度，相关系数随模型包装载。

BN 在线下融合到权重与 Bias，运行时使用融合后的整数参数。中间计算保留足够位宽，输入边界、舍入中点、饱和上下限和多块累加使用同一整数参考核对。检测框解码与 NMS 由 CPU 执行，与本单元的张量重定标分开。

2.3.10 寄存器接口

寄存器空间基址为 `0x2000 0000`，提供版本、能力、CONTROL、STATUS、完成/错误中断、错误码、算子编号和帧编号。参数区记录张量地址、形状、步长、卷积模式、量化地址与输出格式。软件先写参数，再写 START；硬件仅在空闲状态接受启动，并原子锁存整组参数。

DONE 表示当前算子输出已写入存储；CPU 核对帧编号及算子编号后继续调度。中断状态使用写 1 清除方式，BUSY 期间软件不改写活跃参数。总周期、有效计算周期和 DMA 等待周期按算子计数，软件以 64 位变量汇总网络统计。

2.4 系统互连与存储

2.4.1 总线结构

CPU 采用 32 bit 地址、64 bit 取指返回和 32 bit 数据访问接口，经本地存储路由及 Cache 连接 SoC 互连，并由桥接通路完成 Cache 行填充事务。控制寄存器采用 32 bit AXI4-Lite，数据 DMA 采用 128 bit AXI4，在 DDR 封装处适配厂商 IP 的用户位宽和时钟。AXI 采用受限的顺序事务组织，每个主端读写各最多保留一个未完成事务。

跨时钟域通路在明确的桥接边界使用异步 FIFO 或握手机制。复位和错误恢复先停止接纳新任务，再处理在途事务与旧缓冲引用，避免将旧响应写入已重新分配的帧槽。

2.4.2 存储组织

CPU 启动存储包括 8 KiB Boot ROM 和 32 KiB SRAM。板载 DDR3 物理容量为 1 GB，软件映射其中 128 MiB 窗口：32 MiB 供 CPU 私有程序和数据使用，96 MiB 作为不可缓存共享区。系统地址映射按下表组织，地址属性同时用于 CPU 访存检查、驱动和链接配置。

表 2-6 系统地址映射表

区域	基址	容量或用途
Boot ROM	`0x0000 0000`	8 KiB，启动代码
片上 SRAM	`0x0001 0000`	32 KiB，启动栈及片上数据
定时器	`0x0200 0000`	定时器及软件中断
基础外设	`0x1000 0000`	UART、GPIO、I^2C 等及外部中断控制
NPU 控制	`0x2000 0000`	算子参数、状态及计数
Video 控制	`0x2001 0000`	采集、显示提交、释放及状态
系统信息	`0x2002 0000`	构建信息、时间戳和错误计数
CPU 私有 DDR	`0x8000 0000`	32 MiB，可缓存
共享 DDR	`0x8200 0000`	96 MiB，不可缓存

共享区中，帧槽 A、B、C 的基址分别为 `0x8200 0000`、`0x8220 0000` 和 `0x8240 0000`，每槽 2 MiB。原图位于槽内偏移 0，有效数据量为 1,440,000 B；预览图位于偏移 `0x160000`，有效数据量为 360,000 B。头像素材区从 `0x8260 0000` 开始，容量 2 MiB。

检测输入区从 `0x8280 0000` 开始，三槽基址依次为 `0x8280 0000`、`0x8285 0000` 和 `0x828A 0000`，每槽预留 320 KiB，有效张量为 307200 B。检测输出区从 `0x8290 0000` 开始，占 2 MiB，按模型包导出的输出字节数和对齐要求分配各帧区域。320×320 输入对应 2100 个预测位置，每位置 16 个 16 bit 值时共 67200 B，槽容量还需覆盖元数据及对齐空间。权重、量化参数及查找表从 `0x82B0 0000` 开始，占用 8 MiB；中间张量工作区从 `0x8330 0000` 开始，占用 64 MiB；余下 13 MiB 用于装载、测试和扩展。跨层张量按生命周期分配，帧槽、输入输出槽、素材及模型工作区分别管理。

2.4.3 访问仲裁

CPU、NPU 和视频 DMA 共享 DDR。显示 FIFO 进入危险水位时优先补充显示数据，正常状态采用有界轮询，避免持续大突发长期占用总线。仲裁保留每笔事务的主端归属，并将响应、背压和错误返回到原请求方。

带宽统计分别使用实际采集率和显示扫描率。原图、预览图及模型输入的写流量按被接纳的完整帧计数，全屏显示读流量按原图扫描率及行缓冲的实际读取量计算。并发验证同时检查平均吞吐、最大等待和 FIFO 最低水位。

2.4.4 共享数据访问约定

CPU 私有程序、栈和检测候选工作区允许缓存，DMA 共享张量、帧数据和 MMIO 使用不可缓存映射。共享数据没有可缓存别名，避免 DMA 写入后 CPU 继续命中旧副本。生产者完成数据写入并执行所需屏障后发布就绪事件，消费者完成读取后才交还缓冲区。

采集完成等待三条写入分支的成功响应，NPU 完成等待当前算子全部输出写响应，显示切换等待安全帧边界，旧槽释放等待所有显示 DMA、行缓存预取和 FIFO 引用排空。缓存属性、总线完成和帧所有权共同决定数据何时可读、可写。

2.5 图像采集与显示链路

图像采集与显示链路连接外部视频源、DDR3 存储器和系统计算模块，完成视频接收、输入预处理、帧缓存读写及显示合成，整体结构如图2-11所示。MS7200 将 HDMI 输入转换为并行 RGB888 像素，FPGA 从同一源帧生成原图、预览图和模型输入三类数据并写入 DDR；显示端按行读取已提交的原图，叠加同帧检测框、关键点或头像，经 MS7210 输出 800×600 全屏画面。

像素处理和数据搬运由 Video 硬件及 DMA 完成，CPU 负责帧调度、结果组织和显示提交，NPU 从输入槽读取张量。原图（RAW）、检测标记（BOX）和头像叠加（AVATAR）三种模式共用原分辨率底图，模式与叠加参数在有效图像边界整体切换。400×300 预览分支保留独立的小尺寸图像数据，供预览和数据检查使用。

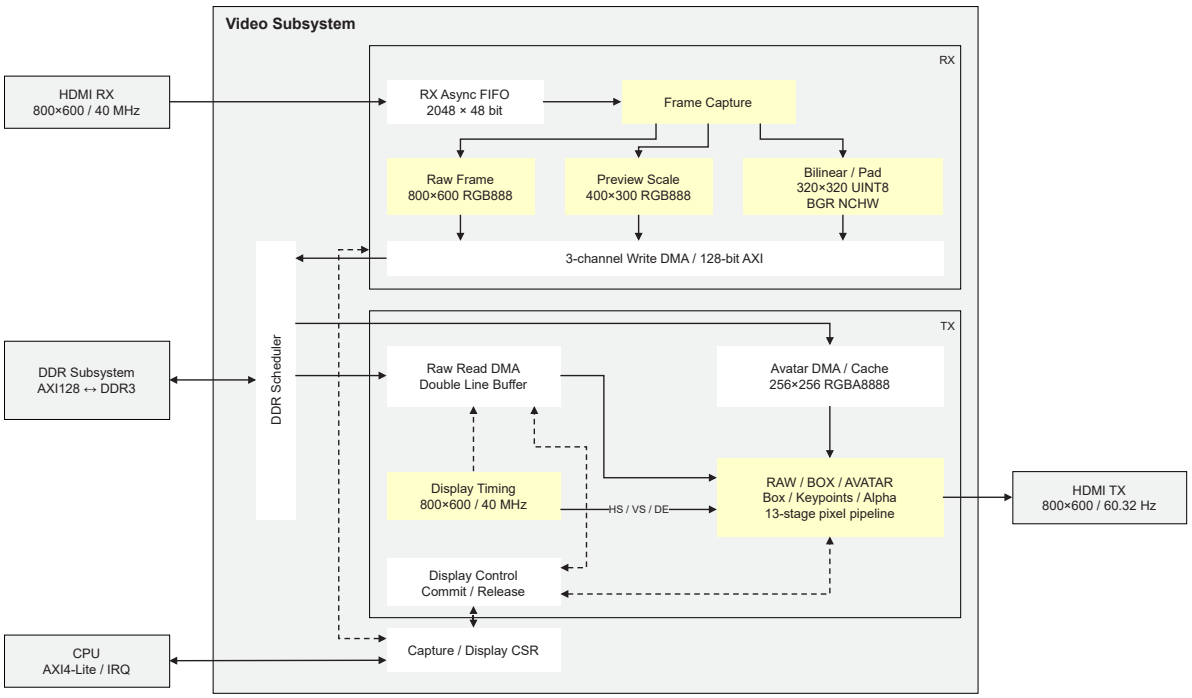


图 2-11 视频子系统结构框图

2.5.1 图像采集

HDMI 接收端提供 24 bit RGB888 像素，以及输入像素时钟、行同步 HS、场同步 VS 和数据有效 DE 信号。HS、VS 用于识别行场边界，DE 限定有效图像区；消隐期间的数据不写入图像缓冲区。板级 I^2C 控制器先配置 MS7210 发送端，再配置 MS7200 接收端。时钟锁定和芯片初始化完成后，入口继续检查输入光栅，并经过默认 128 个连续有效帧的预热过程；DDR 初始化、时钟和接口状态满足要求后，采集控制器才启动写入任务。输入与采集参数见表2-7。

表 2-7 视频输入与采集参数

项目	配置	功能
输入有效区	800×600	每帧 480000 个有效像素
像素格式	RGB888，24 bit	三个颜色分量均为 8 bit 无符号数
输入像素时钟	标称 40 MHz	接收随源输入的像素与同步信号
核心处理时钟	125 MHz	预处理、DMA 与存储访问控制
边界与有效信号	HS、VS、DE	行场边界及有效区标记
入口跨域缓冲	2048 项，48 bit/项	像素、行尾及帧结束信息成包传递
完整帧条件	几何、计数及校验正确	与三路 DMA 写响应共同判定完成

采集请求锁存槽号、帧号及预处理版本，入口等待下一完整有效帧的起点，再接收该帧。像素包携带 RGB 数据和行尾标记，帧结束包携带有效性、CRC 及入口缓冲占用信息。核心域独立核对行尾位置、帧尾状态和原图 CRC，并分别统计原图 480000 个像素、预览图 120000 个像素和模型有效图像区 76800 个像素。完整帧检查和三路 DMA 写入结果全部成功后才发布完成事件，完成记录同时给出任务身份、有效字节数及预处理版本。

接收像素域与核心域通过异步 FIFO 连接。核心域仅在原图、预览和模型输入三条路径均可接收当前像素时共同推进，保持三类输出来自同一源帧。HDMI 源按外部像素时钟连续发送，FIFO 用于吸收有限速率波动；入口溢出、尺寸或同步异常、CRC 错误、DMA 故障及 DDR 就绪丢失均锁存首错原因。发生故障后停止发布有效完成结果，已接纳的存储事务继续排空；端口清空、旧事件处理及重新装载条件满足后恢复采集。

2.5.2 输入预处理

原图分支保持 800×600 和 RGB888 格式，用于全屏显示；预览分支生成 400×300 RGB888；模型分支生成 320×320 UINT8 BGR NCHW 张量。三条路径共享帧身份，分别组织输出和 DMA 写入事务。

预览分支采用 2×2 均值降采样。对每个颜色分量，将相邻两行、两列的四个像素

求和并执行整数舍入：

$$P_{\text{preview}} = \left\lfloor \frac{P_{00} + P_{01} + P_{10} + P_{11} + 2}{4} \right\rfloor .$$

行缓存保存相邻行的中间结果，避免在片上缓存完整输入帧。

模型分支将 800×600 图像按横纵均为 2/5 的比例缩小为 320×240，再在顶部和底部各补 40 行零值，保持有效图像的 4:3 宽高比。固定 5:2 双线性结构以五个源位置为一组生成两个目标位置，水平和垂直方向交替使用 1:3 及 3:1 的权重。行缓存保存水平加权和，再完成垂直组合；分级截断和最终舍入顺序与整数参考一致。补边在写入路径中显式生成，每次任务均写入全部补零区域。

输入张量形状为 $[1, 3, 320, 320]$，内存按 B、G、R 三个平面顺序存放，数据类型为 UINT8，尺度为 1，零点为 0。Video 保留 0 至 255 的像素数值，模型内部的量化及算子重定标由模型包定义。每个平面包含 102400 B，每帧共 307200 B；行跨度为 320 B，平面跨度为 102400 B。像素坐标为 (x, y)、通道编号为 c 时，输入槽内字节偏移为

$$\Delta A = c \times 102400 + y \times 320 + x, \qquad c \in \{0, 1, 2\}.$$

其中 $c = 0, 1, 2$ 分别对应 B、G、R，$0 \le x, y < 320$。原图和预览图按交错 RGB 字节打包，模型输入按通道平面和行组织，三路写 DMA 经 128 bit 接口和轮转仲裁器共享存储写端口。CPU 核对完成事件中的帧身份、字节数和预处理版本，并按共享存储访问约定确认可见性，再发布输入就绪并启动 NPU。

2.5.3 帧缓存管理

帧缓存放置在 DDR3，片上保留跨域 FIFO、缩放行缓存和显示行缓冲。公共配置分配三个 2 MiB 帧槽和三个 320 KiB 模型输入槽；帧槽内保存原图及预览图，模型输入使用独立输入槽。有效数据量与预留容量分开管理，三类数据的组织见表2-8。

表 2-8 图像数据与缓存组织

数据对象	尺寸与格式	有效数据量	槽内布局
原图	800×600 RGB888	1,440,000 B	帧槽偏移 0，行跨度 2400 B
预览图	400×300 RGB888	360,000 B	帧槽偏移 `0x160000`，行跨度 1200 B
模型输入	320×320×3 UINT8 BGR NCHW	307,200 B	独立输入槽，行跨度 320 B

Video 本地存储通路将采集写入和显示读取汇合到同一 DDR 接口，桥接层完成 128 bit 访问与 HMIC 256 bit 用户接口之间的转换。显示供数紧急时优先服务读请求，连续读突发默认限额为 4，使等待中的采集写入能够获得服务；正常状态按有界轮转调度。系统互连进一步组织 CPU、NPU 和 Video 的跨主端仲裁、地址属性与写入可见性。

槽号标识物理位置，帧号关联原图、模型输入和结果，代次区分复位及槽复用前后的任务，预处理版本限定输入数值含义。CPU 帧池依次维护空闲、采集、就绪、计算、待显示、显示提交等待、正在显示和退休状态；故障帧进入失效状态。整图计算所有权保持至最终输出、后处理及工作区释放完成，显示端独立保持对当前原图和叠加参数的引用。

显示记录被接收、画面实际切换和旧槽释放分别产生事件。新帧在安全图像边界生效后，旧帧仍需等待读事务、行缓冲和像素流水中的引用退休。CPU 核对释放事件中的槽号、帧号和代次，确认成功后才将对应槽恢复为空闲。无空闲槽时等待释放或显式放弃允许丢弃的待处理帧；恢复采集前统一处理旧事务、事件和任务代次。

2.5.4 头像叠加

头像素材采用 256×256 RGBA8888，每张占 256 KiB，Alpha 为 8 bit 非预乘透明度。CPU 根据主人脸框、素材锚点和覆盖系数生成目标矩形，提交素材标识、版本及目标位置。素材装载器从 DDR 读取完整纹理并核对 CRC，全部像素校验成功后封存片上缓存，显示记录通过素材身份与就绪检查后使用该纹理。

最近邻映射将目标矩形内的显示坐标转换为纹理坐标，边界裁剪限制实际输出范围，采样比例保持以未裁剪矩形为基准。对每个颜色分量，背景颜色 C_{raw} 与素材颜色 C_{avatar} 按下式混合：

$$C_{\mathrm{out}} = \left\lfloor \frac{\alpha C_{\mathrm{avatar}} + (255-\alpha) C_{\mathrm{raw}} + 127}{255} \right\rfloor .$$

$\alpha = 0$ 时保留背景，$\alpha = 255$ 时输出素材颜色。最近邻纹理采样与模型输入双线性缩放分别使用各自的坐标及舍入规则。合成通路采用 13 级像素流水，RGB、HS、VS、DE、坐标与帧身份同步延迟，输出像素直接送往 HDMI 发送端，省去整帧叠加图的 DDR 写回。

2.5.5 显示输出

显示读取器依据已提交记录中的原图地址、行跨度和帧身份按行获取 RGB888 数据。每行 2400 B，对应 150 个 128 bit 数据拍；核心域预取下一行，像素域读取当前行，双行缓冲衔接突发访问和连续扫描。新帧数据准备完成后在有效图像起点切换，准备不足时保留已有有效帧。扫描期间检测供数不足和读响应错误，并锁存故障状态。

输出像素时钟为 40 MHz，800×600 光栅参数见表2-9，水平和垂直同步均为正极性。扫描频率为

$$f_{\mathrm{scan}} = \frac{40 \times 10^{6}}{1056 \times 628} \approx 60.32\ \mathrm{Hz}.$$

扫描频率按重复光栅计数，新图更新率按不同采集帧计数，模型处理率按完成整图推理计数。

表 2-9 显示光栅时序参数

时序项目	水平方向（像素时钟）	垂直方向（行）
有效区	800	600
前肩	40	1
同步脉宽	128	4
后肩	88	23
总周期	1056	628
同步极性	正极性	正极性

RAW 模式显示完整原图，BOX 模式叠加最多 16 个人脸框及每框 5 个关键点，AVATAR 模式将主人脸头像映射到目标矩形。主目标框、其他框和关键点分别使用红、绿、黄三种颜色。显示坐标直接使用 800×600 原图坐标，图像与有效检测结果必须匹配同一帧身份。有效无脸结果关闭框和头像，保持原图；模式切换复用已提交的同帧结果。

CPU 通过显示接口写入 640 B 记录，包含原图地址、行跨度、槽身份、模式、检测框、关键点及头像参数。硬件检查记录、地址范围和素材状态，跨域握手期间保持整组参数稳定，在有效图像起点整体应用。实际像素输出的提交反馈与旧引用的退休反馈分别返回核心域，由显示所有者管理提交及释放事件。KEY1/MODE 经去抖后按 RAW、BOX、AVATAR、RAW 循环请求，KEY0 协调复位并恢复默认 RAW 模式。

2.6 模型部署与应用软件

2.6.1 模型选型

系统采用 OpenCV Zoo 的 YuNet 人脸检测网络，以`face_detection_yunet_2023mar.onnx` 为权重来源。模型输出人脸框、分数及五个关键点，固定输入逻辑形状为 $[1, 3, 320, 320]$，其中有效图像为 320×240，上下各补 40 行零值。单次推理结果供全屏 BOX 与 AVATAR 模式复用，预设头像由软件配置，切换显示模式复用同一帧结果。

部署包记录权重来源、版本和哈希，以及固定形状计算图的算子、张量和输出顺序。输入尺寸、预处理、量化和后处理参数作为同一配置管理，浮点与整数参考采用相同的输入画面和候选筛选规则。

2.6.2 量化方案

外部输入采用 UINT8 BGR NCHW，保留 0 至 255 像素值，尺度为 1、零点为 0。网络内部权重采用对称 INT8、逐输出通道尺度，激活尺度和输出格式由模型包定义。Bias 和累加器采用 INT32，检测头采用 INT16 原始回归值或概率变换前输出。BN 融合、重定标系数、移位及饱和参数在线下生成，Add 的两个输入先完成尺度对齐。

离线工具以代表性图像进行 PTQ 校准，记录逐层浮点与整数误差，硬件按整数参考中的舍入及饱和规则执行。模型输入使用固定 5:2 双线性缩放、BGR 顺序和上下各 40 行补零，校准、整数参考和视频前端使用相同的采样及取整顺序。

2.6.3 模型导出

导出流程生成固定形状算子表、权重、Bias、量化参数、查找表及张量工作区布局。逐层记录输入输出形状、有效与存储通道、核尺寸、步长、补边、分组、搬运字节数和生命周期。模型输入使用 B、G、R 三平面 NCHW 布局，中间张量的物理布局、对齐和步长由算子表逐项指定。

模型包包含版本、哈希、预处理配置、输出格式和逐层参考向量，CPU 使用导出的 C 算子表逐项调度。算子表还记录每层的输入输出区域，跨层连接在最后消费者完成前保留原张量。素材包独立记录 RGBA 像素、尺寸、锚点、颜色顺序和校验信息，与模型包共同进行启动检查。

2.6.4 算子映射

普通与逐点卷积使用输入通道归约模式，深度卷积使用通道独立模式；池化、Add 和上采样由 NPU 辅助单元执行。CPU 处理输出候选、定点概率变换、框解码和 NMS，Video 承担输入逐像素处理及显示合成。算子执行方式如下表所列。

表 2-10 算子执行方式表

算子或阶段	执行单元	执行方式	数值约定
普通/逐点卷积	NPU	输出及输入通道分块	INT8 乘法、INT32 累加
深度卷积	NPU	通道独立、空间并行	各通道部分和独立
池化、Add、上采样	NPU	辅助通路和局部缓冲	边界、尺度及采样按算子表
Bias、重定标、激活	NPU	计算输出后处理	INT8/INT16 输出，显式饱和
概率变换、解码、NMS	CPU	有限候选、定点查表	输出尺度统一，检查中间位宽
缩放、输入格式化	Video	流式逐像素处理	UINT8 BGR NCHW、上下补零
全屏叠加与头像混合	Video	行缓存和素材预取	RGB888、非预乘 Alpha

2.6.5 检测后处理

YuNet 按 stride 8、16、32 输出 cls、obj、bbox 和 kps，320×320 输入对应 1600、400 和 100 个位置，共 2100 个预测位置。每位置包含 1 个分类值、1 个目标值、4 个框参数和 10 个关键点坐标。CPU 先进行概率变换与阈值筛选，再对保留候选解码；Sigmoid 及指数使用定点查找表。

参考分数为 $\sqrt{cls \cdot obj}$。在概率非负且尺度统一的条件下，筛选和排序使用乘积及阈值平方，减少逐候选开方。NMS 按约定 Top-K 范围处理候选，IoU 比较和坐标计算保留足够中间位宽。检测坐标扣除补边后恢复到原始 800×600 画面：

$$x_{\text{raw}} = \frac{5}{2} x_{\text{net}}, \qquad y_{\text{raw}} = \frac{5}{2}(y_{\text{net}} - 40).$$

框采用左闭右开范围，左上角向下取整、右下角向上取整，再裁剪至原图边界；删除零面积框及补边区无效候选。

主人脸优先选择有效面积最大者，同面积时按分数和固定候选顺序决定。CPU 输出框、五点、主人脸索引、头像位置与尺寸，BOX 模式最多显示 16 个人脸框，AVATAR 模式叠加一张主人脸头像。图像坐标、显示坐标和素材坐标分别保存，随同帧编号一起提交。

2.6.6 裸机运行时

软件从 Boot ROM 启动，在 SRAM 建立栈并初始化 UART，随后完成 DDR 初始化与自检、程序和模型素材装载、版本及校验值检查。HDMI 固定模式配置后先输出本地背景，初始化三帧槽及检测输入补边，再开放采集和推理。模型准备与装载属于启动阶段，运行循环在板上独立执行。

驱动统一封装 NPU、Video、定时器和中断接口，公共地址、寄存器及清单定义生成硬件与软件使用的数据。帧记录维护地址、状态、算子编号、结果及时间戳；中间张量按导出生命周期分配，CPU 候选工作区与 DMA 共享区分开。状态统计低频输出到 UART 和屏幕，避免逐帧日志影响处理节拍。

2.6.7 帧调度

软件使用 `graph_active` 和 `frame_owner` 管理整图计算所有权。完整帧三分支写入结束后进入 READY；当没有活跃整图任务且 NPU 空闲时，CPU 锁定该帧和共享模型工作区，开始派发首个算子。每次完成事件均核对帧编号和算子编号，随后只继续当前帧的下一算子。

最后算子输出写回后，CPU 完成定点后处理与头像参数计算，发布结果就绪事件并将帧置为待显示状态。确认工作区没有本帧消费者后释放计算所有权，允许处理下一帧。算子之间短暂的硬件 IDLE 不改变整图所有权，防止不同帧覆盖共享中间张量。

CPU 提交显示记录后将帧置为显示提交等待状态，收到实际像素生效事件后转为正在显示状态。旧显示槽经引用退休并收到释放确认后才可再次采集。采集、检测完成和实际显示分别计数；处理能力不足时跳过未接纳输入或释放尚未被消费的过期帧，并记录原因。采集下一帧、处理当前帧、显示上一完成帧通过同一套所有权协议重叠运行。

2.6.8 错误恢复

启动阶段检查 DDR、模型、素材及接口版本，检查失败时保留串口和本地背景输出。运行阶段记录输入失锁、尺寸错误、DMA 错误、NPU 超时和帧事件不匹配，相关帧进入 INVALID。有效无脸结果正常发布，推理错误与无脸结果采用不同状态。

恢复过程停止接纳新计算任务，处理在途 AXI 事务，清空旧 FIFO 和帧元数据，确认共享工作区及显示引用释放后重新初始化。没有有效结果时输出背景，已有有效显示帧可保留并附错误状态。复位、重新装载和恢复后的任务使用新的帧上下文，旧事件不能推进新任务。

2.7 FPGA 实现

FPGA 顶层集成 SoC、时钟复位、DDR 及板级接口封装，模块组成如图2-12所示。`triha_soc` 封装图2-2中的六个模块，通过 DDR 和 HDMI 封装连接板外存储与视频设备。

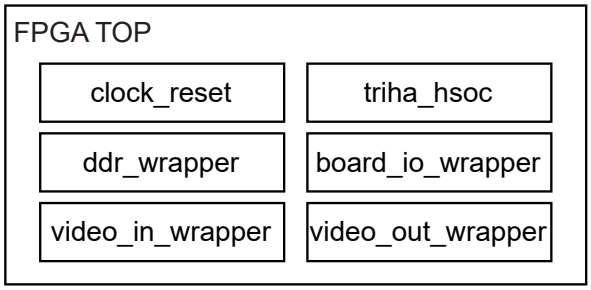


图 2-12 FPGA 顶层模块图

`clock_reset` 生成主逻辑与视频输出时钟，并控制各域复位释放。`ddr_wrapper` 封装厂商 DDR3 IP、初始化状态和用户接口；`board_io_wrapper` 连接 UART、GPIO、SPI 和 I^2C 等板级信号。`video_in_wrapper` 接入 HDMI 接收像素及同步信号，`video_out_wrapper` 连接全屏合成后的像素和同步信号。厂商原语及板级适配集中在封装层，SoC 保留处理、互连和应用逻辑。

2.7.1 目标平台

本设计采用盘古 100pro+ 开发板，主控为紫光同创 Logos-2 系列 PG2L100H-6IFBG676。CPU、NPU 和视频逻辑使用片上 LUT、FF、APM 及 DRM 实现，平台主要配置见表2-11。板载 DDR3 由两颗 4 Gbit 芯片组成，总容量 1 GB，数据通路 32 bit；模型参数、中间张量、三帧槽和头像素材位于软件规划的 DDR 窗口中。

表 2-11 平台配置表

项目	配置	说明或用途
开发板	盘古 100pro+（MES2L676-100HP）	FPGA 及板级外设平台
主控器件	PG2L100H-6IFBG676	紫光同创 Logos-2 系列 FPGA
逻辑资源	66,600 个 LUT，133,200 个 FF	CPU、NPU 及系统控制逻辑
运算资源	240 个 APM	乘法与 MAC 运算
片上存储	155 个 36Kbit DRM	CPU 存储、Cache 及 NPU 缓冲区
外部存储	1GB DDR3，2 颗 4Gbit 芯片，32bit 数据位宽	模型参数、特征图及视频帧存储
视频接口	1 路 HDMI 输入、1 路 HDMI 输出	视频接入与显示
通信接口	USB-UART	板载通信与串口调试
辅助接口	按键、LED、SD 卡接口	控制输入、状态显示及扩展存储
下载与供电	JTAG、12V 电源接口	比特流下载、逻辑调试和板卡供电

2.7.2 工程配置

工程以 SoC、CPU、NPU 和 Video 独立验证顶层共用一份 RTL，板级目录集中管理 DDR、HDMI、PLL、引脚和时序约束。CPU Cache 与分支预测开关、NPU 并行度、地址映射及视频模式由公共配置传递到 RTL 与软件。构建流程依次生成存储初始化文件、综合网表、布局布线结果和位流。

每个运行版本记录源码提交、工具与 IP 版本、硬件配置、固件、模型及素材哈希。软件装载地址与链接布局相互核对，位流、固件和数据包按同一接口版本配套使用。

2.7.3 时钟复位

CPU、NPU 和 SoC 控制逻辑使用主逻辑时钟，提供 50 MHz 调试与 100 MHz 配置；Video 预处理、DMA 及本地存储控制使用 125 MHz 核心时钟。HDMI 输入使用接收侧像素时钟，输出使用 40 MHz 像素时钟；DDR 用户时钟按厂商 IP 配置。相关异步输入分别同步，像素数据跨域使用异步 FIFO，多位地址、帧身份和显示记录使用保持数据束及握手。

复位在各本地域同步释放。CPU 可先从 ROM/SRAM 启动，DDR 初始化成功后才开放外存 DMA，视频输出先产生本地背景，再接受完整有效帧。失锁和整机重初始化同时处理旧事务、FIFO 与帧所有权。

2.7.4 板级接口

HDMI 一入一出均采用 800×600 固定模式，接收和发送芯片通过板级控制接口完成初始化。DDR3 接口连接板载 32 bit 存储通路，USB-UART 承担装载、诊断与性能统计。按键和 LED 提供运行控制及状态指示，JTAG 用于 FPGA 配置和逻辑调试。

板级封装根据原理图定义引脚方向、电平和 I/O 标准，输入输出像素与同步信号保持对应时序。程序、权重和 RGBA 素材通过装载工具写入约定地址，运行期间由 FPGA

执行处理和显示。

2.7.5 实现约束

约束覆盖主逻辑、DDR 用户、输入像素和输出像素时钟，并定义 PLL 生成关系、板级输入输出延迟和复位同步路径。跨域例外对应具体 FIFO 或握手结构，数据总线与控制信号按其同步方式设置约束。布局布线后分别分析各时钟域的建立保持裕量、关键路径和跨域结构。

资源报告统计 CPU、Cache、NPU、Video、DDR 及互连占用，Fmax 与实际闭合工作频率分别记录。完整 SoC 的时序分析同时考虑视频扫描供数、DMA 争用和控制路径，测量数据在第三部分汇总。

第三部分 完成情况及性能参数

3.1 整体介绍

TriHaRiscv 集成了六级单发射 CPU、NPU、AXI 互连、片上与外部存储，以及 HDMI 采集和全屏显示链路。裸机程序运行 YuNet 人脸检测，完成定点后处理、主人脸选择和头像参数生成，单路输出支持同帧原图、检测标记和头像叠加三种模式。工程成果按硬件、软件、功能验证和应用演示组织，性能参数按 CPU、NPU、系统及 FPGA 实现分别统计。

3.2 工程成果

3.2.1 硬件成果

硬件包括六级流水线 CPU、两路组相联 I/D Cache、动态分支预测、机器异常中断，以及 64/128 通路 NPU。NPU 具备普通卷积、深度卷积、辅助算子、定点后处理和 DMA；Video 具备完整帧采集、预览降采样、双线性缩放及补边、BGR NCHW 输入组织、原图行缓冲及 Alpha 混合。SoC 通过 AXI 连接处理单元、视频 DMA 和 DDR，板级封装统一管理时钟、复位及接口。

3.2.2 软件成果

软件由启动程序、外设驱动、NPU 逐算子运行时、检测后处理、帧调度和诊断统计组成。模型工具生成固定 YuNet 图、INT8 权重、量化参数、算子表及张量布局，素材工具生成 RGBA 头像与锚点清单。程序、模型和素材使用版本及校验信息配套装载，运行循环按帧事件调度采集、计算和显示。

3.2.3 功能验证

功能验证覆盖 CPU 指令与退休行为、Cache 和预测恢复、NPU 单算子及整网数值、AXI 完成语义、像素处理和帧所有权。系统联合测试核对输入、推理结果和全屏显示记录的同一帧编号，并注入输入失锁、DMA 错误、超时和重初始化事件。数值验证使用同形状、同预处理、同量化和同后处理配置的参考结果。

3.2.4 应用演示

应用接入 800×600 HDMI 视频，通过 RAW、BOX 和 AVATAR 模式显示完整原图、人脸框与五个关键点，以及主人脸的预设动漫头像。无脸时保留原图；多人场景按面积、分数和固定顺序选择主人脸。运行状态分别统计采集、检测、新图更新率和错误

计数。

Video 独立板级验证使用 `video_live_mode_board` 目标，接入动态 HDMI 源，核对三种模式的连续背景、按键切换及协调复位。BOX 使用人工生成的框和关键点，AVATAR 使用 256×256 RGBA 算术测试纹理。这些输入用于检查视频数据通路、模式控制与逐像素合成，板测数据见表3-1。

表 3-1 Video 独立板级验证结果

检查项目	结果	统计范围
图像与时钟	800×600；40/125 MHz	输出像素时钟/核心时钟
显示扫描频率	约 60.32 Hz	1056×628 光栅
新图更新率	30.1548 fps	复位后 84 s 生命周期采样窗口
连续健康观察	87.000 s	复位后的持续观察时长
模式与复位	三次模式切换、一次复位通过	KEY1 循环三模式，KEY0 恢复 RAW
欠载、溢出及错误	均为 0	已记录的连续三模式及复位观察
LUT / FF	16770 / 18150	整个独立 Video 板级目标
片上存储	73.5 个 DRM36K 等效块	含 64 个测试纹理缓存块
Slow setup / hold	+0.063 / +0.129 ns	实际后布线时序报告
Fast setup / hold	+2.911 / +0.097 ns	实际后布线时序报告
配置下载与校验	Program / Verify 通过	与时序报告对应的板级镜像

板测记录与机器证据保存于`docs/verification/video/live_modes.md` 及其关联证据文件。新图更新率按采集帧身份变化计数；扫描频率按输出光栅计数。真实人脸检测应用的有效输出帧率需包含 NPU 整图推理和 CPU 后处理，单独统计。

3.3 特性成果

3.3.1 CPU 性能

CPU 性能采用 CoreMark 1.0，分别评估 TCM 本地存储和 L1 Cache 配合板载 DDR 两种配置的实际性能。TCM 由直接连接 CPU 本地访问路径的 32 KiB 片上 SRAM 实现，程序、工作数据和栈均驻留其中，不经过 I/D Cache 及 DDR 控制器。L1 Cache 配置将程序、数据和栈放在私有 DDR 区，通过各 16 KiB 的 I/D Cache 访问；两者均为两路组相联、32 B 缓存行，D-Cache 采用写回、写分配及成功加载返回重叠配置，未命中时访问板载 DDR。两种配置均在 PG2L100H 上运行六级单发射 RV32IM/CSR 核，BTB/BHT 各 512 项、RAS 为 4 项，GShare 关闭，CPU 标称频率为 100 MHz。软件均使用 GCC 15.2、`-O2` 优化、ILP32 ABI 和 5000 次迭代，不启用 LTO，板测结果见表3-2。

表 3-2 TCM 与 L1 Cache 配置的 CPU 板级 CoreMark 结果

项目	TCM	L1 Cache ＋板载 DDR
CPU 标称频率	100 MHz	100 MHz
迭代次数	5000	5000
计时区间周期数	1,636,465,356	2,002,053,973
运行时长	16.36465356 s	20.02053973 s
CoreMark	305.536562792	249.743516780
CoreMark/MHz	3.055365628	2.497435168
CPU 核 LUT 分母	6139	6507
CoreMark/LUT	0.049769761	0.038380746

两组完整运行均通过标准 CRC 校验：`seedcrc=e9f5`，链表、矩阵和状态机 CRC 依次为 `e714`、`1fd7` 和 `8e3a`，最终 CRC 为 `bd59`，错误计数和短测标志均为零。运行时长按 CPU 周期数除以标称 100 MHz 换算，均超过 10 s；CoreMark 按迭代次数除以运行时长计算。时钟采用板级配置值，未进行独立仪器校准。TCM 在两种配置中取得最高板测分数，后文以该版本说明片上调试方法。L1 Cache 项在 I/D Cache 开启、D-Cache 采用写回策略的配置下完成板测，反映 CPU 经 L1 Cache 访问板载 DDR 时的整体执行性能。两种配置的存储延迟、访问路径和固件布局不同，以上结果描述各自工作点，不单独用于推导 Cache 开关的性能增益。

图3-1和图3-2分别保留 TCM 与 L1 Cache ＋板载 DDR 的串口实测原始截图，显示编译参数、标准 CRC、精确计时及完成标志。TCM 截图中的 5933 个 CPU 核 LUT 及 0.051497 CoreMark/LUT 为固件保留的原发布版打印值；表3-2按调试构建的 6139 个 CPU 核 LUT 计算资源效率。截图中的整数秒及上游 Iterations/Sec 字段不用于精确评分，表中分数按 `CPU_COREMARK_TIME` 的完整周期数复算。

表3-3按综合报告中的 CPU 核层级统计资源，片上 SRAM、调试核、DDR 及板级逻辑均不计入 CPU 核。各版 CPU 核的 688 个 RAM-LUT 已包含在 LUT 总数中。Cache 及传输控制另列，DRM 以 36 Kbit 块统计。

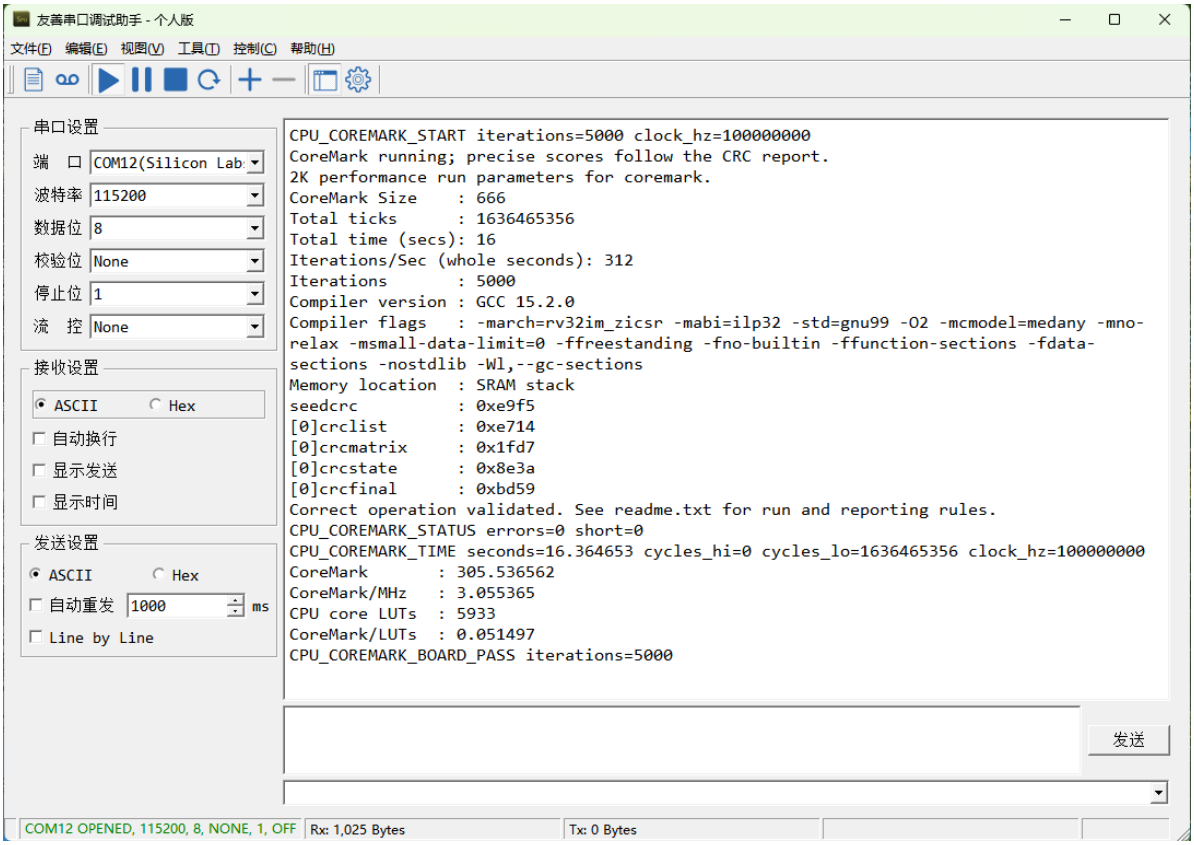


图 3-1 TCM 配置 CoreMark 板测串口原始截图

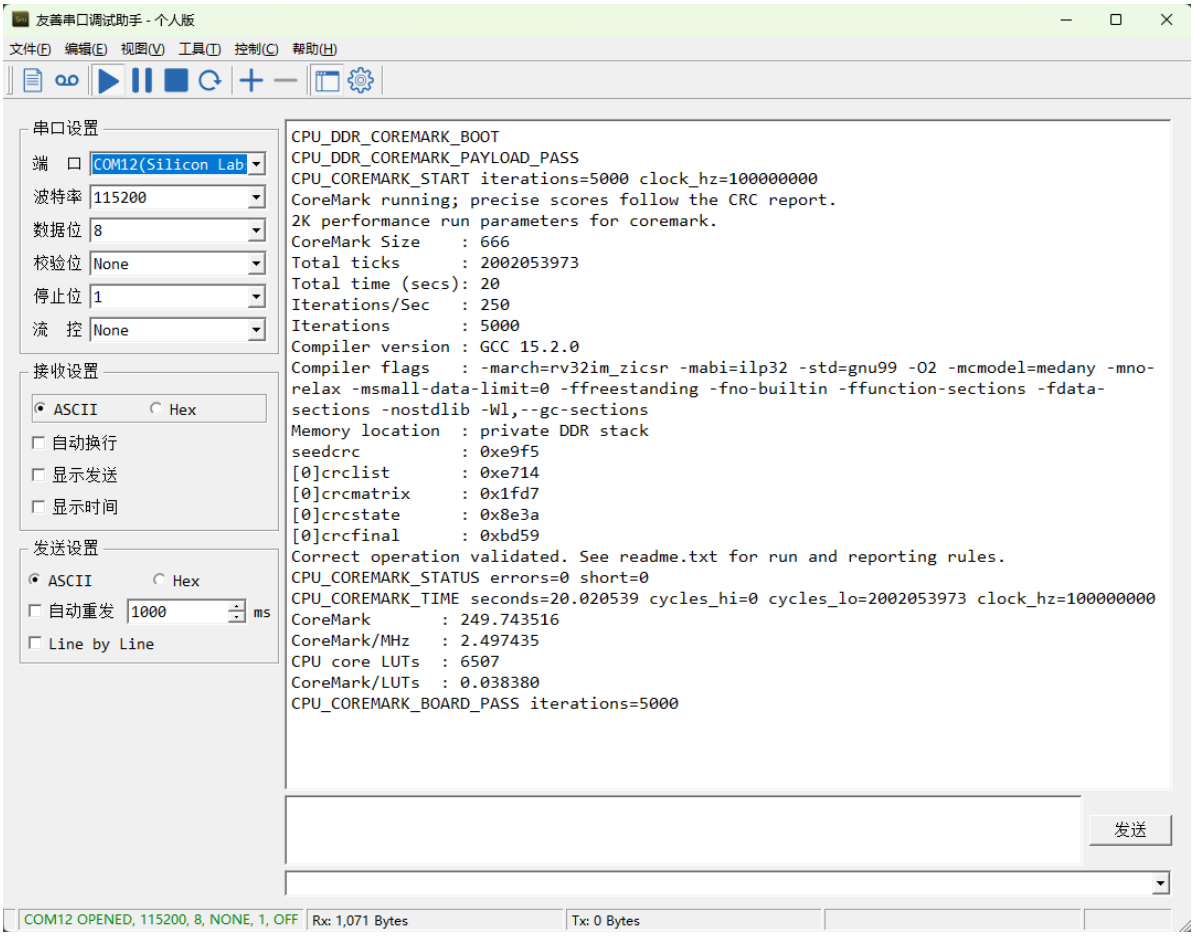


图 3-2 L1 Cache +板载 DDR 配置 CoreMark 板测串口原始截图

表 3-3 CPU 核及缓存资源统计

统计范围	LUT	FF	DRM	APM
TCM 原发布版 CPU 核	5933	3528	0	4
TCM 调试版 CPU 核	6139	3528	0	4
L1 Cache ＋ DDR 版 CPU 核	6507	3562	0	4
L1 写回 Cache 及传输控制	668	807	11	0

TCM 调试版在新综合上下文中得到 6139 个 CPU 核 LUT，CPU 源码和固件与原发布版逐字节相同，板测周期也相同。原固件在计时结束后仍打印历史 5933-LUT 元数据；表3-2使用本次调试构建的 6139 个 LUT 重新计算效率。原无调试核发布包的 5933 个 CPU 核 LUT 和 0.051497819 CoreMark/LUT 保留为该发布包的结果。

TCM 调试版和 L1 Cache ＋ DDR 版分别满足 100 MHz 实际后布线时序，结果见表3-4。TCM 调试版沿用原 100 MHz、I/O 延迟及 hold 约束；四路 LED 均映射到器件输出寄存器，状态显示延后一拍，退出码组合译码不再直接驱动引脚。调试核各时钟与跨时钟检查、复位恢复和移除检查均通过，performance slack 与最差 setup 一致。

表 3-4 CPU 板级最终后布线时序裕量

检查项	TCM 调试版	L1 Cache ＋ DDR 版
Setup，Slow/Fast	+0.366/+3.662 ns	+0.309/+3.551 ns
Hold，Slow/Fast	+0.061/+0.034 ns	+0.151/+0.099 ns
最小脉宽，Slow/Fast	+3.655/+4.202 ns	+3.655/+4.202 ns

100 MHz 是完成时序检查并用于板测的工作点。TCM 调试版最差 hold 裕量为 0.034 ns，更高 Fmax 需在相同完整配置下另行验收。

3.3.2 CPU 流水线效率

片上调试以取得最高板测分数的 TCM 版本为例，采用 PDS DebugCore/Fabric Debugger 关联 CoreMark 函数入口与流水线状态。调试镜像保留已验收的 CPU 及 5000 次`-O2` 固件，仅在板级封装中寄存四路 LED 输出并加入 38 位 DebugCore 探针；实际后布线时序及 Program/Verify 通过后再进行采样。该镜像的完整板测周期与无探针发布版一致，采集完成后保留易失配置，未写入 Flash。采样时钟为 CPU 时钟上升沿。三个触发端口分别为 30 位 WB PC`[31:2]`、1 位 WB 有效和 7 位流水控制状态，合计 38 位，采样深度为 1024 拍。按 32 bit 指令对齐规则在 PC 低位补零，结合 5000 次板测 ELF 定位链表归并排序、矩阵乘法和状态转换的真实函数入口。触发条件同时要求入口 PC 匹配、`valid_wb_q=1` 和 `pipe0_stall_raw_w=0`。

表3-5统计完整采样窗口。WB 事件按有效位与非暂停状态的交集计数，避免指令

保持时重复累加。窗口可能进入被调函数或调用者；这些局部 WB 事件不构成整段 CoreMark 的架构退休 IPC，内存 WB 有效也不等同于 DDR 事务或 Cache 事件。

表 3-5 TCM CPU 三个热点的 1024 拍板上采样

触发算法	触发 PC	WB 事件	内存 WB 有效拍
链表归并排序	`0x00011D48`	951	119
矩阵乘法	`0x000124E8`	964	247
状态转换	`0x00012C1C`	916	182

图3-3、图3-4和图3-5为 Fabric Debugger 最大化窗口的原生波形截图，放大展示各入口附近的局部采样。插图仅裁剪窗口边缘及空白区，保留原生信号名、读值和采样刻度。金色游标 0 为指定函数入口，对应原始 CSV 索引 1；其前一拍为原生刻度 −1，对应 CSV 索引 0。三窗在触发拍均为 WB 有效且非暂停。`wb_pc_word` 为 PC[31:2]，游标读值 `0x4752`、`0x493A` 和 `0x4B07` 分别对应三个字节地址；PC 总线仅在 WB 事件有效时解释为完成指令地址。图中 `valid_e2_q` 表示 E2 在途状态，`squash_e1_e2_w` 表示取消条件，`div_pending_q` 和 `csr_pending_q` 表示对应串行操作未结束，`fetch0_valid_w` 与 `writeback_mem_valid_w` 分别表示前端及内存返回有效。三窗 `pipe0_stall_raw_w` 均为零，但 `valid_wb_q` 仍有空隙；发射空泡、相关性和控制流开销需结合更多前端信号分析，不能只凭该暂停位判断。

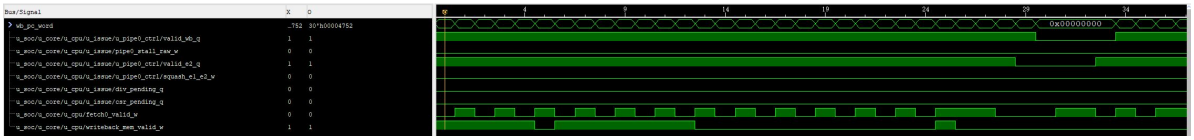


图 3-3 TCM CPU 链表热点的原生采样截图，触发 PC 为 0x00011D48

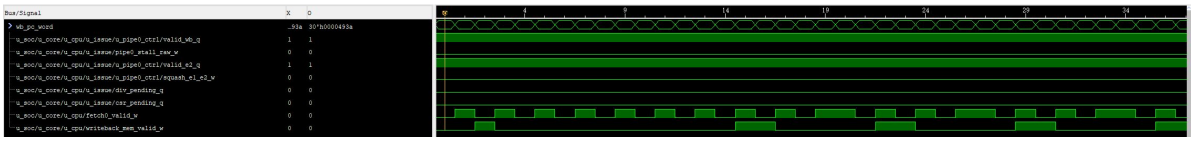


图 3-4 TCM CPU 矩阵热点的原生采样截图，触发 PC 为 0x000124E8

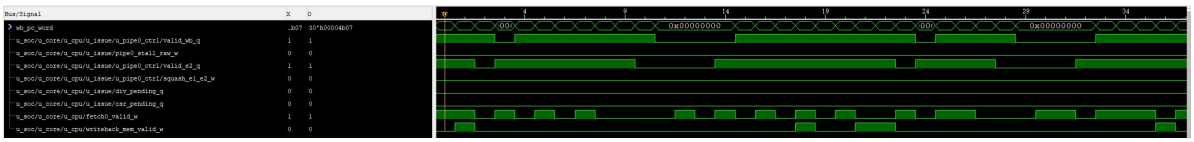


图 3-5 TCM CPU 状态转换热点的原生采样截图，触发 PC 为 0x00012C1C

采样归档包含原生 DPRJ/WDF、逐拍 CSV、触发条件及位流和 ELF 哈希。state 窗口的 38 位通道和 PC 总线在全部 1024 拍上与原生 VCD 逐位一致。原生 VCD 导出的 100 ns 标注属于工具的样本刻度，本节横轴统一使用采样序号；硬件配置的 100 MHz

采样时钟名义周期为 10 ns。本次片上采样与完整板测均属于 TCM 配置，L1 Cache + DDR 版沿用其独立的性能和验证记录。

Cache/DDR 版的整段流水线统计来自 ModelSim 数字 DDR 及 PLL 模型，采用与该版 5000 次板测 CPU/Cache RTL 逐字节一致的配置。软件仍为 GCC 15.2、-O2、无 LTO，仿真执行 5 次迭代，UART 除数仅在测试台中缩短，计时仍来自 CPU 周期 CSR。这项短测保留不足 10 s 提示，用于分析执行过程。2026 年 10 月 28 日的观测重放得到 2,018,576 个计时周期、1,448,453 条 WB 观察序列及 70,840 次 store，标准 CRC 及完整序列哈希均与原记录一致。观测模块不驱动 CPU、Cache 或软件状态。

统计窗口对应软件两次周期 CSR 读取之间的区间。WB 计数包括分支和存储指令，暂停时不重复累加，窗口内异常检查为零。对应的数字 WB 观测 IPC 和 CPI 为

$$IPC_{\mathrm{WB}} = \frac{1,448,453}{2,018,576} = 0.717562, \qquad CPI_{\mathrm{WB}} = 1.393608. \tag{3-1}$$

该数值属于 Cache/DDR 数字模型，既不是 TCM 板测 IPC，也不是 Cache/DDR 整段板测退休计数。

每周期按发射成功、取消、LSU 等待、后端流水等待、除法、CSR、前端无有效指令、执行资源、相关性及其他原因的优先顺序归类。发射成功优先，未发射时取首个阻塞原因；各项互斥且总和等于计时周期数，结果见表3-6。

表 3-6 Cache/DDR 数字 CoreMark 的互斥周期分类

周期类别	周期数	占比（%）
发射成功	1,448,453	71.756179
LSU 等待	358,437	17.756924
后端流水等待	12,993	0.643672
前端无有效指令	98,793	4.894193
相关性等待	99,891	4.948588
CSR 等待	9	0.000446
取消、除法、执行资源及其他类别合计	0	0.000000
总计	2,018,576	100.000000

该数字窗口中，D-Cache 接受 273,477 次读及 70,840 次写，读命中 273,461 次、读未命中 16 次，读命中率为 99.994149%。写命中 70,824 次、写分配未命中 16 次，共 32 次行填充；计时窗口内没有脏行逐出或显式清理，下级写次数为零。高命中率仍伴随 17.756924% 的 LSU 等待，说明请求接纳、同步 RAM 读取和返回节拍也限制吞吐；成功加载返回与下一请求的重叠发生 75,408 次。Cache 与总线状态驻留可以相互重叠，不能与上述互斥类别直接相加。CPU、NPU、Video 并发访存性能仍需按完整系统配置测量。

3.3.3 NPU 性能

NPU 按层记录有效 MAC 数、有效计算周期、填充和尾块开销、DMA 等待、辅助算子周期及总周期。普通卷积、逐点卷积和深度卷积分开统计，64 与 128 通路配置使用相同输入和整数规则比较。DMA 与计算重叠按时间线计算，未重叠的阶段计入总时间。

算术峰值按乘加通路数与实际时钟计算，阵列利用率采用有效 MAC 和测量周期统计。整网时间包括算子派发、实际计算、搬运和算子间控制，不以裸乘法阵列时间代替网络执行时间。

3.3.4 系统性能

有效输出帧率按统计窗口内首次实际显示的不同采集帧数除以窗口时间计算。每帧完成实际推理和后处理，原图、检测结果及显示记录共用一个帧身份，首次实际输出时计一帧；重复扫描和仅模式、状态更新不计入有效帧数。采集率、检测完成率和显示扫描率分别记录，稳态时间包含等待、重复扫描和丢帧造成的间隔。

完整处理时间从 INPUT_READY 计至 RESULT_READY，包含排队、算子执行及 CPU 后处理。板内可见延迟从选中输入帧的有效画面开始，到全屏输出首次显示该帧对应结果的有效像素为止，以实际像素退休事件标记终点。延迟分解覆盖采集与写入排空、等待调度、NPU 执行、CPU 解码/NMS、头像参数生成、等待显示边界及光栅扫描相位。

优化评价以有效输出帧率不低于 24 fps 和板内可见延迟 P95 不高于 50 ms 为参考，分别记录达成情况。报告均值、P50/P95/P99、最大值、更新间隔、额外丢帧及错误计数，并在同一检测质量和采集策略下比较。

3.3.5 FPGA 实现指标

FPGA 资源统计以 PG2L100H-6IFBG676 为对象，分列 CPU、Cache、NPU、Video、DDR 及互连的 LUT、FF、DRM 和 APM 占用。时序记录主逻辑、DDR 用户和输入输出像素域的约束、实际工作频率、建立保持裕量和关键路径。Fmax 记录对应的配置和时序分析方法，实际闭合频率与分析上限分别列出。

资源使用率以 66,600 LUT、133,200 FF、155 个 36 Kbit DRM 和 240 个 APM 为器件基数。NPU 乘加、量化和视频 Alpha 混合的 APM 用途分别计数；综合报告和布局布线报告注明生成阶段。

第四部分 总 结

4.1 可扩展之处

系统保留同帧原图、人脸框、五点及头像编号接口，可在检测后增加裁脸、缩放和表情分类。扩展时在帧记录中增加消费者阶段，延长原图生命周期，并把表情结果映射为头像编号。新增模型的推理、数据搬运和后处理开销需要计入整机帧率与延迟分析。

CPU 可进一步研究前端供数、预测策略和流水线并行度，NPU 可扩展算子及并行规模。扩展工作沿用公共地址、寄存器和版本管理方式，分别检查指令语义、整数结果、共享存储顺序及 FPGA 时序资源。

4.2 工程取舍与心得体会

六级单发射结构将取指请求与返回分开，在保持顺序提交的同时组织 Cache 和总线等待。CPU 承担控制与有限候选处理，NPU 承担卷积及张量运算，Video 承担逐像素处理，使软件调度粒度与硬件数据粒度对应。深度卷积采用通道独立的供数和累加方式，避免将稀疏连接转换成低利用率的密集计算。

共享 DDR 通过不可缓存区、完成响应和帧所有权管理保证数据交接。三种全屏模式共用原分辨率底图和同帧结果，逐像素合成直接输出，省去整帧叠加图的写回；三帧槽允许采集、计算和显示重叠，旧引用退休并完成释放确认后再复用缓存。性能分析从 CPU 退休事件、NPU 逐层周期延伸到实际显示事件，使计算开销、排队和扫描相位能够分别定位。

模型导出、整数参考和视频前端采用相同的预处理与舍入规则，减少跨模块数值差异。统一配置和版本标识将硬件、固件、模型及素材组织为可复现的运行组合，便于比较配置变化及定位接口问题。

第五部分 参考文献

附 录

A 仿真环境与验证方法

A.1 工具配置

A.2 测试平台

A.3 判定标准

B CPU 仿真验证

B.1 指令集验证

测试配置为 `rv32im_zicsr`。`riscv-tests` 功能测试结果为 **50/50 PASS**，ACT4 架构一致性测试结果为 **53/53 PASS**。支撑材料保存于`docs/verification/cpu/isa_support/`。

B.1.1 riscv-tests 逐项结果

表B-1列出 RV32I 的 40 项、RV32M 的 8 项及机器 CSR 的 2 项测试。

表 B-1 riscv-tests 逐项用例清单（50/50 PASS）

序号	配置	指令集	用例	状态
1	`rv32im_zicsr`	RV32I	`rv32ui-p-simple`	PASS
2	`rv32im_zicsr`	RV32I	`rv32ui-p-add`	PASS
3	`rv32im_zicsr`	RV32I	`rv32ui-p-addi`	PASS
4	`rv32im_zicsr`	RV32I	`rv32ui-p-and`	PASS
5	`rv32im_zicsr`	RV32I	`rv32ui-p-andi`	PASS
6	`rv32im_zicsr`	RV32I	`rv32ui-p-auipc`	PASS
7	`rv32im_zicsr`	RV32I	`rv32ui-p-beq`	PASS
8	`rv32im_zicsr`	RV32I	`rv32ui-p-bge`	PASS
9	`rv32im_zicsr`	RV32I	`rv32ui-p-bgeu`	PASS
10	`rv32im_zicsr`	RV32I	`rv32ui-p-blt`	PASS
11	`rv32im_zicsr`	RV32I	`rv32ui-p-bltu`	PASS
12	`rv32im_zicsr`	RV32I	`rv32ui-p-bne`	PASS
13	`rv32im_zicsr`	RV32I	`rv32ui-p-jal`	PASS
14	`rv32im_zicsr`	RV32I	`rv32ui-p-jalr`	PASS
15	`rv32im_zicsr`	RV32I	`rv32ui-p-lb`	PASS

表 B-1 riscv-tests 逐项用例清单（50/50 PASS）（续）

序号	配置	指令集	用例	状态
16	rv32im_zicsr	RV32I	rv32ui-p-lbu	PASS
17	rv32im_zicsr	RV32I	rv32ui-p-ld_st	PASS
18	rv32im_zicsr	RV32I	rv32ui-p-lh	PASS
19	rv32im_zicsr	RV32I	rv32ui-p-lhu	PASS
20	rv32im_zicsr	RV32I	rv32ui-p-lui	PASS
21	rv32im_zicsr	RV32I	rv32ui-p-lw	PASS
22	rv32im_zicsr	RV32I	rv32ui-p-or	PASS
23	rv32im_zicsr	RV32I	rv32ui-p-ori	PASS
24	rv32im_zicsr	RV32I	rv32ui-p-sb	PASS
25	rv32im_zicsr	RV32I	rv32ui-p-sh	PASS
26	rv32im_zicsr	RV32I	rv32ui-p-sll	PASS
27	rv32im_zicsr	RV32I	rv32ui-p-slli	PASS
28	rv32im_zicsr	RV32I	rv32ui-p-slt	PASS
29	rv32im_zicsr	RV32I	rv32ui-p-slti	PASS
30	rv32im_zicsr	RV32I	rv32ui-p-sltiu	PASS
31	rv32im_zicsr	RV32I	rv32ui-p-sltu	PASS
32	rv32im_zicsr	RV32I	rv32ui-p-sra	PASS
33	rv32im_zicsr	RV32I	rv32ui-p-srai	PASS
34	rv32im_zicsr	RV32I	rv32ui-p-srl	PASS
35	rv32im_zicsr	RV32I	rv32ui-p-srli	PASS
36	rv32im_zicsr	RV32I	rv32ui-p-st_ld	PASS
37	rv32im_zicsr	RV32I	rv32ui-p-sub	PASS
38	rv32im_zicsr	RV32I	rv32ui-p-sw	PASS
39	rv32im_zicsr	RV32I	rv32ui-p-xor	PASS
40	rv32im_zicsr	RV32I	rv32ui-p-xori	PASS
41	rv32im_zicsr	RV32M	rv32um-p-div	PASS
42	rv32im_zicsr	RV32M	rv32um-p-divu	PASS
43	rv32im_zicsr	RV32M	rv32um-p-mul	PASS
44	rv32im_zicsr	RV32M	rv32um-p-mulh	PASS
45	rv32im_zicsr	RV32M	rv32um-p-mulhsu	PASS
46	rv32im_zicsr	RV32M	rv32um-p-mulhu	PASS
47	rv32im_zicsr	RV32M	rv32um-p-rem	PASS
48	rv32im_zicsr	RV32M	rv32um-p-remu	PASS
49	rv32im_zicsr	CSR	rv32mi-p-csr	PASS
50	rv32im_zicsr	CSR	rv32mi-p-mcsr	PASS

B.1.2 ACT4 逐项结果

表B-2列出 I 类 39 项、M 类 8 项及 Zicsr 类 6 项测试。

表 B-2 ACT4 逐项用例清单（53/53 PASS）

序号	配置	指令集	用例	状态
1	rv32im_zicsr	I	I-add-00	PASS
2	rv32im_zicsr	I	I-addi-00	PASS
3	rv32im_zicsr	I	I-and-00	PASS
4	rv32im_zicsr	I	I-andi-00	PASS
5	rv32im_zicsr	I	I-auipc-00	PASS
6	rv32im_zicsr	I	I-beq-00	PASS
7	rv32im_zicsr	I	I-bge-00	PASS
8	rv32im_zicsr	I	I-bgeu-00	PASS
9	rv32im_zicsr	I	I-blt-00	PASS
10	rv32im_zicsr	I	I-bltu-00	PASS
11	rv32im_zicsr	I	I-bne-00	PASS
12	rv32im_zicsr	I	I-fence-00	PASS
13	rv32im_zicsr	I	I-jal-00	PASS
14	rv32im_zicsr	I	I-jalr-00	PASS
15	rv32im_zicsr	I	I-lb-00	PASS
16	rv32im_zicsr	I	I-lbu-00	PASS
17	rv32im_zicsr	I	I-lh-00	PASS
18	rv32im_zicsr	I	I-lhu-00	PASS
19	rv32im_zicsr	I	I-lui-00	PASS
20	rv32im_zicsr	I	I-lw-00	PASS
21	rv32im_zicsr	I	I-nop-00	PASS
22	rv32im_zicsr	I	I-or-00	PASS
23	rv32im_zicsr	I	I-ori-00	PASS
24	rv32im_zicsr	I	I-sb-00	PASS
25	rv32im_zicsr	I	I-sh-00	PASS
26	rv32im_zicsr	I	I-sll-00	PASS
27	rv32im_zicsr	I	I-slli-00	PASS
28	rv32im_zicsr	I	I-slt-00	PASS
29	rv32im_zicsr	I	I-slti-00	PASS
30	rv32im_zicsr	I	I-sltiu-00	PASS
31	rv32im_zicsr	I	I-sltu-00	PASS
32	rv32im_zicsr	I	I-sra-00	PASS
33	rv32im_zicsr	I	I-srai-00	PASS

表 B-2 ACT4 逐项用例清单（53/53 PASS）（续）

序号	配置	指令集	用例	状态
34	rv32im_zicsr	I	I-srl-00	PASS
35	rv32im_zicsr	I	I-srli-00	PASS
36	rv32im_zicsr	I	I-sub-00	PASS
37	rv32im_zicsr	I	I-sw-00	PASS
38	rv32im_zicsr	I	I-xor-00	PASS
39	rv32im_zicsr	I	I-xori-00	PASS
40	rv32im_zicsr	M	M-div-00	PASS
41	rv32im_zicsr	M	M-divu-00	PASS
42	rv32im_zicsr	M	M-mul-00	PASS
43	rv32im_zicsr	M	M-mulh-00	PASS
44	rv32im_zicsr	M	M-mulhsu-00	PASS
45	rv32im_zicsr	M	M-mulhu-00	PASS
46	rv32im_zicsr	M	M-rem-00	PASS
47	rv32im_zicsr	M	M-remu-00	PASS
48	rv32im_zicsr	Zicsr	Zicsr-csrrc-00	PASS
49	rv32im_zicsr	Zicsr	Zicsr-csrrci-00	PASS
50	rv32im_zicsr	Zicsr	Zicsr-csrrs-00	PASS
51	rv32im_zicsr	Zicsr	Zicsr-csrrsi-00	PASS
52	rv32im_zicsr	Zicsr	Zicsr-csrrw-00	PASS
53	rv32im_zicsr	Zicsr	Zicsr-csrrwi-00	PASS

B.2 单元级仿真验证

测试台分别实例化 ALU、乘法器、除法器、CSR 和寄存器堆，在 ModelSim 中按组合求值或时钟边沿检查运算结果与控制行为。表B-3汇总自检结果，图B-1至图B-5给出关键场景的原生波形。分别向五类测试的被测输出注入错误后，自检均报告数值不符并拒绝通过。测试台、运行日志、WLF/VCD 和截图登记保存于`docs/verification/cpu/unit_support/`。

表 B-3 CPU 单元级功能仿真结果

单元	测试场景	检查统计	结果
乘法器	四种乘法、符号与边界输入、无效气泡、暂停保持、低位部分积及早期有效条件	2730 次有效结果检查，904 个暂停周期	PASS
除法器	四种除法/求余、除零、有符号溢出、重复操作数、忙时重启及复位	308 次算术操作，312 次完成检查，16 次快速完成	PASS

表 B-3 CPU 单元级功能仿真结果（续）

单元	测试场景	检查统计	结果
CSR	六种读改写形式、只读访问、EPC 对齐、非法指令与 ECALL 异常、MRET 返回	21 次 CSR 访问，91 项检查，2 次异常，1 次返回	PASS
ALU	加减、逻辑、无符号/有符号比较、逻辑/算术移位、边界值及伪随机组合	17804 次结果检查，384 组移位扫描	PASS
寄存器堆	同步写入、异步双读、x0 保持零、复位屏蔽和单发射次读口约束	2476 次读值检查，1006 次写入，6 次扫描	PASS

B.2.1 乘法器

测试台以 66 bit 有符号整数乘法计算参考值，按 `MUL`、`MULH`、`MULHSU` 和 `MULHU` 的符号规则选择低 32 位或高 32 位结果。输入包括零、`0xFFFFFFFF`、`0x80000000`、`0x7FFFFFFF` 及伪随机数，并交错插入无效指令和 `hold`。图B-1中，`0x80000000` × `0xFFFFFFFF` 的低位结果为 `0x80000000`；`hold` 拉高时，结果与低位部分积保持，解除暂停后继续推进。`valid_out` 为测试台按相同暂停条件生成的参考有效标志，用于限定结果比较时刻。自检完成 2730 次有效结果检查，并检查 904 个暂停周期内的保持行为。

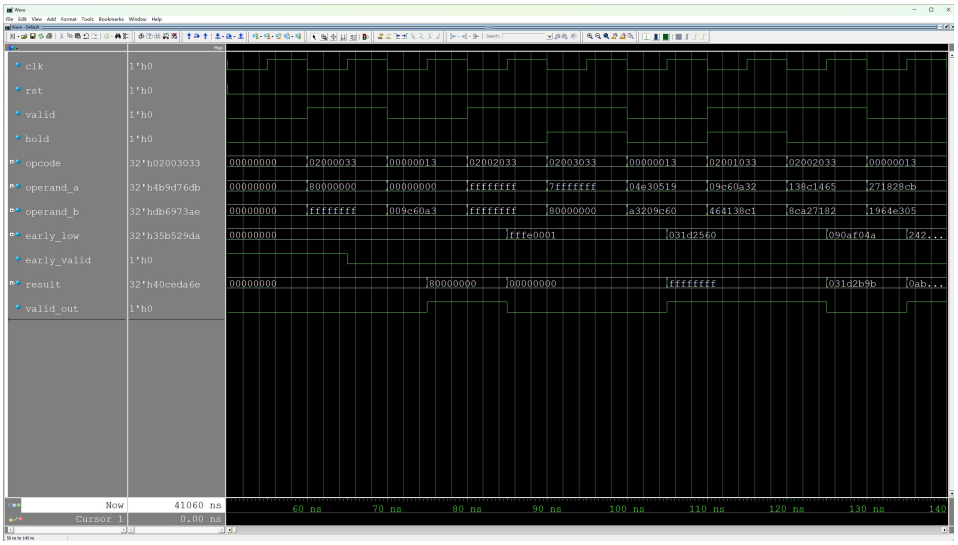


图 B-1 乘法器的运算结果、无效气泡与暂停保持波形

B.2.2 除法器

测试台按 RISC-V 除法规则独立计算商和余数，覆盖 `DIV`、`DIVU`、`REM` 和 `REMU`。除零时商为 `0xFFFFFFFF`、余数为被除数；有符号最小值除以 −1 时，商为 `0x80000000`、余数为零。完成检查同时比较参考除法单元的结果，并核对当前单元相对参考单元延后一周期的完成时序。

图B-2给出 $-21 \div 4$：输入为 0xFFFFFFEB 和 0x00000004，输出商为 0xFFFFFFFB（-5），符合向零截断规则。首次操作经过迭代完成，随后再次提交相同操作数时利用保存的结果快速完成；两次 candidate_valid 均比 golden_valid 晚一个时钟周期。算术测试完成 308 次操作；随后 1000 周期输入流继续覆盖忙时重启、连续有效输入和复位，累计检查 312 次完成事件。

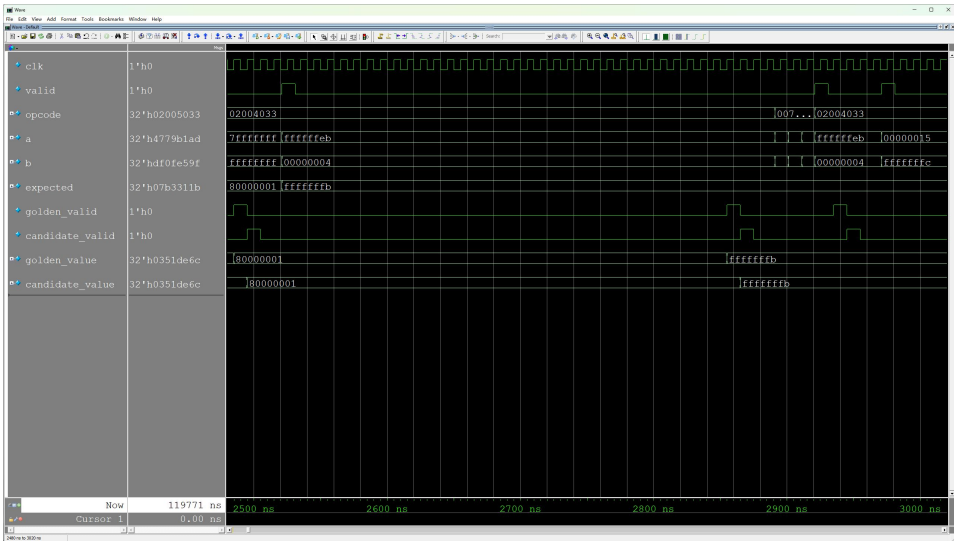


图 B-2 有符号除法结果、完成时序及重复操作数复用波形

B.2.3 CSR

测试台分别驱动指令输入与后续提交接口，检查 CSR 旧值、读改写结果和提交后的寄存器状态。图B-3以 mscratch（地址 0x340）展示六种 CSR 指令：先写入 0x12345678，再用 0x00F0000F 置位得到 0x12F4567F、清位得到 0x12045670；立即数交换、置位和清位依次得到 0x1F、0x1F 和 0x1C。read_valid 对应指令读值，wb_write 对应后续提交写入；CSRRS 以 x0 为源寄存器时只读取旧值。

异常测试检查 mepc 低两位清零、非法指令的 mcause=2 与 mtval、ECALL 的 mcause=11，以及异常入口和 MRET 的重定向地址与状态恢复。自检完成 21 次 CSR 访问和 91 项检查，覆盖两次异常与一次返回。

B.2.4 ALU

ALU 测试台对每个输入组合用独立的 RV32 参考函数计算结果，覆盖加法和减法的 32 位回绕、按位逻辑、无符号与有符号小于比较，以及逻辑左移、逻辑右移和算术右移。边界集合包含零、全一、最高位、最高位减一、交替位和 0x80000001；移位测试逐项检查 0 至 31 的移位量，并令移位操作数高位非零以检查只使用低 5 位。图B-4中，op 依次选择加法、移位、比较和逻辑运算，expected 与 result 在每个检查窗口相同，check_ok 标记该组合已经完成比较。测试还对 1024 组伪随机操作数执行全部 16 种控

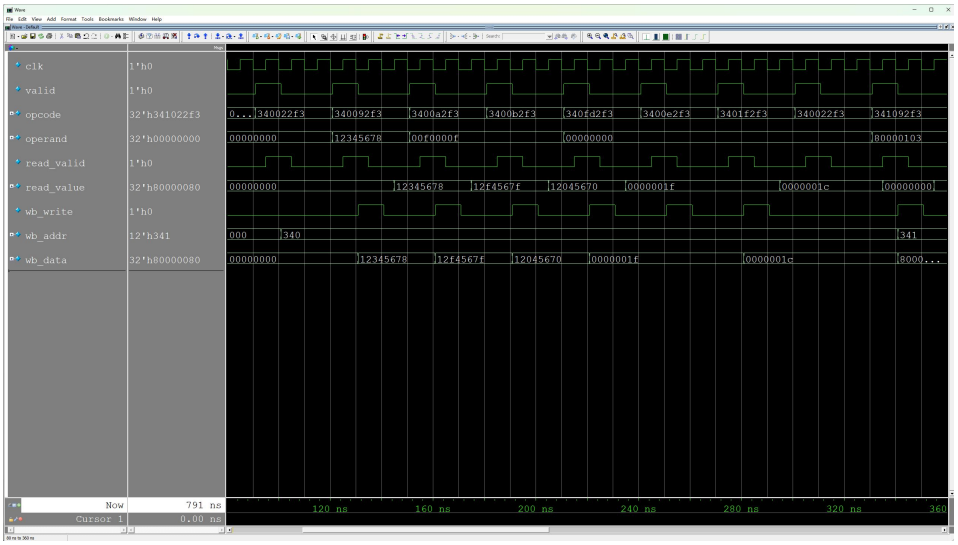


图 B-3 CSR 六种读改写形式的读值与提交写入波形

制编码，并覆盖默认操作的直通结果。

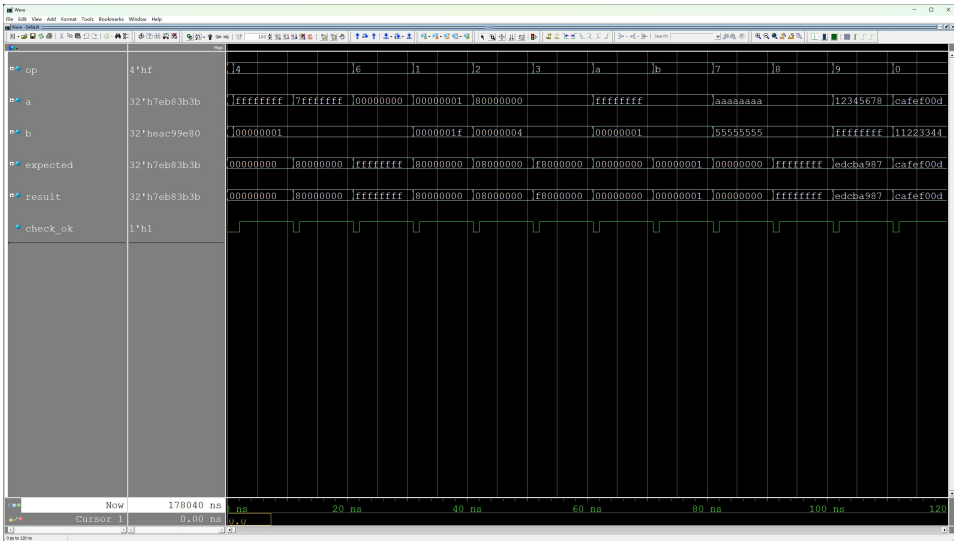


图 B-4 ALU 算术、移位、比较和逻辑运算结果波形

B.2.5 寄存器堆

寄存器堆测试台以数组模型作为参考，按时钟上升沿更新写入，并在地址变化后立即比较两个异步读端口。测试首先验证复位后的全部地址读为零，再逐字尝试写入 32 个寄存器；随后以伪随机地址和数据执行 1000 轮读写，并检查 x0 写入被丢弃。单发射配置的次写口置零，测试台同时检查次读口输出为零。图B-5显示 `rd`=5、6 的写入在时钟边沿后分别出现在 `value_a` 和 `value_b`，地址改变无需额外时钟即可读出；对 x0 的写入没有改变零值。测试同时确认复位在时钟边沿后使全部地址读零，未在该边沿前提前改变读出数据；重写一个寄存器后，其余地址仍保持零值。

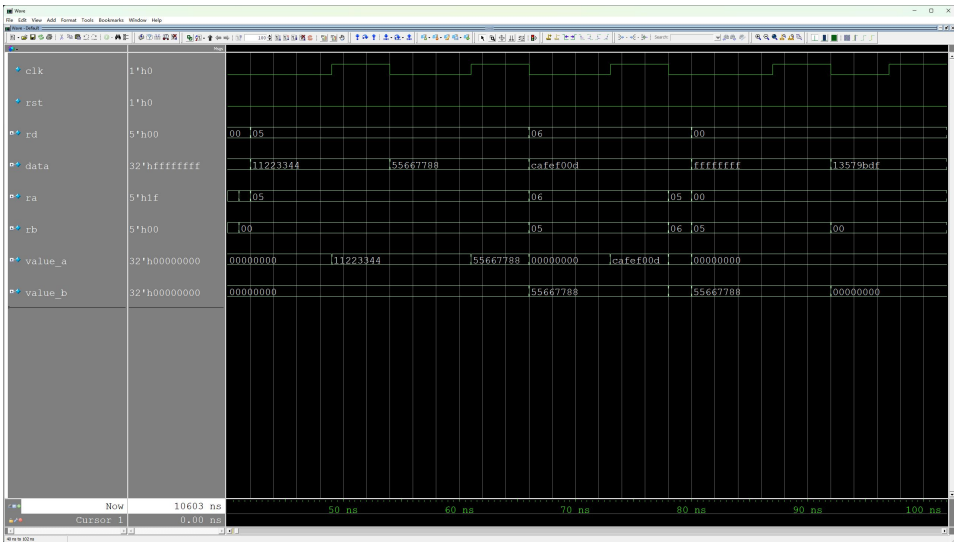


图 B-5 寄存器堆同步写入、异步读出及 x0 保持波形

B.3 模块级仿真验证

模块级测试覆盖前端重定向、发射与旁路、LSU 和 I/D Cache，自检结果见表B-4。前端测试从取指请求、返回和消费接口注入延迟，发射测试通过完整核的取指和数据接口运行固定指令序列；LSU 与 Cache 测试分别连接各自的请求和响应端口。结果、自检日志、WLF/VCD 和截图集中保存于`docs/verification/cpu/module_support/`。

表 B-4 CPU 模块级功能仿真结果

模块	场景	检查统计	结果
前端重定向	注册重定向、迟到取指响应、消费背压和连续重定向	4 次重定向，12 项交付，4 项迟到响应丢弃	PASS
发射/旁路	连续 ALU、加载、低位乘法、高位乘法和同目的寄存器写回	16 次发射，16 次退休，26 项操作数检查，38 周期	PASS
LSU	加载扩展、写掩码、请求保持、非对齐和总线错误	18 次访问，220/220 项检查	PASS
I/D Cache	命中、填充、两路替换、错误填充、维护及并发	1297/1297 项检查	PASS
数据 Cache 写回	脏行清理、失败重试、替换、刷新与共享区旁路	1355/1355 项检查	PASS
数据 Cache 响应重叠	连续命中、响应标签、年轻请求及错误隔离	142/142 项检查，12 次连续读取	PASS

B.3.1 前端重定向

测试台关闭动态分支预测，使用固定 32 bit 指令返回，分别设置取指返回延迟、消费暂停和分支目标。重定向请求在时钟边沿登记，旧取指响应在返回时丢弃；恢复后只允许目标地址开始的低字和高字进入消费端。图B-6中，`branch` 在目标 `0x80001000` 和 `0x80002000` 处触发，`pending` 覆盖等待窗口，`response` 晚于请求返回，`dropped` 累计迟到响应；`pc0` 保持目标序列，未交付旧地址指令。测试完成 4 次重定向、12 项正确交付和 4 项迟到响应丢弃。

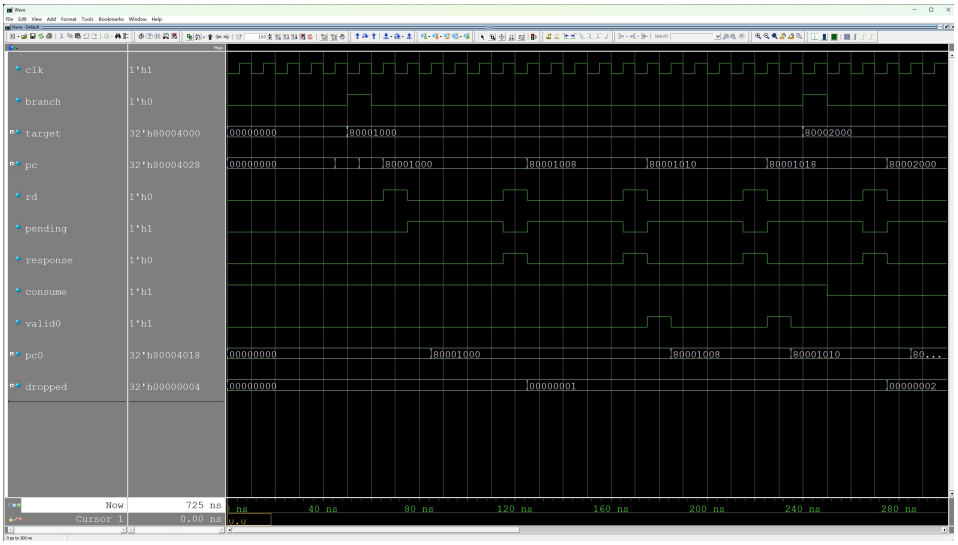


图 B-6 前端注册重定向、迟到响应丢弃和目标指令交付波形

B.3.2 发射与旁路

测试台通过真实指令取指返回运行 16 条顺序指令，包含连续加法、一次加载、低位乘法、高位乘法和两次写入同一目的寄存器。数据端口在接纳请求前插入等待，并推迟加载返回；测试检查相关指令在数据可用后发射，高位乘法保持互锁，满足条件的低位乘法使用 E1 早期旁路。连续两条指令写入同一目的寄存器时，下一条相关指令必须读取较新结果。图B-7中，`d_rd`、`d_accept` 和 `d_ack` 形成一次完整加载事务；`load_e1/load_e2` 和 `pipe_stall` 标出等待区间，`mul_e1/early_mul` 标出低位乘法旁路，`wb_valid/wb_pc` 保持顺序退休。自检在 38 个周期内完成 16 次发射和 16 次退休，检查 26 个操作数，记录 10 个加载相关等待周期和 1 个高位乘法等待周期；ALU E1 旁路、低位乘法早旁路和最新结果优先级各命中一次。

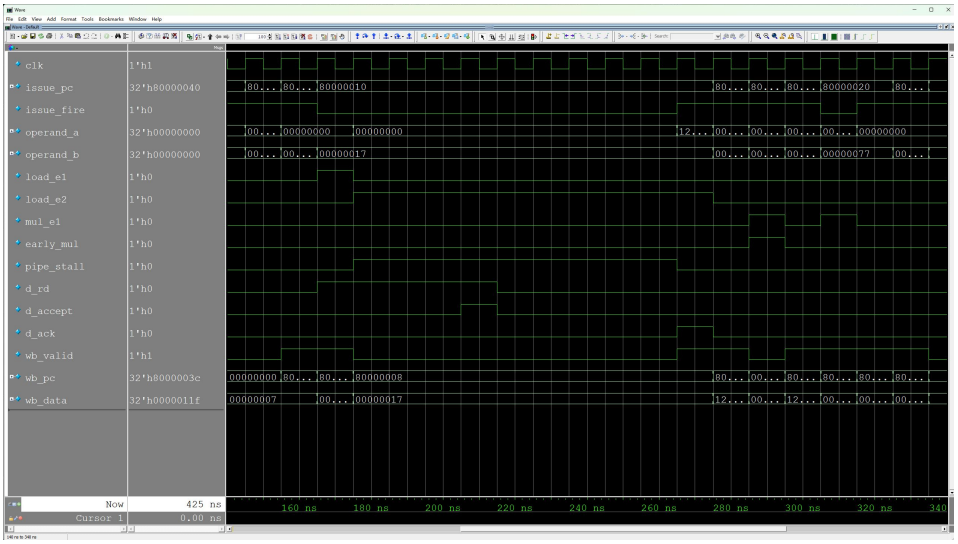


图 B-7 真实取指序列中的加载等待、乘法旁路和顺序写回波形

B.3.3 LSU 端口与异常响应

LSU 测试台连接公开的指令、操作数、请求和写回端口，按 RISC-V 访存规则给出预期数据、写掩码和异常编码。测试覆盖字节、半字和字访问，覆盖未接纳时保持地址和写掩码、加载符号扩展、存储数据排列，以及非对齐和总线错误的精确响应。18 次访问中，11 次加载和 4 次存储形成总线事务，其中 2 次返回总线错误；另有 3 次非对齐访问在发出总线请求前产生异常。测试累计检查 45 个未接纳保持周期，完成 220/220 项检查。图B-8依次给出字节、半字和字存储：写掩码为 `8`、`C` 和 `F`，对应数据为 `DD000000`、`CCDD0000` 和 `AABBCCDD`。`accept` 之前请求字段保持不变；后续非对齐访问仅产生 `wb_valid`，没有读写请求。

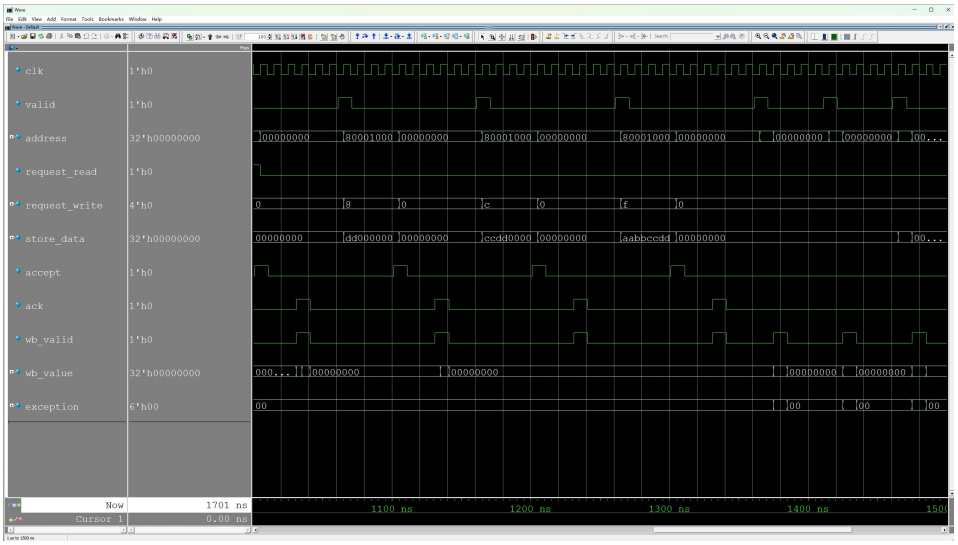


图 B-8 LSU 请求保持、字节写掩码和写回完成波形

B.3.4 I/D Cache 命中、填充与维护

Cache 测试台使用独立的字节寻址内存模型，分别检查指令 Cache 填充和命中、数据 Cache 读写、行替换、写回、失效、刷新和错误重试。组合测试完成 1297/1297 项检查，产生 25 次指令下层读、27 次数据下层读和 25 次下层写，记录 2 次指令命中、2 次数据命中及 35 个 I/D 同时在途周期。图B-9中，`arvalid` 发起八拍填充，`rvalid/rlast` 返回整行数据，第二次相同地址直接由 `i_valid` 完成。

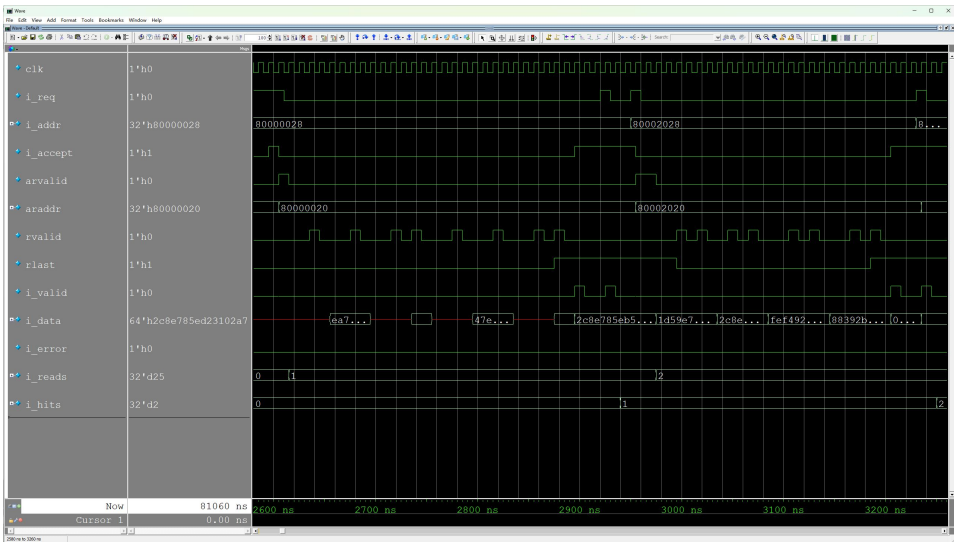


图 B-9 指令 Cache 填充、返回和命中波形

B.3.5 数据 Cache 写回与维护

写回测试完成 1355/1355 项检查、35 次下层读和 184 次下层写，覆盖全部 15 种非零字节掩码、脏行清理、替换前写回、整 Cache 刷新和共享区旁路。测试逐一在八

个写回字上注入下层错误，检查失败后仍能读取脏数据，重新清理时完整写回八个字；填充错误也逐拍注入，失败行不得变为有效行。图B-10中，clean 启动行清理，writes 由 0 递增至 8，ack 在最后一次下层写响应后给出；随后再次读取和清理没有新增下层读写。

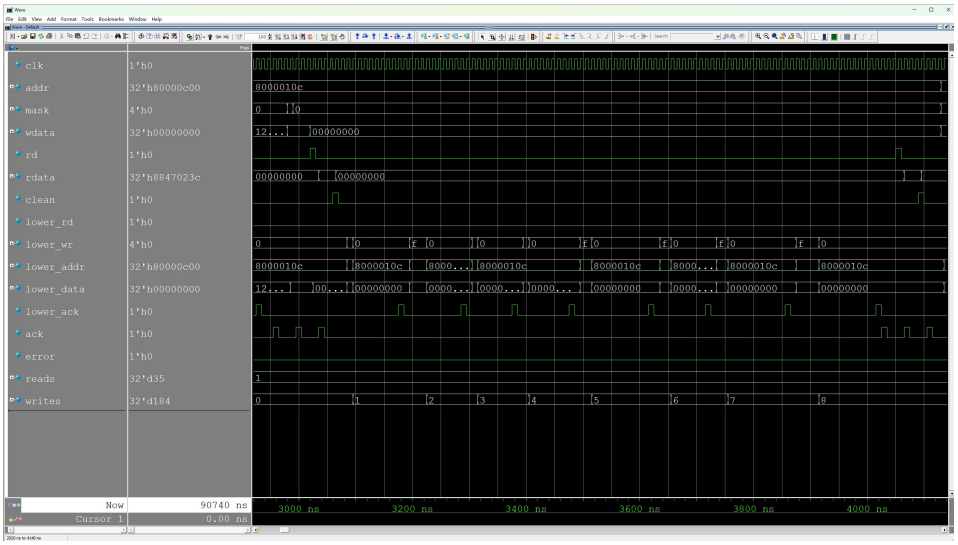


图 B-10 数据 Cache 脏行写回和清理完成波形

B.3.6 数据 Cache 响应重叠

重叠测试完成 142/142 项检查、11 次下层读和 25 次下层写，覆盖命中读取响应与下一请求重叠、标签顺序、跨组跨标记访问和错误响应隔离。图B-11在 score_active 期间连续发出 12 次命中读取，tail 统计接纳数、head 统计返回数，最终均为 12。命中读取每两个时钟周期接纳一次，accept 与旧请求 ack 在完成边沿重叠，resp_tag 按顺序返回，期间 lower_rd 保持为零。测试还分别接入存储、Cache 维护、共享区读取及下层错误，检查错误响应不会接纳年轻请求、存储更新保持串行，脏行替换失败后数据仍可读取。

B.4 系统级仿真验证

B.4.1 存储访问与故障处理

系统级测试使用最小固件连接 Boot ROM、片上 SRAM、UART、私有 Cache、共享区旁路和模型 AXI128 下层。缓存配置下完成启动、私有区读写、共享区读写、FENCE.I 后的代码执行、DDR 就绪等待、下层读错误和平台故障后的本地访问；固件通过 mtvec/mepc/mcause/mtval 检查三次受控异常返回。正例自检完成 1277 项检查，统计 26356 个时钟周期、14 次读事务、35 次写事务和 3 次陷阱，返回码为零。图B-12选取第三次异常：平台故障使 ddr_ready 撤销、fault 锁存；随后访存错误使 trap_count 从 2 变为 3，mcause=5、mtval=0x80000000。异常处理程序将 mepc 从

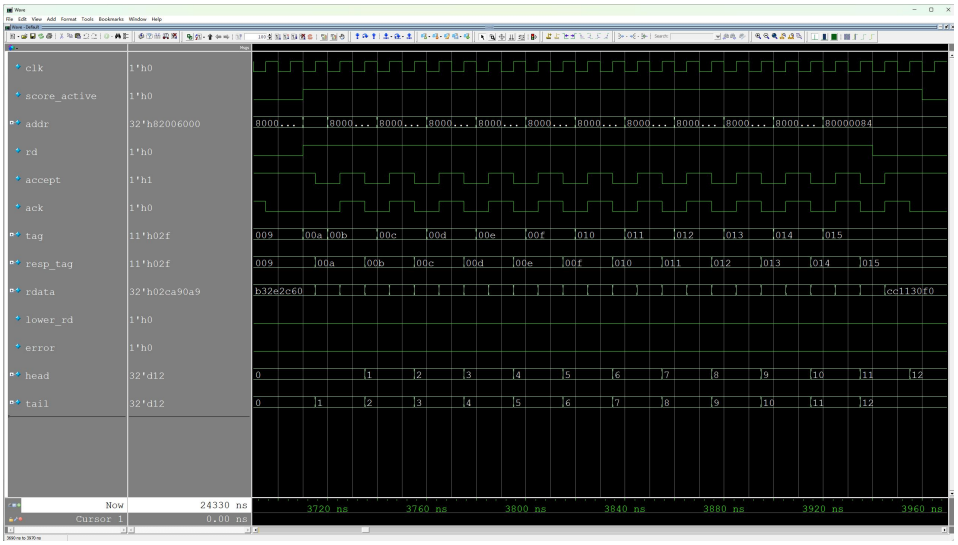


图 B-11 数据 Cache 响应重叠、标签顺序和连续命中波形

`0x000101FC` 加 4 至 `0x00010200` 后执行 `MRET`；程序继续访问片上存储，最终 `done=1` 且 `exit_code=0`。

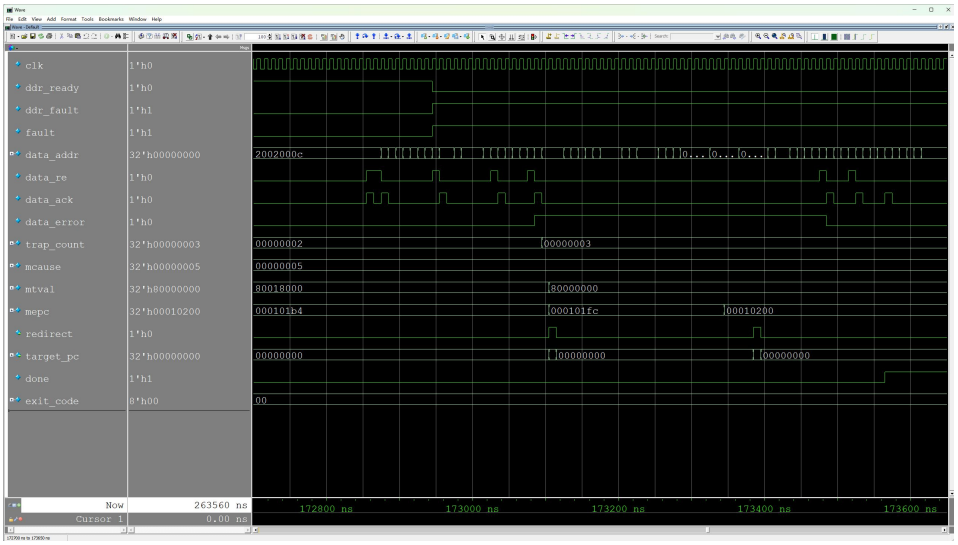


图 B-12 整核固件的平台故障、访存异常与 MRET 返回波形

独立路由测试完成 476 项检查、12 次读事务和 9 次写事务，覆盖片上地址不出 DDR、DDR 未就绪时请求保持、I/D 命中与共享区旁路、部分写掩码、响应所有权、错误锁存及复位恢复。同一固件关闭 Cache 后通过 809 项检查，作为存储属性对照。下层读数据异或一位的错误注入在固件退出检查处触发失败，负例退出码为 1。原始日志、固件、WLF/VCD 与截图保存在`docs/verification/cpu/system_support/`。上述测试使用 ModelSim 和行为级 AXI 内存模型；不代表真实 DDR 控制器、跨时钟域、综合布线或板卡时序结果。

B.4.2 外部中断进入与返回

中断测试采用单发射核、512 项直接映射 BTB、512 项 BHT 和 4 项 RAS，关闭 GShare，连接 128 KiB 行为级指令与数据存储器。固件先检查 8 类同步异常，再设置 `mie.MEIE` 和 `mstatus.MIE`；测试台在触发地址的存储请求被接纳后驱动外部 `irq`。中断处理程序检查机器外部中断原因，清除 `mie`、将处理计数加一，再执行 `MRET`；返回后固件检查计数恰好为一并正常退出。

图B-13中，中断进入时 `mcause=0x8000000B`，`mepc=0x80000150`，`mtval=0`。`mstatus` 由 `0x00000088` 变为 `0x00001880`，对应 MIE 清零、MPIE 置位和机器态保存；`MRET` 后恢复为 `0x00000088`，退休记录从保存的 `mepc` 继续执行。固件完成 3582 个周期、864 次退休和 92 次存储，8 类同步异常与 1 次外部中断的覆盖掩码为 `0x1FF`。

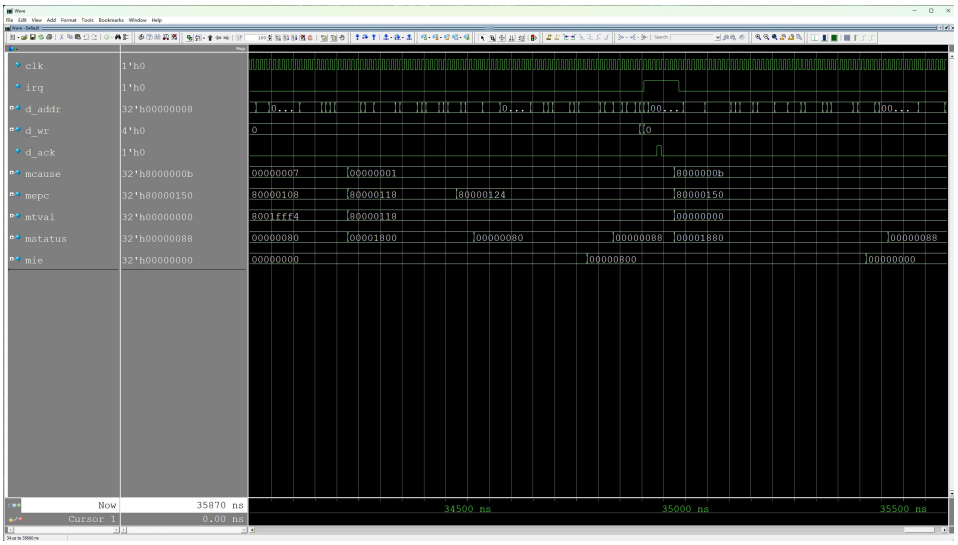


图 B-13 单发射整核的机器外部中断进入、CSR 保存和 MRET 返回波形

B.4.3 Cache 访存与中断交互

整核测试开启 16 KiB 指令 Cache 和 16 KiB 数据 Cache，两者均为两路组相联、32 B 缓存行，数据 Cache 采用写回与写分配。启动程序从片上 SRAM 复制测试代码到私有可缓存区，执行 `FENCE.I` 后转入该区；测试程序与中断处理程序均通过指令 Cache 取指。测试台在数据 Cache 命中返回、缺失填充等待和缺失加载返回三个时刻分别驱动外部中断，结果见表B-5。

表 B-5 Cache 开启时的外部中断场景

场景	中断触发条件	保存的 `mepc`	结果
命中返回	驻留数据加载返回，不产生新的下层填充	`0x80004060`	PASS

表 B-5 Cache 开启时的外部中断场景（续）

场景	中断触发条件	保存的 mepc	结果
填充等待	缺失读事务已接纳，下层尚未返回数据	0x80004094	PASS
缺失返回	新缓存行填充完成，目标加载返回	0x800040C8	PASS

三次中断入口均检查 mcause=0x8000000B、mtval=0、mstatus=0x1880 和 mie=0x800。处理程序清除 mie，保持原 mepc 并执行 MRET；返回后 mstatus 恢复为 0x88，首条退休指令的 PC 等于原 mepc。三个目标加载分别得到 0x13579BDF、0x2468ACE0 和 0x31415926，各加载及其后继指令均只退休一次，软件计数依次为 1、2、3。

图B-14展示填充等待场景：irq 在 114590 ns 置位，随后下层返回缓存行并完成加载，115395 ns 进入中断，115855 ns 执行 MRET。外部中断有效且读事务待完成、下层无有效返回的状态累计持续 66 周期。整次自检完成 11758 周期、1110 次退休和 12655 项检查，包含三次中断进入、三次返回及三次恢复执行。将处理程序中的 mepc 故意加 4 后，返回地址检查检出错误，测试入口返回 1。

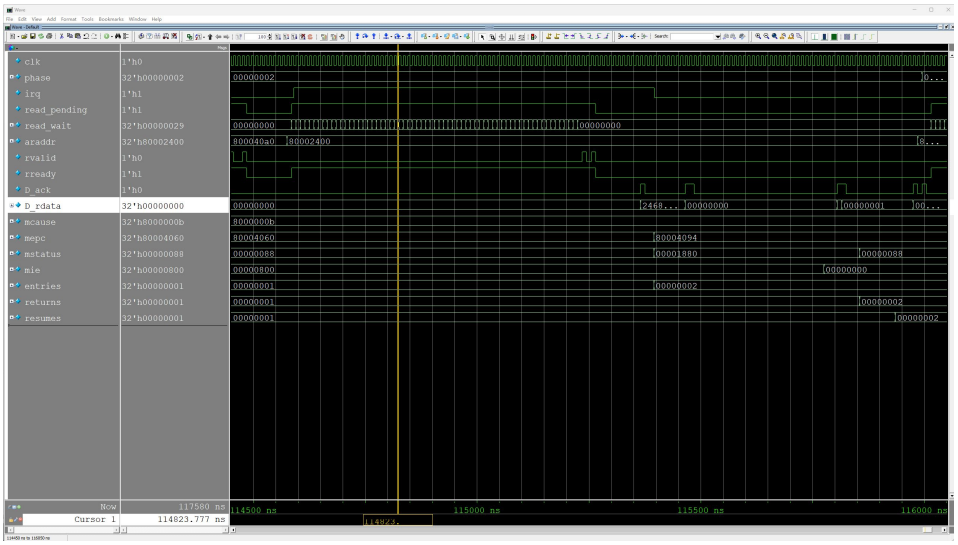


图 B-14 Cache 缺失填充等待期间的中断请求、加载完成与 MRET 返回波形

B.4.4 流水冲刷后的副作用隔离

整核测试通过公开取指和数据端口运行定向固件，分别检查分支误预测重定向与加载访问异常后的年轻指令。三处分支的错误路径分别放置 SRAM 存储、MMIO 写入和 mscratch 写入；四处长延迟错误加载后分别放置这三类写入及一条写同一目的寄存器的加载。测试台在请求接纳前插入背压，将加载返回延迟设为 32 或 64 周期，并以 0xDEAFBEEF 作为错误响应和无效响应周期的数据。表B-6列出七个检查场景。

表 B-6 流水冲刷后的副作用隔离测试

触发条件	年轻指令	检查结果	状态
分支误预测重定向	SRAM 存储	受保护字保持 `0x13579BDF`	PASS
分支误预测重定向	MMIO 写入	仅有一次合法的 `0x42` 写入	PASS
分支误预测重定向	CSR 写入	`mscratch` 保持 `0x12345678`	PASS
加载访问异常	SRAM 存储	无额外存储，受保护字不变	PASS
加载访问异常	MMIO 写入	无额外外设写入	PASS
加载访问异常	CSR 写入	无额外 CSR 写入	PASS
加载访问异常	同目的寄存器加载	年轻请求未接纳；返回后重新加载正确值	PASS

七条禁止执行的指令均被取回，退休流中均未出现；其中四条异常后的年轻指令进入译码。四次异常均保存故障加载的 PC，`mcause=5`、`mtval=0x8001F080`，故障加载不改变目的寄存器。处理程序跳过故障片段后执行 `MRET`，随后四次加载复用同一目的寄存器 `x20`，均返回 `0x11223344`。两组返回延迟分别完成 750 周期、2199 项检查和 1102 周期、3031 项检查，各退休 162 条指令，完整退休序列一致。另设四组错误输入：分别反转三处分支条件及将加载错误响应改报成功，均触发副作用检查失败。

图B-15展示第一次加载访问异常：2285 ns 返回错误响应，随后产生 `squash` 并转入异常入口；`mepc` 保存 `0x800000C0`、`mcause` 置为 5，`mscratch` 始终保持原值。异常返回后，恢复加载使 `recovered` 由 0 变为 1。该场景验证阻塞式 LSU 等待期间禁止接纳年轻请求，以及异常后的恢复执行。本组测试不实例化 Cache，MMIO 由测试台状态模型提供；冻结输入、原始波形、退休流和运行索引见`docs/verification/cpu/system_support/SQUASH_README.md`。

C NPU 仿真验证

C.1 计算正确性

C.2 分块计算

C.3 DMA 传输

C.4 任务控制

C.5 辅助算子

D 系统互连与存储仿真验证

本节记录当前可复现的数字互联证据。测试台均在 Windows 原生 ModelSim 中运行，图中波形是直接打开已保存 WLF 后从 ModelSim Wave 窗口截取的原始界面，不

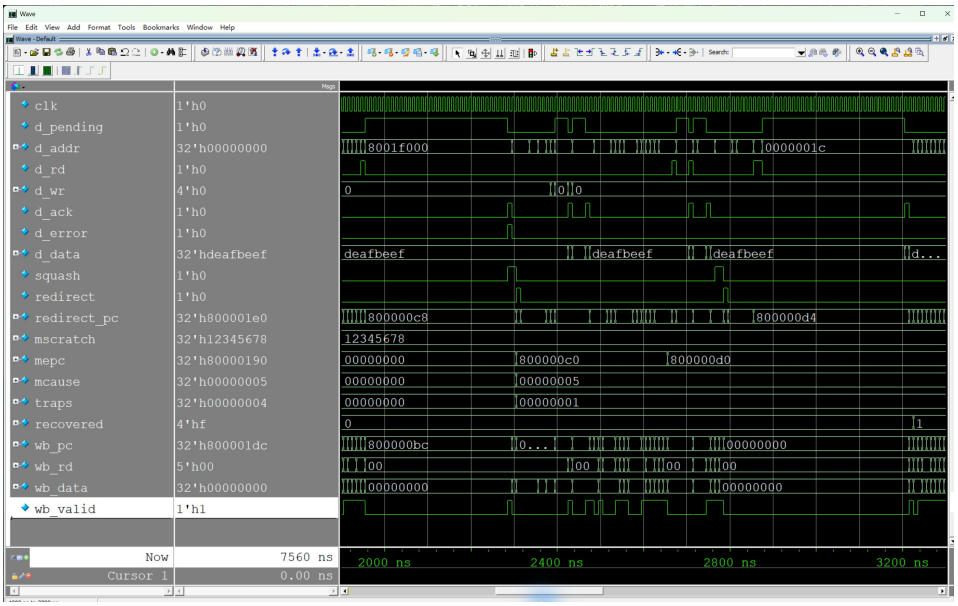


图 B-15 长延迟加载错误、流水冲刷和异常返回后的恢复加载波形

使用脚本重绘。结果索引和源文件哈希见`docs/verification/soc/system_interconnect_storage_20261030.json`。

D.1 CPU 主端控制与外设访问

CPU 到 Video 的固件测试覆盖 6 个正例和 2 个负例。正例检查控制请求握手、三路 AXI 写通道、独立写响应、响应排空、完成事件和中断；负例分别注入错误响应和提前完成条件，均被测试台拒绝。图D-1给出一组原生波形：请求握手后，AW/W/B 通道交错推进，`port_drained` 保持为零直到写响应全部排空，随后才产生 `completion_valid` 和中断。

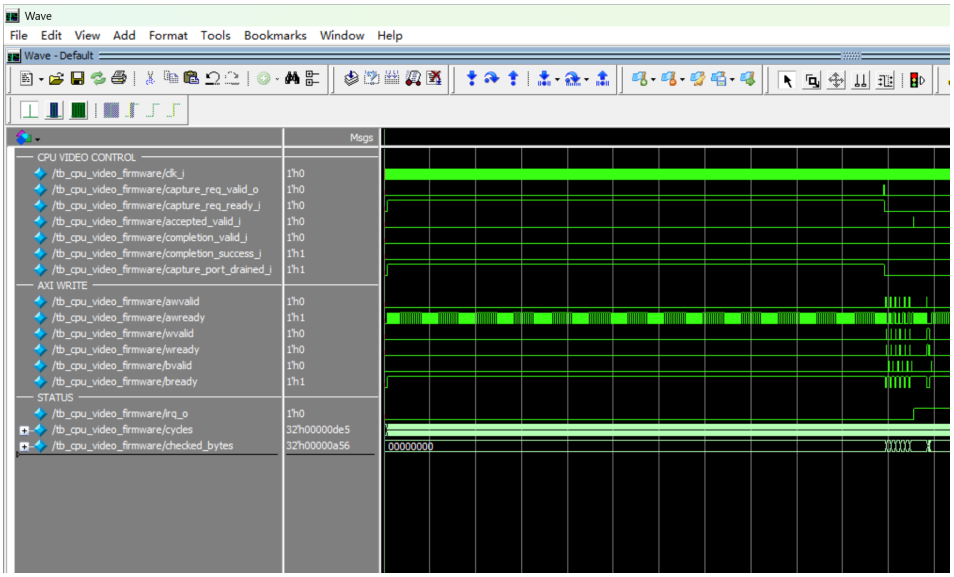


图 D-1 CPU 到 Video 控制请求、AXI 写入、响应排空和完成中断波形

CPU 到 NPU 的操作员测试覆盖 Dense U8 和 DW S8 两类算子，分别检查轮询和中断完成路径，共 4 个正例通过。图D-2展示 AXI-Lite 参数写入与 AXI128 读写启动阶段，图D-3展示尾部写响应、读写握手背压以及事务计数收敛；`DONE` 只在最后一个写响应完成后产生。

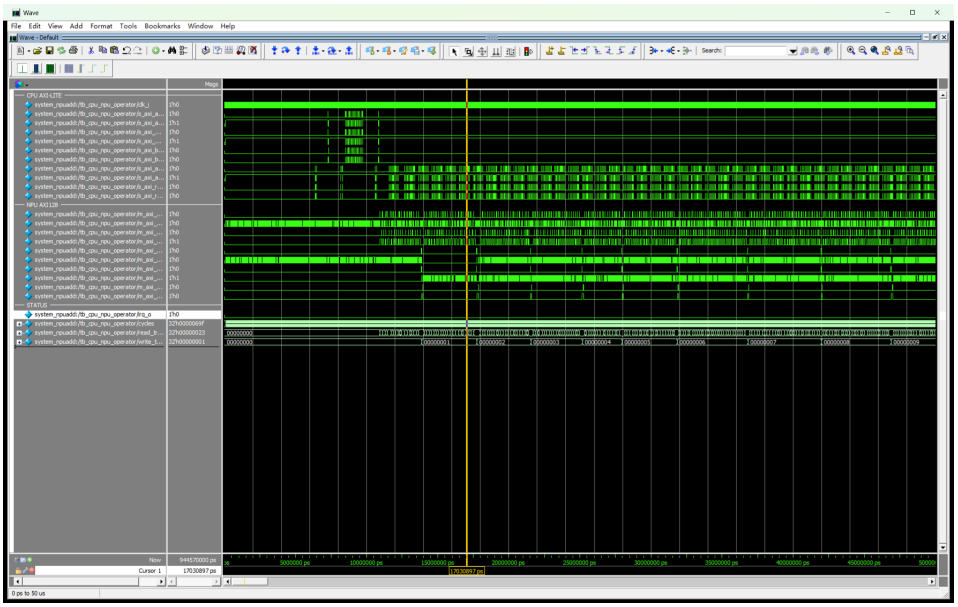


图 D-2 CPU 到 NPU 的 AXI-Lite 配置与 AXI128 初始传输波形

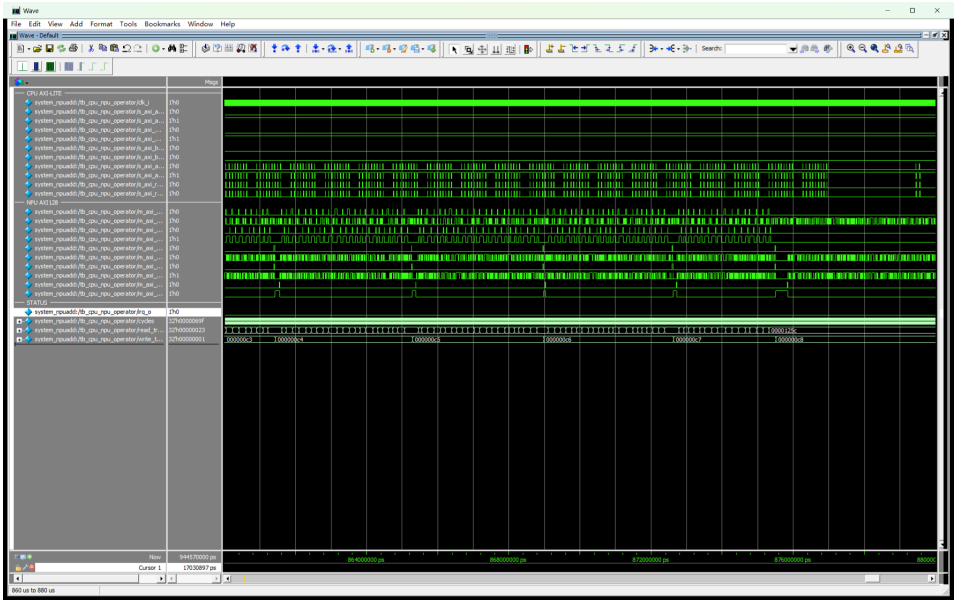


图 D-3 CPU 到 NPU 尾部写响应、背压和事务计数收敛波形

D.2 共享存储与地址译码

Video 共享 DDR 测试覆盖读写竞争、配额、释放、错误保持和背压传播；地址译码回归完成 118 项正常检查。对“错误目标选择”“过早接受 W 通道”和“错误读响

应归属”分别进行变异，三项均以非零诊断退出，说明译码和响应归属检查有效。

表 D-1 系统互联与存储数字仿真结果

场景	覆盖内容	统计	结果
CPU 到 Video	控制握手、三路写入、响应排空、完成和中断	6 个正例、2 个负例	PASS
CPU 到 NPU	AXI-Lite 配置、AXI128 读写、轮询和中断完成	4 个正例	PASS
Video 共享 DDR	读写竞争、配额、释放、错误保持和背压	共享 DDR 回归	PASS
Video 地址译码	目标选择、边界访问和错误响应归属	118 项正常、3 项变异	PASS

上述结果覆盖当前测试台中的数字握手和本地存储模型。正式三主端共享 Fabric、跨主端 Cache 可见性与 `fence`、物理 DDR 跨时钟域、PDS 时序和整机吞吐仍需在对应集成版本中单独验收；本节不将这些未覆盖项目记为通过。

E 图像采集与显示链路仿真验证

本节选取采集重武装、显示槽所有权、共享 DDR 背压和采显联合四类高价值系统场景。测试台在 Windows 原生 ModelSim 中运行，接口响应由可背压数字模型给出，所有比较均由测试台自检完成。完整回归日志、WLF/VCD、输入哈希和原生截图由`docs/verification/video/evidence/video_high_value_waveforms_20261030.json` 统一登记。

E.1 四类系统级场景

表E-1给出短场景波形对应的通过统计。短场景用于保留关键事件的原生波形，完整通过结果仍由同一测试入口的回归日志核对。

表 E-1 Video 链路高价值系统级仿真结果

场景	测试顶层	关键检查	结果
采集重武装	`capture_rearm`	7 次采集成功、7 次重武装，三路写入共 18522 字节；完成响应排空后才发布完成事件	PASS
显示槽所有权	`slot_owner`	128 位状态检查 155 项；提交、实际生效、代次、RETIRING、释放确认和空槽位图顺序一致	PASS
共享 DDR 背压	`memory_scheduler`	12 次读、12 次写；读写最大等待均为 0，调度配额为 4，故障保持为 0	PASS

表 E-1 Video 链路高价值系统级仿真结果（续）

场景	测试顶层	关键检查	结果
采显联合	`shared_memory`	7 个任务、7800 个像素、13 次光栅；读写竞争重叠 11690 拍，读授权 262 次、写授权 441 次，欠载 0	PASS

E.2 采集与槽所有权

采集波形中，`req_valid/req_ready` 的握手先锁存帧号和槽号。随后 linebreak`raw_written_bytes`、`preview_written_bytes` 和 `input_written_bytes` 逐步增加；`port_drained` 置位后才出现 `completion_valid`。成功场景中 `error` 保持为零，重武装次数与成功次数相等。

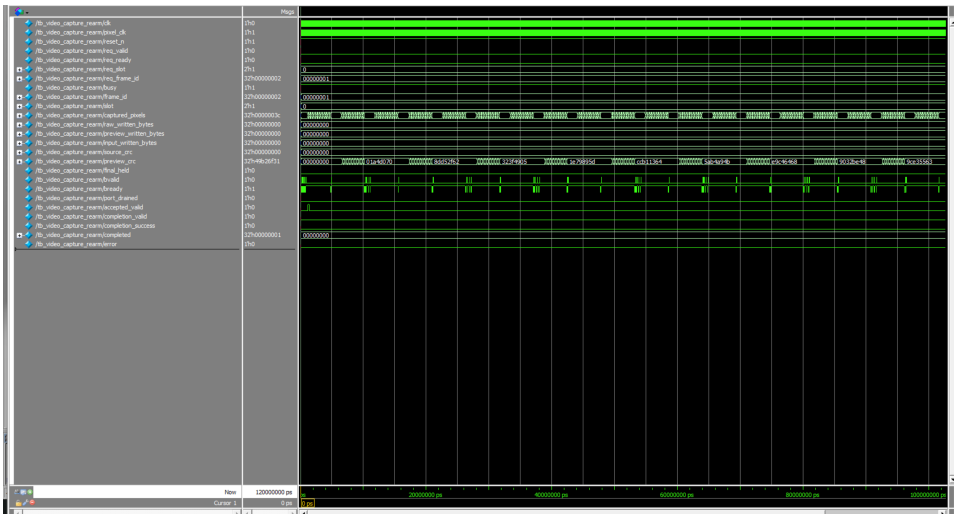


图 E-1 采集请求、三路写入计数、响应排空和完成事件波形

槽所有权场景将提交接收和显示生效分开。`active_slot_o` 及 `active_frame_o` 只在反馈与提交握手完成后更新；旧槽在 `release_valid_o` 与释放确认之前保持占用，`protocol_error_o` 全程为零。

E.3 共享 DDR 背压与采显联合

背压场景同时驱动写地址/写响应和读地址/读数据通道。波形显示读写通道分别被暂停时，仲裁仍保持请求归属，`busy` 和 `port_drained` 在事务排空前不会提前结束；读写计数与测试台的 12 次读写请求一致。

联合场景把采集写、显示读、槽身份和像素输出放在同一时间轴上。linebreak`commit_frame_id`、`release_frame_id` linebreak`display_frame` 保持同一帧身份；`underflows` 保持为零，`seen_commits` 与 `seen_releases` 按提交和释放事件推进，`checked_pixels` 持续增加。2 毫秒短窗口已

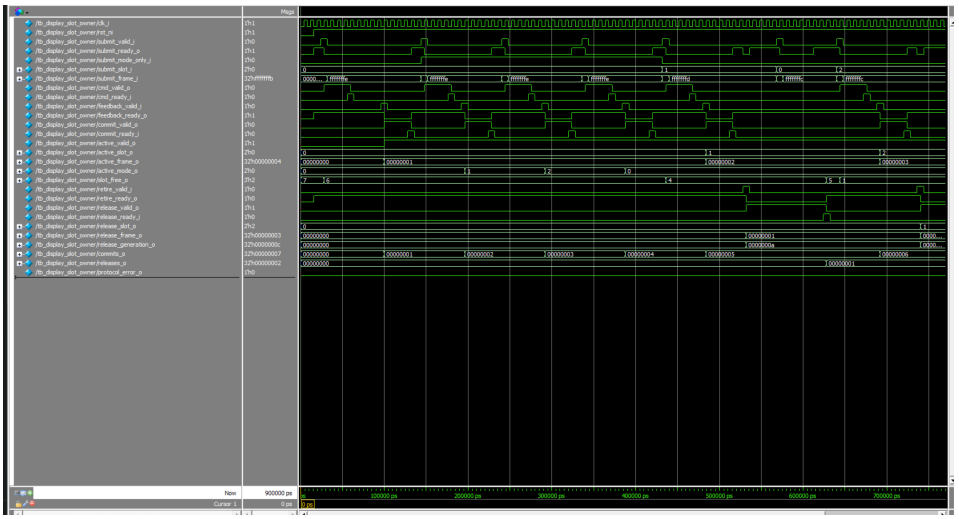


图 E-2 显示槽提交、生效、代次和释放确认波形

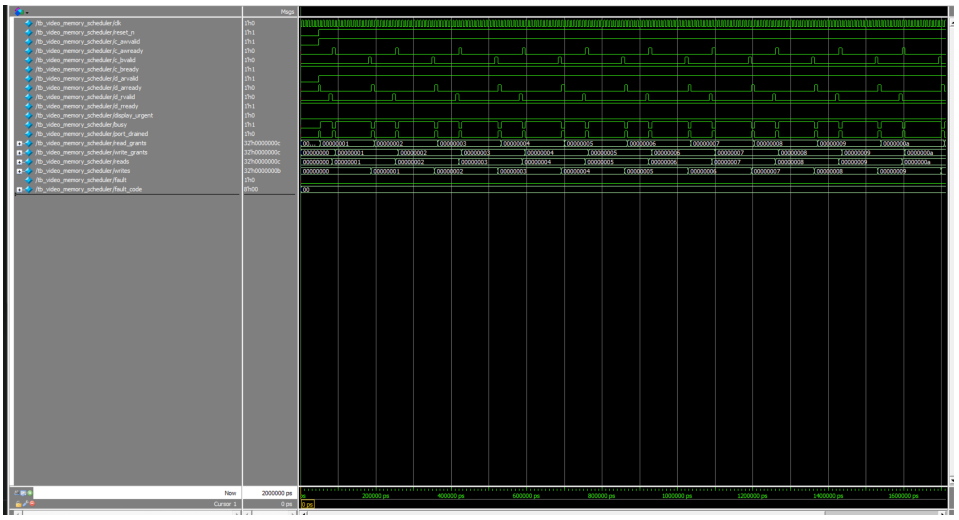


图 E-3 共享 DDR 读写握手、背压、仲裁计数和排空波形

经覆盖四个连续帧的提交/释放和采显重叠，完整 13 次光栅的数值结果见表E-1。

上述场景证明数字模型中的握手、所有权和背压传播；外部 HDMI 电气时序、板级吞吐、PDS 物理时序及真实 NPU 推理仍分别验收。

F 系统联合仿真验证

F.1 完整模型推理

F.2 连续帧处理

F.3 运行配置

F.4 结果核对

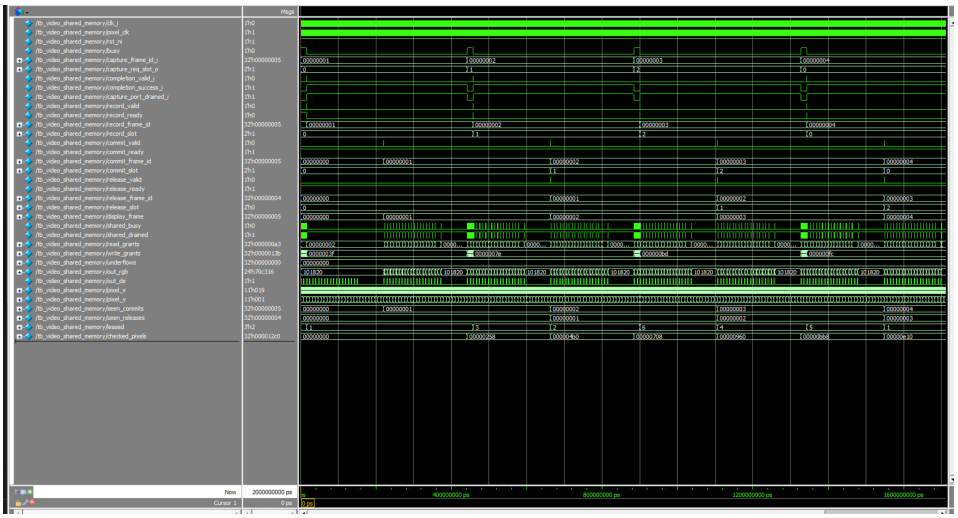


图 E-4 采集、共享 DDR、槽释放和像素输出联合波形